

Framework/Mobile/Fly01Login.prw

1272 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'Protheus.ch'
2 #INCLUDE 'POSCSS.CH'
3 #INCLUDE 'FWMVCDEF.CH'
4 #INCLUDE 'FLY01LOGIN.CH'
5
6 Static oDlg          := Nil //Obj dialog bkg
7 Static lAuthentic    := .F.      //valida se o usuario foi autenticado
8 Static cCryptKey     := "!TOTVS@PDV#MOBILE$" //Chave de criptografia
9
10 //-----
11 /*{Protheus.doc} Fly01Login
12 Classe responsavel pelo Login do Fly01
13
14 @param
15 @return
16 @obs
17 @sample
18 */
19 //-----
20 CLASS Fly01Login FROM LongNameClass
21
22     DATA oOwner
23     DATA oAlertEmailPwd
24     DATA oAlertWrongPwd
25     DATA oWrongAlert
26
27     //Panels
28     DATA oPnHeader
29     DATA oPnLogin
30     DATA oPnAskRegister
31     DATA oPnRegister
32     DATA oPnForgetPassword
33
34     //Panels - dimensoes
35     DATA nHeightLogin
36     DATA nHeightAskRegister
37     DATA nHeightHeader
38     DATA nHeightForgotPsw
39
40
41     //auxiliares de posicionamento
42     DATA nMarginLeft
43     DATA nMarginTop
44
45     //dimensoes gerais (total)
46     DATA nWidthTotal
47     DATA nHeightTotal
48     DATA nTopDefault
49     DATA nLeftDefault
50
51     //dimensoes Campos
52     DATA nWidthUtil

```

```
53  DATA nHeightField
54
55  DATA cGetUser
56  DATA cGetPsw
57  DATA cRepUsrPwd
58  DATA cEmail
59  DATA cNomeLoja
60  DATA oGetEmail
61  DATA oEmail
62  DATA cEndereco
63  DATA cCidade
64
65  DATA oGetLoja
66  DATA oGetPwd
67  DATA oGetRepPwd
68  DATA oGetEnd
69  DATA oListEstado
70  DATA oGetCid
71
72  DATA lAlwaysConnected
73
74  DATA aFontLogin
75  DATA aFontAccount
76
77  DATA lUserCreated
78
79  //METODOS DA CLASSE
80  METHOD New()
81  METHOD SetOwner()
82
83  METHOD Activate()
84  METHOD DeActivate()
85
86  METHOD GetHeader()
87  METHOD GetLogin()
88  METHOD GetForgotPassword()
89  METHOD GetAskRegister()
90  METHOD GetRegister()
91
92  METHOD GetAlertWrongPwd()
93  METHOD GetAlertEmailPwd()
94  METHOD AlertWrong()
95
96
97  METHOD ValidLogin()
98  METHOD ValidEmail()
99  METHOD ValidCadastro()
100 METHOD ValidGetDef()
101 METHOD ValidPsw()
102
103 METHOD HideHeader()
104 METHOD HideLogin()
```

```

105 METHOD HideForgotPassword()
106 METHOD HideAskRegister()
107 METHOD HideRegister()
108 METHOD HideAlertWrongPwd()
109 METHOD HideAlertEmailPwd()
110 METHOD HideAlertWrong()
111
112 METHOD ShowHeader()
113 METHOD ShowLogin()
114 METHOD ShowForgotPassword()
115 METHOD ShowAskRegister()
116 METHOD ShowRegister()
117
118 METHOD ShowAlert()
119
120
121 METHOD Fly01Cadastro()
122
123 END CLASS
124
125
126 //-----
127 /*{Protheus.doc} Fly01Login
128 Metodo construtor Login do Fly01
129
130 @param
131 @return
132 @obs
133 @sample
134 */
135 //-----
136 METHOD NEW(oOwner) CLASS Fly01Login
137
138 If !Empty(oOwner)
139     self:oOwner := oOwner
140     oOwner:ReadClientCoors(.T., .T.)
141 EndIf
142
143 self:oAlertEmailPwd := nil
144 self:oAlertWrongPwd := nil
145 self:nMarginLeft := 15
146 self:nMarginTop := 8
147
148 self:lAlwaysConnected := PADR(STFUserGetInfo("LF_FRTAUT0"),TamSx3("LF_FRTAUT0")[1]) == "1" //manter conectado
149
150 self:lUserCreated := UserCreated() //usuario já criado
151
152 self:cEmail := Space(50)
153 self:cNomeLoja := Space(40)
154 self:cEndereco := Space(50)
155 self:cCidade := Space(50)
156

```

```

157 If self:lAlwaysConnected
158
159     //usuario
160     self:cGetUser := PADR(STFUserGetInfo("LF_ACESS0"),TamSx3("LF_ACESS0")[1]) //usuario por email
161
162     //senha
163     self:cGetPsw := STFUserGetInfo("LF_NUMCART")//senha ainda criptografada
164     self:cGetPsw := STFR4DeCrypt( AllTrim(self:cGetPsw) ,cCryptKey, .F.) //Senha descriptografada
165     self:cGetPsw := PADR(self:cGetPsw,TamSx3("LF_NUMCART")[1])//senha
166
167 Else
168     self:cGetUser := Space(TamSx3("LF_ACESS0")[1]) //usuario por email
169     self:cGetPsw := Space(TamSx3("LF_NUMCART")[1])//senha
170 EndIf
171
172 self:aFontLogin := {'15' ,.T.}
173 self:aFontAccount := {'16' ,.F.}
174
175 self:nHeightAskRegister := 80
176 Self:nHeightHeader := 25
177 Self:nHeightLogin := 135
178 Self:nHeightForgotPsw := 120
179
180 self:nHeightField := 16
181
182 self:nHeightTotal := self:nHeightHeader + self:nHeightLogin + self:nHeightAskRegister
183 self:nWidthTotal := 180
184 self:nTopDefault := (oOwner:nClientHeight/2-self:nHeightTotal)/2
185 self:nLeftDefault := (oOwner:nWidth/2-self:nWidthTotal)/2
186
187 self:nWidthUtil := ( self:nWidthTotal - (self:nMarginLeft * 2) )
188
189
190 Return
191
192
193 //-----
194 /*/{Protheus.doc} Activate()
195 Metodo Ativacao da classe
196
197 @param
198 @return
199 @obs
200 @sample
201 */
202 //-----
203 METHOD Activate() Class FLY01LOGIN
204
205 self:GetHeader()
206 Self:GetLogin()
207 Self:GetAskRegister()
208

```

```

209 Return
210
211
212 //-----
213 /*{Protheus.doc} GetHeader
214 Metodo GetHeader
215
216 @param
217 @return
218 @obs
219 @sample
220 */
221 //-----
222 METHOD GetHeader() CLASS Fly01Login
223
224 Local oTitle      := Nil //Obj Titulo
225 Local oPanelLine  := Nil //painel apenas para desenhar linha branca
226
227 Self:oPnHeader := tPanel():New(self:nTopDefault,self:nLeftDefault,"",self:oOwner,,,,,self:nWidthTotal,self:nHeightHeader)
228 Self:oPnHeader:SetCss(POSCSS (GetClassName(Self:oPnHeader), CSS_PANEL_LOGINGHEADER, .T. ))
229
230
231 @ self:nMarginTop,self:nMarginLeft SAY oTitle PROMPT STR0001 SIZE self:nWidthUtil,30 OF Self:oPnHeader PIXEL //"PONTO DE VENDA - PDV"
232 oTitle:SetCss(POSCSS (GetClassName(oTitle), CSS_BREADCUMB ))
233
234 Return
235
236
237 //-----
238 /*{Protheus.doc} GetLogin
239 Metodo GetLogin
240
241 @param
242 @return
243 @obs
244 @sample
245 */
246 //-----
247 METHOD GetLogin() CLASS Fly01Login
248
249 Local nLine      := self:nMarginTop //Determina a posicao linha
250
251 self:oPnLogin := tPanel():New((Self:oPnHeader:nTop+Self:oPnHeader:nHeight)/2,Self:oPnHeader:nLeft/2,"",self:oOwner,,,,,self:nWidthTotal,self:nHeightLogin)
252 self:oPnLogin:SetCss(POSCSS (GetClassName(self:oPnLogin), CSS_PANEL_LOGINMAIN ))
253 CreatePanelLine(Self:oPnLogin)
254
255 @ nLine,self:nMarginLeft SAY oUser PROMPT STR0002 SIZE self:nWidthUtil,020 OF self:oPnLogin PIXEL //"Email"
256 oUser:SetCss(POSCSS (GetClassName(oUser), CSS_LABEL_FOCAL, self:aFontLogin ))
257
258 nLine += 12
259 @ nLine,self:nMarginLeft GET oGetUser VAR self:cGetUser WHEN {|| .T. } SIZE self:nWidthUtil,self:nHeightField OF self:oPnLogin PIXEL
260 oGetUser:SetCss(POSCSS (GetClassName(oGetUser), CSS_GET_FOCAL ))

```

```

261
262 nLine += 25
263 @ nLine,self:nMarginLeft SAY oPassword PROMPT STR0003 SIZE self:nWidthUtil,020 OF self:oPnLogin PIXEL //"Senha"
264 oPassword:SetCss(POSCSS (GetClassName(oPassword), CSS_LABEL_FOCAL, self:aFontLogin))
265
266 nLine += 12
267 @ nLine,self:nMarginLeft GET oGetPsw VAR Self:cGetPsw SIZE self:nWidthUtil,self:nHeightField OF self:oPnLogin PIXEL PASSWORD
268 oGetPsw:SetCss(POSCSS (GetClassName(oGetPsw), CSS_GET_FOCAL ))
269
270 nLine += 30
271 @ nLine,self:nMarginLeft CHECKBOX oChkConnected VAR self:lAlwaysConnected PROMPT STR0004 SIZE (self:nWidthUtil/2)-5,020 OF self:oPnLogin PIXEL //"Manter conectado"
272 oChkConnected:SetCss(POSCSS (GetClassName(oChkConnected), CSS_CHECKBOX_DEFAULT ))
273
274 nLine += 20
275 @ nLine, self:nMarginLeft BUTTON oBtnLogin PROMPT STR0005 SIZE self:nWidthUtil,self:nHeightField+4 ACTION {|| self:ValidLogin() } OF self:oPnLogin PIXEL //"ENTRAR"
276 oBtnLogin:SetCss(POSCSS( GetClassName(oBtnLogin), CSS_BTN_FOCAL) )
277
278 Return
279
280
281 //-----
282 /*{Protheus.doc} ValidLogin
283 Metodo GetLogin
284
285 @param
286 @return
287 @obs
288 @sample
289 */
290 //-----
291 METHOD ValidLogin() CLASS Fly01Login
292
293 Local aArea      := GetArea()           //Salva area
294 Local lRet       := .F.                 //Retorno
295
296 lAuthentic := .F.
297
298 //esconde msgs de erros anteriores
299 self:HideAlertWrongPwd()
300 self:HideAlertWrong()
301
302 DbSelectArea("SLF")
303 DbSetOrder(1)//LF_FILIAL+LF_COD
304 If DbSeek( xFilial("SLF")+ "USR" )
305
306     If !Empty(self:cGetUser) .AND. !Empty(Self:cGetPsw)
307         lRet := .T.
308         //Valida usuario e senha
309
310         lRet := lRet .AND. AllTrim(SLF->LF_ACESSO) == AllTrim(self:cGetUser) //Email
311
312         lRet := lRet .AND. STFR4DeCrypt( AllTrim(SLF->LF_NUMCART) ) == AllTrim(Self:cGetPsw) //Senha

```

```

313
314     EndIf
315
316 EndIf
317
318 __cInterNet := Nil //Força essa variavel para nao entrar em modo automatico(IsBlind())
319
320 //Atualiza variavel estatica de controle de acesso
321 lAuthentic := lRet
322
323 //Validade Versao Demonstrativa
324 //Se vencida nao deixa prosseguir login
325 If lAuthentic .AND. STFTYPEoperation() == "DEMONSTRACAO"
326     If !VerDemValid()
327         MS_QUIT() //Fecha aplicacao
328     EndIf
329 EndIf
330
331 If lAuthentic
332     //Atualiza informacoes de lembrete de usuario
333     self:Fly01Cadastro()
334     //Fecha tela de login para iniciar o sistema
335     oDlg:End()
336 Else
337     //mostra mensagem de erro
338     self:GetAlertWrongPwd()
339 EndIf
340
341 RestArea(aArea)
342
343 Return lRet
344
345 //-----
346 /*{Protheus.doc} GetAskRegister
347 Metodo GetAskRegister
348
349 @param
350 @return
351 @obs
352 @sample
353 */
354 //-----
355 METHOD GetAskRegister() CLASS Fly01Login
356
357 Local nLine := self:nMarginTop
358
359 self:oPnAskRegister:= tPanel():New((self:oPnLogin:nTop+self:oPnLogin:nHeight)/2,self:oPnLogin:nLeft/2,"",self:oOwner,,,,,self:nWidthTotal,self:nHeightAskRegister)
360 self:oPnAskRegister:SetCss(POSCSS (GetClassName(self:oPnAskRegister), CSS_PANEL_LOGINASKREG ))
361 CreatePanelLine(Self:oPnAskRegister)
362
363 @ nLine,self:nMarginLeft SAY oTitle PROMPT STR0006 SIZE self:nWidthUtil,030 OF self:oPnAskRegister PIXEL //"Não tem cadastro? Informe Seu email"
364 oTitle:SetCss(POSCSS (GetClassName(oTitle), CSS_LABEL_FOCAL, self:aFontAccount ))

```

```

365
366
367 nLine += 15
368 @ nLine,self:nMarginLeft GET self:oEmail VAR Self:cEmail WHEN {|| .T. } SIZE self:nWidthUtil,self:nHeightField OF self:oPnAskRegister PIXEL
369 self:oEmail:lReadOnly := UserCreated() //so habilita se nao tiver user cadastrado
370 self:oEmail:bValid := {|| self:ValidEmail(self:oEmail ,self:cEmail ) }
371 self:oEmail:SetCss(POSCSS (GetClassName(self:oEmail), CSS_GET_FOCAL ))
372
373
374 nLine += 26
375 @ nLine, self:nMarginLeft BUTTON oBtnReg PROMPT STR0007 SIZE self:nWidthUtil,self:nHeightField+4 ACTION {|| IIF(self:ValidEmail(self:oEmail ,self:cEmail
),self:GetRegister(),) } OF self:oPnAskRegister PIXEL //"CADASTRE-SE"
376 oBtnReg:SetCss(POSCSS( GetClassName(oBtnReg), CSS_BTN_NORMAL) )
377 oBtnReg:bWhen := {|| !UserCreated() }//so habilita se nao tiver user cadastrado
378
379 Return
380
381
382 //-----
383 /*{Protheus.doc} GetForgotPassword
384 Metodo GetForgotPassword
385
386 @param
387 @return
388 @obs
389 @sample
390 */
391 //-----
392 METHOD GetForgotPassword() CLASS Fly01Login
393
394 Local nLine := self:nMarginTop //Linha
395
396 self:oPnForgotPassword := tPanel():New((Self:oPnHeader:nTop+Self:oPnHeader:nHeight)/2,Self:oPnHeader:nLeft/2,"",self:oOwner,,,,,Self:nWidthTotal,self:nHeightForgotPsw)
397 self:oPnForgotPassword:SetCss(POSCSS (GetClassName(self:oPnForgotPassword), CSS_PANEL_LOGINFOOTER, .T. ))
398
399
400 @ nLine,self:nMarginLeft SAY oTitle PROMPT STR0008 SIZE self:nWidthUtil,030 OF self:oPnForgotPassword PIXEL //"Esqueci minha senha"
401 oTitle:SetCss(POSCSS (GetClassName(oTitle), CSS_LABEL_FOCAL, {'17' ,.T.} ))
402
403 CreatePanelLine(self:oPnForgotPassword)
404
405 nLine += 20
406 @ nLine,self:nMarginLeft SAY oSayEmail PROMPT STR0009 SIZE self:nWidthUtil,030 OF self:oPnForgotPassword PIXEL //"Informe seu email de cadastro"
407 oSayEmail:SetCss(POSCSS (GetClassName(oSayEmail), CSS_LABEL_FOCAL,{'15' ,.F.} ))
408
409 nLine += 15
410 @ nLine,self:nMarginLeft GET self:oEmail VAR self:cEmail WHEN {|| .T. } SIZE self:nWidthUtil,self:nHeightField OF self:oPnForgotPassword PIXEL
411 self:oEmail:SetCss(POSCSS (GetClassName(self:oEmail), CSS_GET_FOCAL ))
412
413 nLine += 35
414 @ nLine,self:nMarginLeft BUTTON oBtnReg PROMPT STR0010 SIZE self:nWidthUtil,self:nHeightField+4 ACTION {|| self:HideForgotPassword(), self:ShowLogin(),
self:ShowAskRegister(), self:GetAlertEmailPwd() } OF self:oPnForgotPassword PIXEL //"RECUPERAR SENHA"

```



```

415 oBtnReg:SetCss(POSCSS( GetClassName(oBtnReg), CSS_BTN_FOCAL ) )
416
417 Return
418
419
420 //-----
421 /*{Protheus.doc} GetRegister
422 Metodo GetRegister
423
424 @param
425 @return
426 @obs
427 @sample
428 */
429 //-----
430 METHOD GetRegister() CLASS Fly01Login
431
432 Local oPnHeader      := Nil      //Panel cabecario
433 Local oPnBody        := Nil      //Panel principal
434 Local nPanelWidth    := 400      //Largura panel
435 local nHeightBody    := 180      //altura paenl principal
436 Local nHeightFooter  := 40       //Altura panel inferior
437 Local nHeightHeader  := 25       //Altura panel cabecalho
438 Local nWidthUtil     := nPanelWidth - self:nMarginLeft           //Tamanho Panel
439 Local nWidthField    := (nPanelWidth - (3 * self:nMarginLeft))/2 //Tamanho campo
440 Local nTop           := (self:oOwner:nClientHeight/2-(nHeightBody+nHeightHeader+nHeightFooter))/2 //Posicao Top
441 Local nLeft          := (self:oOwner:nWidth/2-nPanelWidth)/2    //Posicao Esquerda
442 Local nLine          := 7.5    //Linha
443 Local nLeft2column   := self:nMarginLeft * 2 + nWidthField      //Posicao esquerda
444
445 //Oculta mensagens anteriores
446 self:HideAlertWrongPwd()
447
448 self:cGetPsw          := Space(TamSx3("LF_NUMCART")[1])//senha
449 self:cRepUsrPwd       := Space(TamSx3("LF_NUMCART")[1])//Confirmar senha
450
451 self:oPnRegister:= tPanel():New(nTop,nLeft,"",self:oOwner,,,,,nPanelWidth,nHeightBody+nHeightHeader+nHeightFooter)
452
453 oPnHeader:= tPanel():New(0,0,"",self:oPnRegister,,,,,nPanelWidth,nHeightHeader)
454 oPnHeader:SetCss(POSCSS (GetClassName(oPnHeader), CSS_PANEL_LOGINGHEADER ))
455
456 @ nLine,self:nMarginLeft SAY oTitle PROMPT STR0011 SIZE nWidthUtil,nHeightHeader OF oPnHeader PIXEL //"Estamos quase lá! Só precisamos de mais algumas informações"
457 oTitle:SetCss(POSCSS (GetClassName(oTitle), CSS_BREADCUMB ))
458
459 oPnBody := tPanel():New(nHeightHeader,0,"",self:oPnRegister,,,,,nPanelWidth,nHeightBody)
460 oPnBody:SetCss(POSCSS (GetClassName(oPnBody), CSS_PANEL_LOGINMAIN ))
461 CreatePanelLine(oPnBody)
462
463
464 nLine += 5
465 @ nLine,self:nMarginLeft SAY oSayLoja PROMPT STR0012 SIZE nWidthField,020 OF oPnBody PIXEL //"Nome da Empresa"
466 oSayLoja:SetCss(POSCSS (GetClassName(oSayLoja), CSS_LABEL_FOCAL, self:aFontLogin ))

```

```

467
468 @ nLine,nLeft2column SAY oSayEmail PROMPT STR0002 SIZE nWidthField,020 OF oPnBody PIXEL //"Email"
469 oSayEmail:SetCss(POSCSS (GetClassName(oSayEmail), CSS_LABEL_FOCAL, self:aFontLogin ))
470
471 nLine += 12
472
473 //Nome da Empresa
474 @ nLine,self:nMarginLeft GET self:oGetLoja VAR self:cNomeLoja WHEN {|| .T. } SIZE nWidthField,self:nHeightField OF oPnBody PIXEL Picture "@!"
475 self:oGetLoja:SetCss(POSCSS (GetClassName(self:oGetLoja), CSS_GET_FOCAL ))
476
477 //Email
478 @ nLine, nLeft2column GET self:oGetEmail VAR self:cEmail WHEN {|| .T. } SIZE nWidthField,self:nHeightField OF oPnBody PIXEL
479 self:oGetEmail:SetCss(POSCSS (GetClassName(self:oGetEmail), CSS_GET_FOCAL ))
480
481 nLine += 30
482
483 @ nLine,self:nMarginLeft SAY oSayPsw PROMPT STR0003 SIZE nWidthField,020 OF oPnBody PIXEL //"Senha"
484 oSayPsw:SetCss(POSCSS (GetClassName(oSayPsw), CSS_LABEL_FOCAL, self:aFontLogin ))
485
486 @ nLine,nLeft2column SAY oSayRepPwd PROMPT STR0013 SIZE nWidthField,020 OF oPnBody PIXEL //"Repetir Senha"
487 oSayRepPwd:SetCss(POSCSS (GetClassName(oSayRepPwd), CSS_LABEL_FOCAL, self:aFontLogin ))
488
489 nLine += 12
490
491 //Senha
492 @ nLine,self:nMarginLeft GET self:oGetPwd VAR self:cGetPsw WHEN {|| .T. } SIZE nWidthField,self:nHeightField OF oPnBody PIXEL PASSWORD
493 self:oGetPwd:SetCss(POSCSS (GetClassName(self:oGetPwd), CSS_GET_FOCAL ))
494
495 //Confirmar Senha
496 @ nLine, nLeft2column GET self:oGetRepPwd VAR self:cRepUsrPwd WHEN {|| .T. } SIZE nWidthField,self:nHeightField OF oPnBody PIXEL PASSWORD
497 self:oGetRepPwd:SetCss(POSCSS (GetClassName(self:oGetRepPwd), CSS_GET_FOCAL ))
498
499 nLine += 30
500
501 @ nLine,self:nMarginLeft SAY oSayEnd PROMPT STR0014 SIZE nWidthField,020 OF oPnBody PIXEL //"Endereço"
502 oSayEnd:SetCss(POSCSS (GetClassName(oSayEnd), CSS_LABEL_FOCAL, self:aFontLogin ))
503
504 @ nLine,nLeft2column SAY oSayPhone PROMPT STR0015 SIZE nWidthField,020 OF oPnBody PIXEL //"Estado"
505 oSayPhone:SetCss(POSCSS (GetClassName(oSayPhone), CSS_LABEL_FOCAL, self:aFontLogin ))
506
507 nLine += 12
508
509 //Endereco
510 @ nLine,self:nMarginLeft GET self:oGetEnd VAR self:cEndereco WHEN {|| .T. } SIZE nWidthField,self:nHeightField OF oPnBody PIXEL Picture "@!"
511 self:oGetEnd:SetCss(POSCSS (GetClassName(self:oGetEnd), CSS_GET_FOCAL ))
512
513 //Estado
514 self:oListEstado := TListBox():Create(oPnBody, nLine, nLeft2column, Nil, STDUF(), nWidthField,self:nHeightField * 4 ,,,,,.T.,{|| },,,, )
515 self:oListEstado:SetCSS( POSCSS (GetClassName(self:oListEstado), CSS_LISTBOX ))
516 self:oListEstado:GoTop()
517
518

```

```

519 nLine += 30
520
521 @ nLine,self:nMarginLeft SAY oSayCid PROMPT STR0016 SIZE nWidthField,020 OF oPnBody PIXEL //"Cidade"
522 oSayCid:SetCss(POSCSS (GetClassName(oSayCid), CSS_LABEL_FOCAL, self:aFontLogin ))
523
524 nLine += 12
525
526 //Cidade
527 @ nLine,self:nMarginLeft GET self:oGetCid VAR self:cCidade WHEN {|| .T. } SIZE nWidthField,self:nHeightField OF oPnBody PIXEL Picture "@!"
528 self:oGetCid:SetCss(POSCSS (GetClassName(self:oGetCid), CSS_GET_FOCAL ))
529
530 oPnFooter := tPanel():New(nHeightHeader+nHeightBody,0,"",self:oPnRegister,,,,,nPanelWidth,nHeightFooter)
531 oPnFooter:SetCss(POSCSS (GetClassName(oPnFooter), CSS_PANEL_LOGINFOOTER ))
532 CreatePanelLine(oPnFooter)
533
534 @ 10, nPanelWidth/2 - nWidthField/2 BUTTON oBtnNewUser PROMPT STR0017 SIZE nWidthField,self:nHeightField+4 ACTION {|| IIF( self:ValidCadastro() , (self:Fly01Cadastro(),
    self:HideRegister() ) , ) } OF oPnFooter PIXEL //"COMEÇAR A USAR O PDV"
535 oBtnNewUser:SetCss(POSCSS( GetClassName(oBtnNewUser), CSS_BTN_FOCAL) )
536
537 //Foco inicial
538 self:oGetLoja:SetFocus()
539
540 Return
541
542
543 //-----
544 /*{Protheus.doc} ValidEmail
545 Metodo valida email
546
547 @param
548 @return
549 @obs
550 @sample
551 */
552 //-----
553 METHOD ValidEmail(oObjGetValid , cEmail) CLASS Fly01Login
554
555 Local lRet := .F. //Controle de Validacao
556
557 Default oObjGetValid := Nil
558 Default cEmail      := ""
559
560
561 If oObjGetValid <> Nil
562
563     lRet := .T.
564
565     IIf(lRet , lRet := "@" $ cEmail ,)
566     IIf(lRet , lRet := ".COM" $ UPPER(cEmail) ,)
567     IIf(lRet , lRet := Len(AllTrim(cEmail)) > 6 ,)
568
569     //troca CSS

```

```

570     If lRet
571         oObjGetValid:SetCss(POSCSS (GetClassName(oObjGetValid), CSS_GET_FOCAL ))
572         self:HideAlertWrong()
573     Else
574         oObjGetValid:SetCss(POSCSS (GetClassName(oObjGetValid), CSS_GET_ERROR ))
575         self:AlertWrong(STR0018)//"Email inválido!"
576     EndIf
577
578 EndIf
579
580 Return lRet
581
582
583 //-----
584 /*{Protheus.doc} ValidGetDef
585 Metodo validacao padrao dos Gets
586
587 @param
588 @return
589 @obs
590 @sample
591 */
592 //-----
593 METHOD ValidGetDef(oObjValid , cSTR) CLASS Fly01Login
594
595 Local lRet := .F. //Controle de Validacao
596
597 Default oObjValid := Nil
598 Default cSTR      := ""
599
600
601 If oObjValid <> Nil
602
603     lRet := .T.
604
605     IIf(lRet , lRet := !Empty(cSTR) ,)
606
607     //troca CSS
608     If lRet
609         oObjValid:SetCss(POSCSS (GetClassName(oObjValid), CSS_GET_FOCAL ))
610         self:HideAlertWrong()
611     Else
612         oObjValid:SetCss(POSCSS (GetClassName(oObjValid), CSS_GET_ERROR ))
613         self:AlertWrong(STR0019)//"Campo de preenchimento Obrigatório não informado!"
614     EndIf
615
616 EndIf
617
618 Return lRet
619
620
621 //-----

```

```

622 */{Protheus.doc} ValidPsw
623 Metodo Valida senha digitada igual senha confirmada
624
625 @param
626 @return
627 @obs
628 @sample
629 */
630 //-----
631 METHOD ValidPsw(oObjValid) CLASS Fly01Login
632
633 Local lRet := .T. //Controle de Validacao
634
635 Default oObjValid := Nil
636
637 IIf(lRet , lRet := (AllTrim(self:cRepUsrPwd ) == AllTrim(self:cGetPsw )) ,)
638
639 //troca CSS
640 If lRet
641     oObjValid:SetCss(POSCSS (GetClassName(oObjValid), CSS_GET_FOCAL ))
642     self:HideAlertWrong()
643 Else
644     oObjValid:SetCss(POSCSS (GetClassName(oObjValid), CSS_GET_ERROR ))
645     self:AlertWrong(STR0020) //"Senha diferente da informada!"
646 EndIf
647
648
649 Return lRet
650
651
652 //-----
653 */{Protheus.doc} ValidCadastro
654 Metodo Valida Cadastro
655
656 @param
657 @return
658 @obs
659 @sample
660 */
661 //-----
662 METHOD ValidCadastro() CLASS Fly01Login
663
664 Local lRet := .T. //Controle de Validacao
665
666 lRet := lRet .AND. self:ValidGetDef( self:oGetLoja , self:cNomeLoja )
667 lRet := lRet .AND. self:ValidEmail( self:oGetEmail , self:cEmail )
668 lRet := lRet .AND. self:ValidGetDef( self:oGetLoja , self:cGetPsw )
669 lRet := lRet .AND. self:ValidPsw( self:self:oGetRepPwd )
670 lRet := lRet .AND. self:ValidGetDef( self:oGetEnd , self:cEndereco )
671 lRet := lRet .AND. self:ValidGetDef( self:oGetCid , self:cCidade )
672
673

```

```
674 Return lRet
675
676
677 //-----
678 /*{Protheus.doc} HideHeader
679 Metodo HideHeader
680
681 @param
682 @return
683 @obs
684 @sample
685 */
686 //-----
687 METHOD HideHeader() CLASS Fly01Login
688 Self:oPnHeader:Hide()
689 Return
690
691
692 //-----
693 /*{Protheus.doc} HideLogin
694 Metodo HideLogin
695
696 @param
697 @return
698 @obs
699 @sample
700 */
701 //-----
702 METHOD HideLogin() CLASS Fly01Login
703 Self:oPnLogin:Hide()
704 Return
705
706
707 //-----
708 /*{Protheus.doc} HideForgotPassword
709 Metodo HideForgotPassword
710
711 @param
712 @return
713 @obs
714 @sample
715 */
716 //-----
717 METHOD HideForgotPassword() CLASS Fly01Login
718 Self:oPnForgetPassword:Hide()
719 Return
720
721
722 //-----
723 /*{Protheus.doc} HideAskRegister
724 Metodo HideAskRegister
725
```

```
726 @param
727 @return
728 @obs
729 @sample
730 /*/
731 //-----
732 METHOD HideAskRegister() CLASS Fly01Login
733 Self:oPnAskRegister:Hide()
734 Return
735
736
737 //-----
738 /*/{Protheus.doc} HideRegister
739 Metodo HideRegister
740
741 @param
742 @return
743 @obs
744 @sample
745 /*/
746 //-----
747 METHOD HideRegister() CLASS Fly01Login
748 Self:oPnRegister:Hide()
749 Return
750
751
752 //-----
753 /*/{Protheus.doc} ShowHeader
754 Metodo ShowHeader
755
756 @param
757 @return
758 @obs
759 @sample
760 /*/
761 //-----
762 METHOD ShowHeader() CLASS Fly01Login
763 Self:oPnHeader:Show()
764 Return
765
766
767 //-----
768 /*/{Protheus.doc} ShowLogin
769 Metodo ShowLogin
770
771 @param
772 @return
773 @obs
774 @sample
775 /*/
776 //-----
777 METHOD ShowLogin() CLASS Fly01Login
```

```
778 Self:oPnLogin:Show()
779 Return
780
781 //-----
782 /*{Protheus.doc} ShowForgotPassword
783 Metodo ShowForgotPassword
784
785 @param
786 @return
787 @obs
788 @sample
789 */
790 //-----
791 METHOD ShowForgotPassword() CLASS Fly01Login
792 Self:oPnForgetPassword:Show()
793 Return
794
795
796 //-----
797 /*{Protheus.doc} ShowAskRegister
798 Metodo ShowAskRegister
799
800 @param
801 @return
802 @obs
803 @sample
804 */
805 //-----
806 METHOD ShowAskRegister() CLASS Fly01Login
807 Self:oPnAskRegister:Show()
808 Return
809
810
811 //-----
812 /*{Protheus.doc} ShowRegister
813 Metodo ShowRegister
814
815 @param
816 @return
817 @obs
818 @sample
819 */
820 //-----
821 METHOD ShowRegister() CLASS Fly01Login
822 Self:oPnRegister:Show()
823 Return
824
825
826 //-----
827 /*{Protheus.doc} AlertWrong
828 Metodo AlertWrong
829
```



```

830 @param
831 @return
832 @obs
833 @sample
834 */
835 //-----
836 METHOD AlertWrong(cMsg) CLASS Fly01Login
837
838 Local nWidth := 0 //Largura
839 Local nHeight := 0 //Altura
840
841 Default cMsg := ""
842
843 If self:oWrongAlert == nil
844
845     self:oWrongAlert := FlyMsgBalloon():New(self:oOwner)
846     self:oWrongAlert:SetText(cMsg, .T.)
847     Self:oWrongAlert:SetMinWidth(self:nWidthTotal)
848     self:oWrongAlert:SetBackgroundColor('#FFC926')
849     self:oWrongAlert:SetTextColor('FFFFFF')
850     nHeight := self:oWrongAlert:GetHeight()
851     self:oWrongAlert:SetTop((self:nTopDefault/2 + nHeight/2)/2)
852     nWidth := self:oWrongAlert:GetWidth()
853     self:oWrongAlert:SetLeft((self:oOwner:nWidth/2-nWidth)/2)
854     self:oWrongAlert:SetImg("fwskin_alert_white.svg")
855     self:oWrongAlert:Activate()
856
857 Else
858     self:oWrongAlert:SetText(cMsg, .T.)
859     self:oWrongAlert:Show()
860 EndIf
861
862 Return
863
864 //-----
865 */{Protheus.doc} HideAlertWrong
866 Metodo HideAlertWrong
867
868 @param
869 @return
870 @obs
871 @sample
872 */
873 //-----
874 METHOD HideAlertWrong() CLASS Fly01Login
875 If self:oWrongAlert <> nil
876     Self:oWrongAlert:Hide()
877 EndIf
878 Return
879
880
881 //-----

```

```

882 /*/{Protheus.doc} GetAlertWrongPwd
883 Metodo GetAlertWrongPwd
884
885 @param
886 @return
887 @obs
888 @sample
889 */
890 //-----
891 METHOD GetAlertWrongPwd() CLASS Fly01Login
892
893 Local nWidth      := 0    //Largura
894 Local nHeight     := 0    //Altura
895
896 If self:oAlertWrongPwd == nil
897
898     self:oAlertWrongPwd := FlyMsgBalloon():New(self:oOwner)
899     self:oAlertWrongPwd:SetText(STR0021, .T.)// "Usuário ou senha incorretos"
900     Self:oAlertWrongPwd:SetMinWidth(self:nWidthTotal)
901     self:oAlertWrongPwd:SetBackgroundColor('#FFC926')
902     self:oAlertWrongPwd:SetTextColor('#FFFFFF')
903     nHeight := self:oAlertWrongPwd:GetHeight()
904     self:oAlertWrongPwd:SetTop((self:nTopDefault/2 + nHeight/2)/2)
905     nWidth := self:oAlertWrongPwd:GetWidth()
906     self:oAlertWrongPwd:SetLeft((self:oOwner:nWidth/2-nWidth)/2)
907     self:oAlertWrongPwd:SetImg("fwskin_alert_white.svg")
908     self:oAlertWrongPwd:Activate()
909
910 Else
911     self:oAlertWrongPwd:Show()
912 EndIf
913
914 Return
915
916 //-----
917 /*/{Protheus.doc} HideAlertWrongPwd
918 Metodo HideAlertWrongPwd
919
920 @param
921 @return
922 @obs
923 @sample
924 */
925 //-----
926 METHOD HideAlertWrongPwd() CLASS Fly01Login
927 If self:oAlertWrongPwd <> nil
928     Self:oAlertWrongPwd:Hide()
929 EndIf
930 Return
931
932
933 //-----

```

```

934 /*{Protheus.doc} GetAlertEmailPwd
935 Metodo GetAlertEmailPwd
936
937 @param
938 @return
939 @obs
940 @sample
941 */
942 //-----
943 METHOD GetAlertEmailPwd() CLASS Fly01Login
944
945 Local nWidth := 0 //Tamanho
946 Local nHeight := 0 //Altura
947
948 If self:oAlertEmailPwd == nil
949
950     self:oAlertEmailPwd := FlyMsgBalloon():New(self:oOwner)
951     self:oAlertEmailPwd:SetText(STR0022, .T.)//Verifique as informações de senha no e-mail de cadastro"
952     self:oAlertEmailPwd:SetBackgroundColor('#7BC400')
953     self:oAlertEmailPwd:SetTextColor('FFFFFF')
954     nHeight := self:oAlertEmailPwd:GetHeight()
955     self:oAlertEmailPwd:SetTop((self:nTopDefault/2 + nHeight/2)/2)
956     nWidth := self:oAlertEmailPwd:GetWidth()
957     self:oAlertEmailPwd:SetLeft((self:oOwner:nWidth/2-nWidth)/2)
958     self:oAlertEmailPwd:SetImg("fwskin_success_white.svg")
959     self:oAlertEmailPwd:Activate()
960
961 Else
962     self:oAlertEmailPwd:Show()
963 EndIf
964
965 Return
966
967 //-----
968 /*{Protheus.doc} HideAlertEmailPwd
969 Metodo HideAlertEmailPwd
970
971 @param
972 @return
973 @obs
974 @sample
975 */
976 //-----
977 METHOD HideAlertEmailPwd() CLASS Fly01Login
978 If self:oAlertEmailPwd <> nil
979     Self:oAlertEmailPwd:Hide()
980 EndIf
981 Return
982
983
984 //-----
985 /*{Protheus.doc} CreatePanelLine

```

```

986 Function CreatePanelline
987 Cria uma linha para fazer a borda. Solução para não ficar exibindo
988 sobre a borda lateral (diminuindo um pixel de cada lado)
989 @param
990 @return
991 @obs
992 @sample
993 /*/
994 //-----
995 Static Function CreatePanelline(oOwner)
996
997 Local oPanel := Nil //Panel principal
998
999 oPanel := tPanel():New(0,0,"",oOwner,,,,,(oOwner:nWidth/2),1)
1000 oPanel:SetCss(POSCSS (GetClassName(oPanel), CSS_PANEL_LINEAUX, .T. ))
1001
1002 Return oPanel
1003
1004
1005 //-----
1006 /*/{Protheus.doc} OpenFLY010open
1007 User Function testes Fly01Login
1008
1009 @param
1010 @return
1011 @obs
1012 @sample
1013 /*/
1014 //-----
1015 Function FLY010open()
1016
1017 Local aRes           := GetScreenRes()           // Recupera Resolução atual
1018 Local nWidth         := aRes[1]                 // Largura
1019 Local nHeight        := aRes[2]                 // Altura
1020 Local oLogin         := Nil                     //Obj Login
1021 Local cUser          := "1"                     //Usuario do sistema
1022 Local cPasswd        := "1"                     //Senha
1023 Local cEmp           := "T1"                     //Empresa
1024 Local cFil           := "D MG 01 "              //Filial
1025 Local cCallBack      := "STIPosMain()"          //Programa principal
1026 Local aTables        := {"SLF"}                 //Tabelas a serem abertas antes do login
1027 Local oPanel         := Nil                     //Painel principal
1028
1029 //abre o ambiente com a empresa e filial
1030 RpcSetEnv( cEmp,cFil, , " ", "FRT", , aTables)
1031
1032 //Seta autenticacao inicial como falso
1033 lAuthentic := .F.
1034
1035 DEFINE DIALOG oDlg Pixel Of GetWndDefault() STYLE nOr(WS_VISIBLE, WS_POPUP)
1036
1037     oDlg:nWidth      := nWidth

```

```

1038     oDlg:nHeight      := nHeight
1039
1040     oPanel:= tPanel():New(00,000,"",oDlg,,,,,000,000)
1041     oPanel:Align:= CONTROL_ALIGN_ALLCLIENT
1042     oPanel:SetCss('QFrame{ background-color: #DDDDDD; }')
1043
1044
1045     oLogin := FLY01LOGIN():New(oPanel)
1046     oLogin:Activate()
1047
1048
1049 ACTIVATE DIALOG oDlg CENTER
1050
1051 //Limpa o ambiente, liberando a licença e fechando as conexões
1052 RpcClearEnv()
1053
1054 //Caso valid o usuario, inicia autenticao no Protheus
1055 If lAuthentic
1056     Fly01Authentication(cUser, cPasswd, cEmp, cFil, cCallBack)
1057 Else
1058     MS_QUIT() //Fecha aplicacao
1059 EndIf
1060
1061 Return .T.
1062
1063
1064 //-----
1065 /*{Protheus.doc} STFUserGetInfo
1066 Retorna informacoes do usuario
1067
1068 @param      uCampos - Informa o campo ou os campos que deseja retornar o valor da SLG
1069 @return      uReturn - Retorna o valor do campo ou dos campos que foi solicitado na funcao
1070 @obs
1071 @sample
1072 */
1073 //-----
1074 Function STFUserGetInfo(uCampos)
1075
1076 Local aArea      := GetArea()           //Salva area
1077 Local uReturn     := ""                 //Retorno da funcao
1078 Local nI          := 0                  //Variavel de loop
1079 Local xRet        := ""                 //Variavel que armazena o nome do campo
1080
1081 Default uCampos   := Nil
1082
1083 DbSelectArea("SLF")
1084 DbSetOrder(1)//LF_FILIAL+LF_COD
1085
1086 If DbSeek( xFilial("SLF")+ "USR" )
1087
1088     If ValType(uCampos) == "A"
1089

```

```

1090     uReturn := {}
1091
1092     For nI := 1 To Len(uCampos)
1093
1094         xRet := FieldGet( FieldPos( If(! Left(uCampos[nI],3) == "LF_", "LF_", "") + uCampos[nI]))
1095
1096         If ValType(xRet) == "C"
1097             xRet := xRet
1098         EndIf
1099
1100         AAdd(uReturn,xRet)
1101
1102     Next nI
1103
1104 Else
1105
1106     uReturn := FieldGet( FieldPos( If(! Left(uCampos,3) == "LF_", "LF_", "") + uCampos))
1107
1108     If Valtype(uReturn) == "C"
1109         uReturn := uReturn
1110     EndIf
1111
1112 EndIf
1113
1114 EndIf
1115
1116 RestArea(aArea)
1117
1118 Return uReturn
1119
1120
1121 //-----
1122 /*{Protheus.doc} UserCreated()
1123 Retorna se ja possui usuario criado
1124
1125 @param
1126 @return    lRet - Retorna se ja possui usuario criado
1127 @obs
1128 @sample
1129 */
1130 //-----
1131 Static Function UserCreated()
1132
1133 Local aArea      := GetArea()           //Salva area
1134 Local lRet       := .T.                 //Retorno
1135
1136
1137 DbSelectArea("SLF")
1138 DbSetOrder(1)//LF_FILIAL+LF_COD
1139 If DbSeek( xFilial("SLF")+ "USR" )
1140
1141     IIf(lRet , lRet := !Empty(SLF->LF_ACESSO) ,)//usuario email

```

```

1142     IIf(lRet      , lRet := !Empty(SLF->LF_NUMCART)      ,)//Senha
1143
1144 Else
1145     lRet          := .F.
1146 EndIf
1147
1148 RestArea(aArea)
1149
1150 Return lRet
1151
1152
1153 //-----
1154 /*/{Protheus.doc} UpdUser()
1155 Cria ou atualiza informações do usuario
1156
1157 @param
1158 @return      lRet - Retorna se cadastrou/atualizou  usuario
1159 @obs
1160 @sample
1161 */
1162 //-----
1163 Method Fly01Cadastro() CLASS Fly01Login
1164
1165 Local aArea      := GetArea()          //Salva area
1166 Local lRet       := .T.                //Retorno
1167
1168
1169 DbSelectArea("SLF")
1170 DbSetOrder(1)//LF_FILIAL+LF_COD
1171 If DbSeek( xFilial("SLF")+ "USR" )
1172
1173     RecLock("SLF",.F.)
1174
1175     SLF->LF_FRTAUTO := IIf(self:lAlwaysConnected,"1","0");// lembrar usuario
1176
1177     SLF->( MsUnLock() )
1178
1179 Else
1180
1181     RecLock("SLF",.T.)
1182
1183     SLF->LF_FILIAL  := xFilial("SLF")
1184     SLF->LF_COD     := "USR"          //Usuario
1185     SLF->LF_ACESSO  := self:cEmail    //Email
1186
1187     //Senha
1188     SLF->LF_NUMCART := Rc4Crypt( AllTrim(self:cGetPsw) , cCryptKey, .T. ) //Senha criptografada
1189
1190
1191     SLF->LF_FRTAUTO := IIf(self:lAlwaysConnected,"1","0");// lembrar usuario
1192
1193     SLF->( MsUnLock() )

```

```

1194
1195 //Seta email cadastrado para tela inicial ja estar preenchida
1196 self:cGetUser := self:cEmail
1197
1198 //Se for versao demonstrativa
1199 If STFTYPEOperation() == "DEMONSTRACAO"
1200     //Valida por 14 dias. Dia atual(1) + 13
1201     //Grava vencimento da versao demonstrativa criptografado
1202     STFSetStat( {{ "LG_PAFMD5" , Rc4Crypt( DtoS(Date()+13) ,cCryptKey, .T.) }} )
1203 EndIf
1204
1205 EndIf
1206
1207 RestArea(aArea)
1208
1209 Return lRet
1210
1211
1212 //-----
1213 /*{Protheus.doc} VerDemValid()
1214 Valida se a versao demonstrativa esta valida
1215
1216 @param
1217 @return      lRet - Retorna se ja possui usuario criado
1218 @obs
1219 @sample
1220 */
1221 //-----
1222 Static Function VerDemValid()
1223
1224 Local cCryptVenc      := "" //Vencimento criptografado
1225 Local cDecryptVenc    := "" //Vencimento decriptografado
1226 Local dDataVenc       := dDatabase-1 //Inicia data de vencimento sempre vencida
1227 Local lRet            := .T. //Retorno
1228
1229 cCryptVenc := AllTrim(STFGetStation("LG_PAFMD5") )
1230
1231 If !Empty(cCryptVenc)
1232     cDecryptVenc := STFR4DeCrypt( cCryptVenc )
1233     dDataVenc := STOD(cDecryptVenc)
1234 EndIf
1235
1236 //Data maior q o vencimento ou menor que o periodo inicial sai do sistema
1237 If (dDatabase > dDataVenc ) .OR. ( dDatabase < dDataVenc-13 )
1238     //Seta data antiga para nao mais permitir entrar no sistema
1239     //Nem se voltar a data conseguiu entrar
1240     lRet := .F.
1241     STFSetStat( {{ "LG_PAFMD5" , Rc4Crypt( DtoS(Date()-(365*15)) ,cCryptKey, .T.) }} )
1242     FWAlertInfo( STR0023 + CRLF + CRLF + STR0024 + " Fly01.com.br" )/"Versão expirada!" ### "Saiba como contratar em: Fly01.com.br"
1243     MS_QUIT() //Fecha aplicacao
1244 EndIf
1245

```



```
1246 Return lRet
1247
1248
1249 //-----
1250 /*{Protheus.doc} STFRC4Crypt
1251 Realiza a geração da Criptografia
1252 @param cTxt      Texto a ser decryptografado
1253 @return cToken    Texto a ser decryptografado
1254 @obs
1255 @sample
1256 */
1257 //-----
1258 Static Function STFRC4DeCrypt(cTxt)
1259
1260 Local cRet := ""
1261 Local nX   := 0
1262
1263 Default cTxt := ""
1264
1265 For nX := 1 To Len(cTxt)
1266     cRet += chr(CTON( SubStr(cTxt, nX, 2) , 16))
1267     nX++
1268 Next nX
1269
1270 cRet := RC4Crypt( cRet , cCryptKey, .F.)
1271
1272 Return cRet
```

Framework/Mobile/STFCadEmp.prw

391 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'Protheus.ch'
2 #INCLUDE 'POSCSS.CH'
3 #INCLUDE 'FWMVCDEF.CH'
4 #INCLUDE "TBICONN.CH"
5 #INCLUDE "STFCADEMP.CH"
6
7 Static oDlg                := Nil // Dialog Principal
8 Static cNomeEmp            := "" // Nome da empresa
9 Static cNFantasia          := "" // Nome fantasia
10 Static cCNPJ               := "" // Cnpj
11 Static cIE                 := "" // Inscricao estadual
12 Static cIM                 := "" // Inscricao municipal
13 Static cCodMun             := "" // codigo do municipio
14 Static cEndereco           := "" // Endereço
15 Static cComplemento        := "" // Complemento do endereço
16 Static cBairro             := "" // Bairro
17 Static cCep                := "" // CEP
18 Static cCidade             := "" // Cidade
19 Static cEstado             := "" // Estado
20 Static cCNAE               := "" // CNAE
21 Static oGetCNAE            := Nil // CNAE
22 Static oGetEmp             := Nil // Nome da empresa
23 Static oGetNFant           := Nil // Nome fantasia
24 Static oGetCnpj            := Nil // Cnpj
25 Static oGetIE              := Nil // Inscricao estadual
26 Static oGetIM              := Nil // Inscricao municipal
27 Static oGetCodMun          := Nil // codigo do municipio
28 Static oGetEnd             := Nil // Endereço
29 Static oGetComp            := Nil // Complemento do endereço
30 Static oGetBairro          := Nil // Bairro
31 Static oGetCep              := Nil // CEP
32 Static oGetCid             := Nil // Cidade
33 Static oGetEstado          := Nil // Estado
34
35
36 //-----
37 /*{Protheus.doc} STFCadEmp
38 Cadastro empresa
39
40 @param
41 @return
42 @obs
43 @sample
44 */
45 //-----
46 Function STFCadEmp()
47
48 Local oPnHeader            := Nil // Painel Superior
49 Local oPnBody              := Nil // Painel Principal
50 Local nPanelWidth          := 600 // Largura
51 Local nHeightBody          := 300 // Altura
52 Local nHeightFooter        := 40 // Altura

```

```

53 Local nHeightHeader      := 25          // Altura
54 Local nMarginLeft        := 15          // Margem Esquada
55 Local nMarginTop          := 8           // Margem direita
56 Local aRes                := GetScreenRes() // Recupera Resolução atual
57 Local nWidth              := aRes[1]     // Largura
58 Local nHeight             := aRes[2]     // Altura
59 Local nWidthUtil          := 0           // Largura
60 Local nWidthField         := 0           // Largura
61 Local nTop                := 0           // Superior
62 Local nLeft               := 0           // Esquerda
63 Local nLine               := 0           // Linha
64 Local nRigth2column       := 0           // Coluna Direita
65 Local aFontLogin          := {'15' ,.T.} // Fonte
66 Local aFontAccount        := {'16' ,.F.} // Fonte
67 Local nHeightField        := 16         // Largura
68 Local oPnRegister         := Nil         // Panel
69 Local oTitle              := Nil         // Objeto titulo
70 Local oSayEmp              := Nil         // Say Empresa
71 Local oSayEmail           := Nil         // Say email
72 Local oSayCnpj            := Nil         // Say CNPJ
73 Local oSayIE              := Nil         // Say IE
74 Local oSayIM              := Nil         // Say IM
75 Local oSayCodMun          := Nil         // SayCodigo do municipio
76 Local oSayCNAE            := Nil         // Say CNAE
77 Local oSayEnd             := Nil         // Say Endereco
78 Local oSayComp            := Nil         // Say Complemento
79 Local oSayBairro          := Nil         // Say Bairro
80 Local oSayCep              := Nil         // Say Cep
81 Local oSayCid             := Nil         // Say Cidade
82 Local oSayEst             := Nil         // Say Estado
83 Local oPnFooter           := Nil         // Painel inferior
84 Local oBtnCad             := Nil         // Botao Cadastrar
85
86 //Inicializa variaveis dos Gets
87 IniFields()
88
89 DEFINE DIALOG oDlg Pixel Of GetWndDefault() STYLE nOr(WS_VISIBLE, WS_POPUP)
90
91 oDlg:nWidth      := nWidth
92 oDlg:nHeight     := nHeight
93 nWidthUtil       := nPanelWidth - nMarginLeft
94 nWidthField      := (nPanelWidth - (3 * nMarginLeft))/2
95 nTop             := (oDlg:nClientHeight/2-(nHeightBody+nHeightHeader+nHeightFooter))/2
96 nLeft           := (oDlg:nWidth/2-nPanelWidth)/2
97 nLine           := 7.5
98 nRigth2column   := nMarginLeft*2+nWidthField
99
100 oPnRegister:= tPanel():New(nTop,nLeft,"",oDlg,,,,,nPanelWIdth,nHeightBody+nHeightHeader+nHeightFooter)
101
102 oPnHeader:= tPanel():New(0,0,"",oPnRegister,,,,,nPanelWIdth,nHeightHeader)
103 oPnHeader:SetCss(POSCSS (GetClassName(oPnHeader), CSS_PANEL_LOGINGHEADER ))
104

```

```

105 //Titulo do cadastro
106 @ nLine,nMarginLeft SAY oTitle PROMPT STR0001 SIZE nWidthUtil,nHeightHeader OF oPnHeader PIXEL //"Dados Cadastrais da Empresa"
107 oTitle:SetCss(POSCSS (GetClassName(oTitle), CSS_BREADCUMB ))
108
109 oPnBody := tPanel():New(nHeightHeader,0,"",oPnRegister,,,,,nPanelWidth,nHeightBody)
110 oPnBody:SetCss(POSCSS (GetClassName(oPnBody), CSS_PANEL_LOGINMAIN ))
111
112 //----- Linha 1
113 nLine += 5
114 @ nLine,nMarginLeft SAY oSayEmp PROMPT STR0002 SIZE nWidthField,020 OF oPnBody PIXEL //"Nome Comercial"
115 oSayEmp:SetCss(POSCSS (GetClassName(oSayEmp), CSS_LABEL_FOCAL, aFontLogin ))
116
117 @ nLine,nRigth2column SAY oSayEmail PROMPT STR0003 SIZE nWidthField,020 OF oPnBody PIXEL //"Nome Fantasia"
118 oSayEmail:SetCss(POSCSS (GetClassName(oSayEmail), CSS_LABEL_FOCAL, aFontLogin ))
119
120 nLine += 12
121
122 //Nome da Empresa
123 @ nLine,nMarginLeft GET oGetEmp VAR cNomeEmp WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL
124 oGetEmp:SetCss(POSCSS (GetClassName(oGetEmp), CSS_GET_FOCAL ))
125
126
127 //Nome Fantasia
128 @ nLine, nRigth2column GET oGetNFant VAR cNFantasia WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL
129 oGetNFant:SetCss(POSCSS (GetClassName(oGetNFant), CSS_GET_FOCAL ))
130
131
132 //----- Linha 2
133 nLine += 30
134 @ nLine,nMarginLeft SAY oSayCnpj PROMPT STR0004 SIZE nWidthField,020 OF oPnBody PIXEL //"CNPJ"
135 oSayCnpj:SetCss(POSCSS (GetClassName(oSayCnpj), CSS_LABEL_FOCAL, aFontLogin ))
136
137 @ nLine,nRigth2column SAY oSayIE PROMPT STR0005 SIZE nWidthField,020 OF oPnBody PIXEL //"Inscrição Estadual"
138 oSayIE:SetCss(POSCSS (GetClassName(oSayIE), CSS_LABEL_FOCAL, aFontLogin ))
139
140 nLine += 12
141
142 //CNPJ
143 @ nLine, nMarginLeft GET oGetCnpj VAR cCNPJ WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL Picture "@E " + Replicate("9",Len(cCNPJ) )
144 oGetCnpj:SetCss(POSCSS (GetClassName(oGetCnpj), CSS_GET_FOCAL ))
145
146 //Inscrição Estadual
147 @ nLine,nRigth2column GET oGetIE VAR cIE WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL Picture "@E " + Replicate("9",Len(cIE) )
148 oGetIE:SetCss(POSCSS (GetClassName(oGetIE), CSS_GET_FOCAL ))
149
150
151 //----- Linha 3
152 nLine += 30
153
154 @ nLine,nMarginLeft SAY oSayIM PROMPT STR0006 SIZE nWidthField,020 OF oPnBody PIXEL //"Inscrição Municipal"
155 oSayIM:SetCss(POSCSS (GetClassName(oSayIM), CSS_LABEL_FOCAL, aFontLogin ))
156

```

```

157 @ nLine,nRigth2column SAY oSayCodMun PROMPT STR0007 SIZE nWidthField,020 OF oPnBody PIXEL //"Código Município IBGE"
158 oSayCodMun:SetCss(POSCSS (GetClassName(oSayCodMun), CSS_LABEL_FOCAL, aFontLogin ))
159
160 nLine += 12
161
162 //Inscrição Municipal
163 @ nLine,nMarginLeft GET oGetIM VAR cIM WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL Picture "@E " + Replicate("9",Len(cIM) )
164 oGetIM:SetCss(POSCSS (GetClassName(oGetIM), CSS_GET_FOCAL ))
165
166 //Código Municipio
167 @ nLine,nRigth2column GET oGetCodMun VAR cCodMun WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL Picture "@E " + Replicate("9",Len(cCodMun) )
168 oGetCodMun:SetCss(POSCSS (GetClassName(oGetCodMun), CSS_GET_FOCAL ))
169
170
171 //----- Linha 4
172 nLine += 30
173
174 @ nLine,nMarginLeft SAY oSayCNAE PROMPT STR0008 SIZE nWidthField,020 OF oPnBody PIXEL //"CNAE"
175 oSayCNAE:SetCss(POSCSS (GetClassName(oSayCNAE), CSS_LABEL_FOCAL, aFontLogin ))
176
177 nLine += 12
178
179 //CNAE
180 @ nLine,nMarginLeft GET oGetCNAE VAR cCNAE WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL
181 oGetCNAE:SetCss(POSCSS (GetClassName(oGetCNAE), CSS_GET_FOCAL ))
182
183
184 //----- Linha 5
185 nLine += 30
186
187 @ nLine,nMarginLeft SAY oSayEnd PROMPT STR0009 SIZE nWidthField,020 OF oPnBody PIXEL //"Endereço"
188 oSayEnd:SetCss(POSCSS (GetClassName(oSayEnd), CSS_LABEL_FOCAL, aFontLogin ))
189
190 @ nLine,nRigth2column SAY oSayComp PROMPT STR0010 SIZE nWidthField,020 OF oPnBody PIXEL //"Complemento"
191 oSayComp:SetCss(POSCSS (GetClassName(oSayComp), CSS_LABEL_FOCAL, aFontLogin ))
192
193 nLine += 12
194
195 //Endereco
196 @ nLine,nMarginLeft GET oGetEnd VAR cEndereco WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL
197 oGetEnd:SetCss(POSCSS (GetClassName(oGetEnd), CSS_GET_FOCAL ))
198
199
200 //Complemento
201 @ nLine,nRigth2column GET oGetComp VAR cComplemento WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL
202 oGetComp:SetCss(POSCSS (GetClassName(oGetComp), CSS_GET_FOCAL ))
203
204
205 //----- Linha 6
206 nLine += 30
207
208 @ nLine,nMarginLeft SAY oSayBairro PROMPT STR0011 SIZE nWidthField,020 OF oPnBody PIXEL //"Bairro"

```

```

209 oSayBairro:SetCss(POSCSS (GetClassName(oSayBairro), CSS_LABEL_FOCAL, aFontLogin ))
210
211 @ nLine,nRigth2column SAY oSayCep PROMPT STR0012 SIZE nWidthField,020 OF oPnBody PIXEL //"CEP"
212 oSayCep:SetCss(POSCSS (GetClassName(oSayCep), CSS_LABEL_FOCAL, aFontLogin ))
213
214 nLine += 12
215
216 //Bairro
217 @ nLine,nMarginLeft GET oGetBairro VAR cBairro WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL
218 oGetBairro:SetCss(POSCSS (GetClassName(oGetBairro), CSS_GET_FOCAL ))
219
220
221 //CEP
222 @ nLine,nRigth2column GET oGetCep VAR cCep WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL Picture "@E " + Replicate("9",Len(cCep) )
223 oGetCep:SetCss(POSCSS (GetClassName(oGetCep), CSS_GET_FOCAL ))
224
225
226 //----- Linha 7
227 nLine += 30
228
229 @ nLine,nMarginLeft SAY oSayCid PROMPT STR0013 SIZE nWidthField,020 OF oPnBody PIXEL //"Cidade"
230 oSayCid:SetCss(POSCSS (GetClassName(oSayCid), CSS_LABEL_FOCAL, aFontLogin ))
231
232 @ nLine,nRigth2column SAY oSayEst PROMPT STR0014 SIZE nWidthField,020 OF oPnBody PIXEL //"Estado"
233 oSayEst:SetCss(POSCSS (GetClassName(oSayEst), CSS_LABEL_FOCAL, aFontLogin ))
234
235 nLine += 12
236
237 //Cidade
238 @ nLine,nMarginLeft GET oGetCid VAR cCidade WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL
239 oGetCid:SetCss(POSCSS (GetClassName(oGetCid), CSS_GET_FOCAL ))
240
241 //Estado
242 @ nLine,nRigth2column GET oGetEstado VAR cEstado WHEN {|| .T. } SIZE nWidthField,nHeightField OF oPnBody PIXEL Picture "@!"
243 oGetEstado:SetCss(POSCSS (GetClassName(oGetEstado), CSS_GET_FOCAL ))
244
245
246 //----- Rodape
247
248 oPnFooter := tPanel():New(nHeightHeader+nHeightBody,0,"",oPnRegister,,,,,nPanelWidth,nHeightFooter)
249 oPnFooter:SetCss(POSCSS (GetClassName(oPnFooter), CSS_PANEL_LOGINFOOTER ))
250
251 @ 10, nPanelWidth/2 - nWidthField/2 BUTTON oBtnCad PROMPT STR0015 SIZE nWidthField,nHeightField+4 ACTION {|| IIF( ValidCadastro() , (STFUpdEmp(),oDlg:End() ) , ) } OF
    oPnFooter PIXEL //"CADASTRAR"
252 oBtnCad:SetCss(POSCSS( GetClassName(oBtnCad), CSS_BTN_FOCAL) )
253
254
255 //Foco inicial
256 oGetEmp:SetFocus()
257
258
259 ACTIVATE DIALOG oDlg CENTER

```

```

260
261 Return .T.
262
263
264 //-----
265 /*{Protheus.doc} ValidCadastro
266 Valida o Cadastro antes da gravacao do dados
267
268 @param
269 @return
270 @obs
271 @sample
272 */
273 //-----
274 Static Function ValidCadastro()
275
276 Local lRet := .T. //Controle de Validacao
277
278 lRet := lRet .AND. !Empty(cNomeEmp )
279 IIF(!Empty(cNomeEmp ), oGetEmp:SetCss(POSCSS (GetClassName(oGetEmp), CSS_GET_FOCAL )) , oGetEmp:SetCss(POSCSS (GetClassName(oGetEmp), CSS_GET_ERROR )) )
280
281 lRet := lRet .AND. !Empty(cNFantasia)
282 IIF(!Empty(cNFantasia), oGetNFant:SetCss(POSCSS (GetClassName(oGetNFant), CSS_GET_FOCAL )) , oGetNFant:SetCss(POSCSS (GetClassName(oGetNFant), CSS_GET_ERROR )) )
283
284 lRet := lRet .AND. !Empty(cCNPJ )
285 IIF(!Empty(cCNPJ ), oGetCnpj:SetCss(POSCSS (GetClassName(oGetCnpj), CSS_GET_FOCAL )) , oGetCnpj:SetCss(POSCSS (GetClassName(oGetCnpj), CSS_GET_ERROR )) )
286
287 lRet := lRet .AND. !Empty(cCodMun)
288 IIF(!Empty(cCodMun), oGetCodMun:SetCss(POSCSS (GetClassName(oGetCodMun), CSS_GET_FOCAL )) , oGetCodMun:SetCss(POSCSS (GetClassName(oGetCodMun), CSS_GET_ERROR )) )
289
290 lRet := lRet .AND. !Empty(cEndereco)
291 IIF(!Empty(cEndereco), oGetEnd:SetCss(POSCSS (GetClassName(oGetEnd), CSS_GET_FOCAL )) , oGetEnd:SetCss(POSCSS (GetClassName(oGetEnd), CSS_GET_ERROR )) )
292
293 lRet := lRet .AND. !Empty(cBairro)
294 IIF(!Empty(cBairro), oGetBairro:SetCss(POSCSS (GetClassName(oGetBairro), CSS_GET_FOCAL )) , oGetBairro:SetCss(POSCSS (GetClassName(oGetBairro), CSS_GET_ERROR )) )
295
296 lRet := lRet .AND. !Empty(cCep )
297 IIF( !Empty(cCep ), oGetCep:SetCss(POSCSS (GetClassName(oGetCep), CSS_GET_FOCAL )) , oGetCep:SetCss(POSCSS (GetClassName(oGetCep), CSS_GET_ERROR )) )
298
299 lRet := lRet .AND. !Empty(cCidade)
300 IIF(!Empty(cCidade) , oGetCid:SetCss(POSCSS (GetClassName(oGetCid), CSS_GET_FOCAL )) , oGetCid:SetCss(POSCSS (GetClassName(oGetCid), CSS_GET_ERROR )) )
301
302 lRet := lRet .AND. !Empty(cEstado)
303 IIF(!Empty(cEstado) , oGetEstado:SetCss(POSCSS (GetClassName(oGetEstado), CSS_GET_FOCAL )) , oGetEstado:SetCss(POSCSS (GetClassName(oGetEstado), CSS_GET_ERROR )) )
304
305 If !lRet
306     FwAlertError(STR0016) //"Campos obrigatórios não informados!"
307 EndIf
308
309
310 Return lRet
311

```

```

312
313 //-----
314 /*{Protheus.doc} IniFields
315 Inicializa Campos com tamanhos corretos de acordo com Sigamat
316
317 @param
318 @return
319 @obs
320 @sample
321 */
322 //-----
323 Static Function IniFields()
324
325 Local aArea      := GetArea()           //Salva area
326
327 DbSelectArea( "SM0" )
328
329 cNomeEmp          := SM0->M0_NOMECON
330 cNFantasia        := SM0->M0_NOME
331 cCNPJ             := SM0->M0_CGC
332 cIE               := SM0->M0_INSC
333 cIM               := SM0->M0_INSCM
334 cCodMun           := SM0->M0_CODMUN
335 cEndereco         := SM0->M0_ENDENT
336 cComplemento      := SM0->M0_COMPENT
337 cBairro           := SM0->M0_BAIRENT
338 cCep              := SM0->M0_CEPENT
339 cCidade           := SM0->M0_CIDENT
340 cEstado           := SM0->M0_ESTENT
341 cCNAE             := SM0->M0_CNAE
342
343 RestArea(aArea)
344
345 Return Nil
346
347 //-----
348 /*{Protheus.doc} STFUpdEmp
349 Atualiza informações da empresa
350
351 @param
352 @return      lRet - Retorna se atualizou
353 @obs
354 @sample
355 */
356 //-----
357 Static Function STFUpdEmp()
358 Local lRet      := .T.           //Retorno
359 Return lRet
360
361
362 //-----
363 /*{Protheus.doc} STFVldCadEmp

```



```
364 Valida se o cadastro de empresas ja foi preenchido
365
366 @param
367 @return      lRet - Retorna se o cadastro de empresas ja foi preenchido
368 @obs
369 @sample
370 /*/
371 //-----
372 Function STFVldCadEmp()
373
374 Local aArea      := GetArea()           //Salva area
375 Local lRet := .T. //Controle de Validacao
376
377 DbSelectArea( "SM0" )
378
379 lRet := lRet .AND. !Empty(SM0->M0_NOMECOM )
380 lRet := lRet .AND. !Empty(SM0->M0_NOME )
381 lRet := lRet .AND. !Empty(SM0->M0_CGC )
382 lRet := lRet .AND. !Empty(SM0->M0_CODMUN )
383 lRet := lRet .AND. !Empty(SM0->M0_ENDENT )
384 lRet := lRet .AND. !Empty(SM0->M0_BAIRENT )
385 lRet := lRet .AND. !Empty(SM0->M0_CEPENT )
386 lRet := lRet .AND. !Empty(SM0->M0_CIDENT )
387 lRet := lRet .AND. !Empty(SM0->M0_ESTENT )
388
389 RestArea(aArea)
390
391 Return lRet
```

Framework/Monitor/STFCompDic.prw

612 linhas após remoção de metadados de autoria/data

```

1 #Include "Protheus.ch"
2 #include "tbiconn.ch"
3 #include "STFCompDic.ch"
4
5 Static cFuncLog      := "Comparador_Bases" //Identificacao da funcionalidade nos Logs
6 Static aSX2          := {} // Campos do SX2 avaliados
7 Static aSX3          := {} // Campos do SX3 avaliados
8 Static aTables       := {} // Tabelas avaliadas
9 Static cMsgHTML      := "" // html retorno para envio de email
10 Static lPDV          := .F. // Verifica se eh um PDV ou Front
11 Static cTipoExec     := STR0003 //Schedule/Job
12
13 //-----
14 /*{Protheus.doc} STFCompDic
15 Compara dicionarios Retaguarda x PDV
16
17 @param
18 @return Nil
19 @obs
20 @sampleq
21 */
22 //-----
23 Function STFCompDic()
24
25 Local lRet           := .T. // Retorno
26 Local cMsg           := "" //Mensagem execucao
27
28 lPDV := nModulo == 23 //Iniciado aqui pois no startjob nao é alimentado
29
30 If !IsBlind()//Se nao for Job ou servico exibe tela de aviso
31
32     cTipoExec := STR0002 //"Menu" // Tipo de execucao
33
34     cMsg := STR0023 + CRLF +; //"Atenção, a rotina será executada em segundo plano(nova Thread). "
35           STR0024 + CRLF +; //"O tempo de execucao dependerá da quantidade de PDVs e disponibilidade de rede. "
36           STR0025 + AllTrim(SuperGetMv("MV_LJMMAIL", "")) + CRLF +; //"O resultado da análise será enviada para o E-mail: "
37           STR0026 + CRLF +; //" configurado no parâmetro MV_LJMMAIL."
38           STR0027 + "STFMailTes. " + CRLF +; //" Caso não consiga receber o e-mail, realize um teste pela rotina "
39           STR0028 + "tdn.totvs.com " + CRLF + CRLF +; //"Para mais informações acesse "
40           STR0029 //"Gostaria de prosseguir?"
41
42
43     If FWAlertYesNo( cMsg , STR0001) //"Comparador de dicionários"
44         FWAlertSuccess(STR0030, STR0031) //"Feche esta janela e aguarde o recebimento do e-mail." "Processo iniciado com sucesso"
45         lRet := .T.
46     Else
47         lRet := .F.
48     EndIf
49
50
51 EndIf
52

```

```

53 If lRet
54     StartJob("STFSrvComp", GetEnvServer(), .F. ,cEmpAnt, cFilAnt , lPDV ,cTipoExec )
55 EndIf
56
57 Return Nil
58
59
60 //-----
61 /*{Protheus.doc} STFSrvComp
62 Servico para comparar dicionarios Retaguarda x PDV
63
64 @param  cEmp - Empresa
65 @param  cFil - Filial
66 @param  lIsPDV - Eh PDV ou Front
67 @param  cTpExec - Tipo de execucao job/Menu
68 @return lRet - Retorno se executou com sucesso.
69 @obs
70 @sample
71 */
72 //-----
73 Function STFSrvComp(cEmp, cFil , lIsPDV , cTpExec)
74
75 Local lRet          := .F.  // Retorno
76 Local aHosts        := {}  // Array de configuracoes dos aHosts
77 Local nX            := 0    // Contador
78 Local cDestinat      := ""  // Destinatários do email separado por (;) ponto e virgula
79 Local nSeconds       := 0    // Armazena segundos inicial
80 Local cMsgHTML       := ""  // Mensagem email
81
82 Default cEmp         := ""
83 Default cFil         := ""
84 Default lIsPDV       := .F.
85 Default cTpExec      := STR0003 //Schedule/Job
86
87 LjGrvLog( cFuncLog , "ID_INICIO" )
88 nSeconds := Seconds() // Armazena segundos inicial
89
90 lPDV := lIsPDV
91 cTipoExec := cTpExec
92
93 If !Empty(cEmp) .AND. !Empty(cEmp)
94     RPCSetType(3)
95     // "FRT" > Liberacao de acesso PDV cTree para o modulo FrontLoja
96     RpcSetEnv(cEmp,cFil,Nil,Nil,"FRT")
97 EndIf
98
99 MsgHTML      := "" //Zera statica
100
101 //Carrega dados da Configuracoes de comunicacao
102 aHosts := STFLoadComunic()
103
104 If Len(aHosts) > 0

```

```

105
106     For nX := 1 To Len(aHosts)
107
108         aHosts[nX][10] := STFTTestRpc( @aHosts[nX] )
109
110     Next nX
111
112     //Prepara/Formata email antes do envio
113     cMsgHTML := STFFormtMail(aHosts)
114
115     //Envia Email para responsavel
116     If Empty( cMsgHTML )
117         Conout("Não existem mensagens a enviar")
118         LjGrvLog( cFuncLog , "Não existem mensagens a enviar" )
119     Else
120
121         cDestinat := SuperGetMv("MV_LJMMAIL",",")
122
123         //Envia Email para responsavel
124         If !Empty(cDestinat) .AND. ExistFunc("STFMail")
125
126             If STFMail( STR0001 , cMsgHTML, .F. , cDestinat) //"Comparador de dicionários"
127                 lRet := .T.
128                 Conout("Email enviado com sucesso")
129                 LjGrvLog( cFuncLog , "Email enviado com sucesso" )
130                 LjGrvLog( cFuncLog , cMsgHTML )
131             Else
132                 Conout("Email não enviado, verifique as configurações")
133                 LjGrvLog( cFuncLog , "Email não enviado, verifique as configurações" )
134                 LjGrvLog( cFuncLog , cMsgHTML )
135             EndIf
136         Else
137             If Empty(cDestinat)
138                 Conout("Destinatários não informados no parâmetro MV_LJMMAIL")
139                 LjGrvLog( cFuncLog , "Destinatários não informados no parâmetro MV_LJMMAIL" )
140             Else
141                 Conout("Função STFMail não encontrada no ambiente. Favor Atualizar.")
142                 LjGrvLog( cFuncLog , "Função STFMail não encontrada no ambiente. Favor Atualizar." )
143             EndIf
144         EndIf
145     EndIf
146
147
148 EndIf
149
150
151 //Grava tempo de execução apenas no Log
152 LjGrvLog(cFuncLog , "Tempo total de análise realizada pelo Comparador de Bases: " + AllTrim(Str(Seconds() - nSeconds) ) + " segundos.")
153 LjGrvLog( cFuncLog , "ID_FIM" )
154
155 Return lRet
156

```

```

157
158 //-----
159 /*{Protheus.doc} STFTTestRpc
160 Realiza Analise diretamente nos hosts via RPC
161
162 @param aHosts    // Array de configuracoes
163
164 @return lConect - Retorna se conectou ou nao
165 @obs
166 @sample
167 */
168 //-----
169 Static Function STFTTestRpc( aHosts )
170
171 Local aArea      := GetArea()    // Armazena alias corrente
172 Local aRet       := {}          // Array de retorno da funcao
173 Local oServer    := Nil         // Objeto que chama Classe de RPC
174 Local cEmpresa   := ""         // Empresa
175 Local cFilHost   := ""         // Filial do Host
176 Local nPorta     := 0          // Porta Rpc do Host
177 Local cAmb       := ""         // Ambiente do Host
178 Local cIPHost    := ""         // IP do Host
179 Local nPosSX     := 0          // Posicao do campo
180 Local nY         := 0          // Contador
181 Local nX         := 0          // Contador
182 Local nZ         := 0          // Contador
183 Local aSX3Ret    := {}         // Campos busca Retaguarda
184 Local aRetHost   := {}         // Retorno do Host
185 Local cValid     := ""         // Valores da validacao opcional
186 Local cHost_M_EMP := ""         // Modo tabela Host
187 Local cHost_M_UN  := ""         // Modo tabela unidade Host
188 Local cHost_M_FIL := ""         // Modo tabela empresa Host
189 Local cLoc_M_EMP  := ""         // Modo tabela Local
190 Local cLoc_M_UN   := ""         // Modo tabela unidade Local
191 Local cLoc_M_FIL  := ""         // Modo tabela empresa Local
192 Local cCampo      := ""         // Campo para analise
193 Local xValHost    := Nil        // Valor campo no Host para comparacao
194 Local xValLocal   := Nil        // Valor campo Local para comparacao
195 Local lConect     := .F.        // Retorno de RPC fez comunicação
196
197 Default aHosts    := {}         // Array de configuracoes
198
199 cEmpresa   := aHosts[01]
200 cFilHost   := aHosts[02]
201 nPorta     := Val( aHosts[05] )
202 cAmb       := aHosts[04]
203 cIPHost    := AllTrim(aHosts[06])
204
205 //Campos que serao retornados dos Hosts
206 If Len(aSX2) <= 0
207     aSX2 := {"X2_CHAVE", "X2_MODAL", "X2_MODALUN", "X2_MODALEMP"}
208 EndIf

```

```

209
210 If Len(aSX3) <= 0
211     aSX3 := {"X3_ARQUIVO", "X3_CAMPO", "X3_TIPO", "X3_TAMANHO", "X3_DECIMAL", "X3_PICTURE", "X3_VALID", "X3_VLDUSER", "X3_USADO", "X3_RELACAO", "X3_CBOX"}
212 EndIf
213
214 If Len(aTables) <= 0
215     aTables := {"SL1", "SL2", "SL4", "SFI", "SB1", "SB0", "DA0", "DA1", "SBI", "SLK", "SA1", "SA3", "SAE", "SE4", "SF4", "SF7", "SA6", "SLF", "SLG"}
216 EndIf
217
218 //Abre Conexao com o Host destino
219 If !Empty(cAmb) .AND. !Empty(cIPHost)
220     oServer := TRPC():New( cAmb )
221     If oServer:Connect( cIPHost, nPorta )
222
223         LjGrvLog( cFuncLog , "Conectou no Ambiente: " + cAmb + " IP: " + cIPHost + " Porta:" + aHosts[05])
224
225         //Prepara Amb diretamente no Host Conectado
226         oServer:CallProc("RPCSetType", 3 )
227         lConect := oServer:CallProc("RPCSetEnv", cEmpresa, cFilHost,, 'FRT')
228         lConect := IIF(ValType(lConect) == "L", lConect, .F.)
229
230         If lConect
231             LjGrvLog( cFuncLog , "Ambiente preparado com sucesso no Host. Empresa: " + cEmpresa + ", Filial: " + cFilHost )
232         Else
233             LjGrvLog( cFuncLog , "Não conseguiu preparar ambiente no Host. Empresa: " + cEmpresa + ", Filial: " + cFilHost )
234         EndIf
235     Else
236         LjGrvLog( cFuncLog , "Não Conectou no Ambiente: " + cAmb + " IP: " + cIPHost + " Porta:" + aHosts[05])
237         aHosts[7] := .F. // 07 - Status conexao
238     EndIf
239
240 EndIf
241
242 If lConect
243
244     LjGrvLog( cFuncLog , "[V] Conectou na Filial: " + cFilHost + " , Empresa: " + cEmpresa + " , Ambiente: " + cAmb + " IP: " + cIPHost + " Porta:" + aHosts[05] )
245     aHosts[7] := lConect // 07 - Status conexao
246
247     //Manter FindFunction mesmo para versao P12 pois ExistFunc da erro em alguns Bin/Lib
248     If oServer:CallProc("FindFunction", "STFPesqTab" )
249
250         For nY := 1 To Len(aTables)
251
252             //=====
253             //Realiza Busca SX2
254             aRetHost := {}
255             aRetHost := oServer:CallProc("STFPesqTab", "SX2", 1 , "SX2->X2_CHAVE" , aTables[nY] , aSX2 , .F. , "" )
256
257             If ValType(aRetHost) == "A" .AND. Len(aRetHost) > 0
258
259                 For nX := 1 To Len(aRetHost)
260

```

```

261      cHost_M_EMP      := aRetHost[nX][Ascan( aRetHost[1] , {|x| x[1] == "X2_MOD0EMP"})][2]
262      cHost_M_UN       := aRetHost[nX][Ascan( aRetHost[1] , {|x| x[1] == "X2_MOD0UN"})][2]
263      cHost_M_FIL      := aRetHost[nX][Ascan( aRetHost[1] , {|x| x[1] == "X2_MOD0"})][2]
264
265      cLoc_M_EMP       := FwModeAccess(aTables[nY],1)//1=Empresa
266      cLoc_M_UN        := FwModeAccess(aTables[nY],2)//2=Unidade de Negócio
267      cLoc_M_FIL       := FwModeAccess(aTables[nY],3)//3=Filial
268
269      //Validacao compartilhamento
270      If cHost_M_EMP <> cLoc_M_EMP
271          Aadd( aRet , {aTables[nY] , "X2_MOD0EMP", "X2_MOD0EMP", cHost_M_EMP , cLoc_M_EMP } )
272      EndIf
273
274      If cHost_M_UN <> cLoc_M_UN
275          Aadd( aRet , {aTables[nY] , "X2_MOD0UN", "X2_MOD0UN", cHost_M_UN , cLoc_M_UN } )
276      EndIf
277
278      If cHost_M_FIL <> cLoc_M_FIL
279          Aadd( aRet , {aTables[nY] , "X2_MOD0", "X2_MOD0", cHost_M_FIL , cLoc_M_FIL } )
280      EndIf
281
282
283      Next nX
284
285      Else
286          LjGrvLog( cFuncLog , "Não conseguiu extrair informações da SX2 no Host solicitado. Alias: " + aTables[nY] )
287      EndIf
288
289      //=====
290      //Realiza Busca SX3
291      aRetHost := {}
292      aRetHost := oServer:CallProc("STFPesqTab","SX3", 1 , "SX3->X3_ARQUIVO" , aTables[nY] , aSX3 , .F. , "" )
293
294      If ValType(aRetHost) == "A" .AND. Len(aRetHost) > 0
295
296          //Busca campos na retaguarda para comparacao
297          aSX3Ret := STFPesqTab( "SX3" , 1 , "SX3->X3_ARQUIVO" , aTables[nY] , ;
298                                aSX3 , .F. , "" )
299
300          For nX := 1 To Len(aRetHost)
301
302              cCampo := AllTrim(aRetHost[nX][Ascan( aRetHost[nX] , {|x| x[1] == "X3_CAMPO"})][2])
303
304              If !Empty(cCampo)
305
306                  nPosSX := Ascan( aSX3Ret , {|x| AllTrim(x[2][2]) == cCampo })
307                  If nPosSX > 0
308
309                      //Compara Base Local x Host
310                      For nZ := 1 To Len(aSX3)
311
312                          //Conteudo no Host

```

```

313         xValHost := aRetHost[nX][Ascan( aRetHost[nX] , {|x| x[1] == aSX3[nZ] })][2]
314
315         //Conteudo Local
316         xValLocal := aSX3Ret[nPosSX][Ascan( aSX3Ret[nPosSX] , {|x| x[1] == aSX3[nZ] })][2]
317
318         If( ValType(xValHost) == "N" )
319             xValHost := STR(xValHost)
320         EndIf
321
322         If( ValType(xValLocal) == "N" )
323             xValLocal := STR(xValLocal)
324         EndIf
325
326         If xValHost <> xValLocal
327             //Campo divergente
328             Add( aRet , { aTables[nY] ,cCampo , aSX3[nZ] , AllTrim(xValHost) ,AllTrim(xValLocal) } )
329         EndIf
330
331     Next nZ
332
333     Else
334         //Campo nao existe na Base Local somente no Host
335         Add( aRet , {aTables[nY] ,cCampo , "X3_CAMPO" , STR0021 ,STR0022 } ) // "Ok" "Campo não existe"
336     EndIf
337 Else
338     LjGrvLog( cFuncLog , "Campo retornou vazio, verificar na SX2/SX3 Alias: " + aTables[nY] )
339 EndIf
340
341 Next nX
342
343 For nX := 1 To Len(aSX3Ret)
344
345     cCampo := AllTrim(aSX3Ret[nX][Ascan( aSX3Ret[nX] , {|x| x[1] == "X3_CAMPO"})][2])
346
347     If !Empty(cCampo)
348
349         If Ascan( aRetHost , {|x| AllTrim(x[2][2]) == cCampo } ) <= 0
350             //Campo nao existe no Host somente na base Local
351             Add( aRet , {aTables[nY] ,cCampo,"X3_CAMPO",STR0022 ,STR0021 } ) // "Campo não existe" "Ok"
352         EndIf
353     EndIf
354
355 Next nX
356
357 Else
358     LjGrvLog( cFuncLog , "Não conseguiu extrair informações da SX3 no Host solicitado. Alias: " + aTables[nY] )
359 EndIf
360
361 Next nY
362
363 Else
364     Conout("Host não preparado para executar função STFPesqTab. Atualizar Host. ")

```



```

365      LjGrvLog( cFuncLog , "Host não preparado para executar função STFPesqTab. Atualizar. " )
366      EndIf
367 Else
368      LjGrvLog( cFuncLog , "[X] Não Conectou na Filial: " + cFilHost + " , Empresa: " + cEmpresa + " , Ambiente: " + cAmb + " IP: " + cIPHost + " Porta:" + aHosts[05])
369      aHosts[7] := .F.      // 07 - Status conexao
370 EndIf
371
372
373 If lConect
374      oServer:CallProc("RpcClearEnv") //Limpa Thread do server conectado
375      oServer:Disconnect()//Disconecta do server
376      FreeObj(oServer)//Limpa Obj
377 EndIf
378
379 RestArea(aArea)
380
381 Return aRet
382
383
384 //-----
385 /*{Protheus.doc} STFLoadComunic
386 Retorna Configuracoes de comunicacao
387
388 @param
389 @return aRet - Retorna configuracao de comunicacao disponivel
390 @obs
391 @sample
392 */
393 //-----
394 Static Function STFLoadComunic()
395
396 Local   aArea      := GetArea()      // Armazena alias corrente
397 Local   aRet       := {}            // Retorno
398
399 LjGrvLog( cFuncLog , "Carrega Configurações: MD4,MD3. " + IIF(!PDV,"PDV.,"RET."))
400
401 DbSelectArea( "MD4" )
402 DbSetOrder(1)//MD4_FILIAL+MD4_CODIGO
403 If MD4->(DbSeek(xFilial("MD4")))
404
405     While MD4->MD4_FILIAL == xFilial( "MD4" ) .AND. MD4->(!EOF())
406
407         //If !Empty(MD4->MD4_AMBPAI)
408         DbSelectArea( "MD3" )
409         DbSetOrder( 1 )//MD3_FILIAL+MD3_CODAMB+MD3_TIPO
410         If MD3->(DbSeek( xFilial( "MD3" ) + MD4->MD4_CODIGO ))
411             While MD3->(!EOF()) .AND. MD3->MD3_CODAMB == MD4->MD4_CODIGO
412                 If AllTrim(MD3->MD3_TIPO) == "R" //Somente ambientes RPC
413
414                     If (!!PDV .AND. !Empty(MD4->MD4_AMBPAI)) .OR. (!PDV .AND. Empty(MD4->MD4_AMBPAI))
415
416                         Aadd( aRet, { MD3->MD3_EMP ,; // 01 - Empresa

```

```

417 MD3->MD3_FIL           ,; // 02 - Filial
418 MD3->MD3_CODAMB        ,; // 03 - Cod Ambiente
419 AllTrim(MD3->MD3_NOMAMB) ,; // 04 - Nome ambiente
420 MD3->MD3_PORTA          ,; // 05 - Porta RPC
421 AllTrim(MD3->MD3_IP)     ,; // 06 - Ip
422 .F.                     ,; // 07 - Status conexao
423 .F.                     ,; // 08 - Divergencia encontrada
424 ""                      ,; // 09 - OBS
425 {}                      ,; // 10 - Array de informacoes Retornadas do Host
426 AllTrim(MD3->MD3_DESCRI) } ) // 11 - Descricao
427
428 EndIf
429
430 EndIf
431 MD3->(DbSkip())
432 End
433
434 EndIf
435
436 MD4->(DbSkip())
437 End
438
439 Else
440 LjGrvLog( cFuncLog , "Registros não encontrados na tabela MD4")
441 EndIf
442
443 If Len(aRet) > 0
444 LjGrvLog( cFuncLog , "Registros encontrados nas tabelas MD3,MD4" , aRet)
445 Else
446 LjGrvLog( cFuncLog , "Registros não encontrados na tabela MD3,MD4")
447 EndIf
448
449 RestArea(aArea)
450
451 Return aRet
452
453
454 //-----
455 /*{Protheus.doc} STFFormtMail
456 Formata html para envio do email
457
458 @param aHosts - Array com as informacoes colhidas nos Hosts
459 @return cMsgHTML - html Formatado
460 @obs
461 @sample
462 */
463 //-----
464 Static Function STFFormtMail(aHosts)
465
466 Local nY := 0 //Contador
467 Local nX := 0 //Contador
468 Local cData := AllTrim(DTOC( date() ) ) //Guarda data e hora formatada

```

```

469 Local cTime      := Time() //Guarda hora formatada
470 Local cStatusHost := "" //Msg aux p compor email principal
471 Local cMsgFinal   := "" //Msg para final da mensagem
472 Local lDiverg     := .F. //Existem divergencias nas bases
473 Local cListAmbs   := "" //Lista de Ambientes analisados
474 Local nTamMaxStr  := 0 //Analisa tamanho max string
475 Local lExit       := .F. //Controla saida quando chegou no limite de Str
476 Local cSrvStrMax  := AllTrim(GetPvProfString("General","MaxStringSize","1",GetAdv97())) )
477
478 Default aHosts    := {} //array de configuracoes
479
480 LjGrvLog( cFuncLog , "Formata Html")
481
482 LjGrvLog( cFuncLog , "Tamanho Maximo da String na seção General. MaxStringSize=" + cSrvStrMax )
483 nTamMaxStr := Val(cSrvStrMax) * 1000000 //Multiplica por 1000000 para sobra um pouco para o rodapé. o limite é 1048575
484
485 //Add Cabecalho principal
486 cMsgHTML := '<table style="border-color: black; float: left;" border="0" width="800">'+;
487 '<tbody>'+;
488 '<tr style="height: 30px;">'+;
489 '<td style="text-align: center; background-color: #0070c0;" colspan="2"><span style="color: #ffffff;"><strong> ' + ;
490 STR0001 + ' </strong></span></td>'+; // "Comparador de dicionários"
491 '</tr>'+;
492 '<tr style="height: 30px;">'+;
493 '<td style="width: 200px; background-color: #d0cece;">&nbsp;' + STR0004 + '</td>'+; // "Data de Analise:"
494 '<td style="background-color: #d0cece;">&nbsp;' + cData + '</td>'+;
495 '</tr>'+;
496 '<tr style="height: 30px;">'+;
497 '<td style="width: 200px; background-color: #d0cece;">&nbsp;' + STR0005 + '</td>'+; // "Hora de Analise:"
498 '<td style="background-color: #d0cece;">&nbsp;' + cTime + '</td>'+;
499 '</tr>'+;
500 '<tr style="height: 30px;">'+;
501 '<td style="width: 200px; background-color: #d0cece;">&nbsp;' + STR0006 + '</td>'+; // "Tipo de Execucao"
502 '<td style="background-color: #d0cece;">&nbsp;' + cTipoExec + '</td>'+;
503 '</tr>'+;
504 '</tbody>'+;
505 '</table>'+;
506 '<br>'
507
508
509 For nX := 1 To Len(aHosts)
510
511 //Se nao conectou ou encontrou divergencias
512 If !aHosts[nX][7] .OR. Len(aHosts[nX][10]) > 0
513
514 lDiverg := .T.
515
516 cListAmbs += IIF(Empty(cListAmbs),"",", " ) + AllTrim(aHosts[nX][3])
517
518 // 07 - Status conexao
519 cStatusHost := IIf(aHosts[nX][7],"",STR0007) + STR0008 + aHosts[nX][6] + STR0009 + aHosts[nX][5] // " Não" "Conectado IP: " " Porta: "
520

```

```

521     cMsgHTML += '<table style="width: 800px;">'+;
522     '<tbody>'+;
523     '<tr style="height: 30px;">'+;
524     '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;</strong></span></td>'+; // "Tabela"
525     IIF(1PDV,"",STR0010) + aHosts[nX][11] + STR0011 + aHosts[nX][2] + " - " + FWFilialName(aHosts[nX][2],2) + " - " + cStatusHost + ;
526     '</strong></span></td>'+; // "PDV: "" Filial: ""
527     '</tr>'+;
528     '</tbody>'+;
529     '</table>'+;
530
531     If aHosts[nX][7] .AND. Len(aHosts[nX][10]) > 0
532
533         cMsgHTML += '<table style="width: 800px;">'+;
534         '<tbody>'+;
535         '<tr style="height: 30px;">'+;
536         '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;</strong></span></td>'+; // "Tabela"
537         '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;</strong></span></td>'+; // "Campo"
538         '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;</strong></span></td>'+; // "Dicionário"
539         '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;</strong></span></td>'+; // "Retaguarda"
540         '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;</strong></span></td>'+; // "PDV"
541         '</tr>'+;
542     EndIf
543
544 EndIf
545
546 For nY := 1 To Len(aHosts[nX][10])
547
548     cMsgHTML += '<tr style="height: 30px;">'+;
549     '<td style="text-align: center; background-color: #d0cece;">&nbsp;</td>'+; // Tabela
550     '<td style="text-align: left; background-color: #d0cece;">&nbsp;</td>'+; // Campo
551     '<td style="text-align: center; background-color: #d0cece;">&nbsp;</td>'+; // Dicionário
552     '<td style="text-align: center; background-color: #d0cece;">&nbsp;</td>'+; // PDV/Retaguarda
553     '<td style="text-align: center; background-color: #d0cece;">&nbsp;</td>'+; // Retaguarda
554     '</tr>'+;
555
556     //Análise tamanho para não estourar limite de strings
557     If Len(cMsgHTML) > nTamMaxStr
558         LjGrvLog( cFuncLog , "ID_ALERT")
559         LjGrvLog( cFuncLog , "Chegou Perto do limite da string. Processo será interrompido.")
560         lExit := .T.
561         cMsgFinal += '<br><p style="color: #ff0808;"><b> ' + STR0035 + cSrvStrMax + " Mb" + CRLF + ; // "ATENÇÃO: Processo interrompido pois foi atingido o limite da
string Padrão: "
562         STR0036 + "tdn.totvs.com" + ; // "Para mais informações consultar configuração MaxStringSize em "
563         '</b></p><br>'+;
564     Exit
565 EndIf
566
567 Next nY
568
569 cMsgHTML += '</tbody>'+;

```

```

570         '</table>'+;
571         '<br>'
572
573
574     If lExit
575         Exit //Forca saida por causa do estouro da string
576     EndIf
577
578 Next nX
579
580 If lDiverg
581     cMsgFinal += STR0017 //"Verifique a(s) divergência(s) encontrada(s)."

```

Framework/Monitor/STFSrvMonitor.prw

571 linhas após remoção de metadados de autoria/data

```

1 #Include "Protheus.ch"
2 #include "tbiconn.ch"
3 #include "STFSrvMonitor.ch"
4
5 Static cFuncLog      := "Monitor_PDV" //Identificacao da funcionalidade nos Logs
6 Static aCFG          := {}           //Array de configuracoes gerais
7
8 #DEFINE CFG_MAXREPROC      1 //Quantidade maxima de tentativa de reprocessar a venda
9 #DEFINE CFG_SHOWPDVOFF    2 //Mostra Pdvs Off-Line
10 #DEFINE CFG_STATUS        3 //Avalia vendas no PDV com estes Status
11 #DEFINE CFG_QTDDIAS       4 //Quantidade de dias a avaliar a partir da data do dia decrescente
12
13
14 //-----
15 /*{Protheus.doc} STFSrvMonitor
16 Servico para monitorar vendas represadas nos PDVs
17
18 @param  nQtdDias      Quantidade de dias a analisar
19 @return Nil
20 @obs
21 @sample
22 */
23 //-----
24 Function STFSrvMonitor( nQtdDias )
25
26 Local aMD3          := {} //array de configuracoes
27 Local nX            := 0 //Contador
28 Local lConect       := .F. //Status conexao
29 Local cDestinat     := SuperGetMv("MV_LJMMAIL", "") //Destinatários do email separado por (;) ponto e virgula
30 Local nSeconds      := 0 // Armazena segundos inicial
31 Local lDiverg       := .F. // Encontrou divergencias
32 Local aRetPdv       := {} // Retorno dos PDVs
33 Local cMsgEmail     := "" // Mensagem email
34 Local nSalesNoSend  := 0 // Quantidade de vendas nao enviadas
35 Local nSalesErro    := 0 // Quantidade de vendas com erros
36 Local cPDV         := "" // Numero do PDV
37
38 Default nQtdDias     := 0 //Quantidade de dias sobrescreve os parametro
39
40 LjGrvLog( cFuncLog , "ID_INICIO" )
41
42 nSeconds := Seconds() // Armazena segundos inicial
43
44 //Atencao - Deixar no começo da funcao
45 //Carrega configuracoes gerais
46 aCFG := STFMnitCfg()
47
48 //Carrega dados da MD3(Configuracoes de comunicacao)
49 aMD3 := STFLoadMD3()
50
51 If Len(aMD3) > 0
52

```

```

53   For nX := 1 To Len(aMD3)
54       lConect      := .F.
55       lDiverg      := .F.
56       nSalesNoSend := 0
57       nSalesErro   := 0
58       cPDV         := ""
59
60       aRetPdv := STFTTestRpc( aMD3[nX]      , @lConect , @lDiverg , @nSalesNoSend ,;
61                             @nSalesErro    , @cPDV    , nQtdDias    )
62
63       aMD3[nX][7]   := lConect      // 7 - Status conexao
64       aMD3[nX][8]   := lDiverg      // 8 - Divergencia encontrada
65       aMD3[nX][9]   := nSalesNoSend // 9 - Vendas Pendentes
66       aMD3[nX][10]  := nSalesErro   // 10 - Vendas com Erro
67       aMD3[nX][11]  := cPDV         // 11 - Numero do PDV
68       aMD3[nX][12]  := aRetPdv      // 12 - Array de vendas Retornadas do PDV
69
70   Next nX
71
72   //Prepara email formatado
73   cMsgEmail := STFFormtMail(aMD3)
74
75   //Envia Email para responsavel
76   If Empty( cMsgEmail )
77       Conout("Não existem mensagens a enviar")
78       LjGrvLog( cFuncLog , "Não existem mensagens a enviar" )
79   Else
80
81       //Envia Email para responsavel
82       If !Empty(cDestinat) .AND. ExistFunc("STFMail")
83
84           If STFMail( STR0001 , cMsgEmail, .F. , cDestinat) //"Monitor de PDVs"
85               Conout("Email enviado com sucesso")
86               LjGrvLog( cFuncLog , "Email enviado com sucesso" )
87               LjGrvLog( cFuncLog , cMsgEmail )
88           Else
89               Conout("Email não enviado, verifique as configurações")
90               LjGrvLog( cFuncLog , "Email não enviado, verifique as configurações" )
91               LjGrvLog( cFuncLog , cMsgEmail )
92           EndIf
93       Else
94           If Empty(cDestinat)
95               Conout("Destinatários não informados no parâmetro MV_LJMMAIL")
96               LjGrvLog( cFuncLog , "Destinatários não informados no parâmetro MV_LJMMAIL" )
97           Else
98               Conout("Função STFMail não encontrada no ambiente. Favor Atualizar.")
99               LjGrvLog( cFuncLog , "Função STFMail não encontrada no ambiente. Favor Atualizar." )
100           EndIf
101       EndIf
102   EndIf
103 EndIf
104

```

```

105 EndIf
106
107
108 //Grava tempo de execução apenas no Log
109 LjGrvLog(cFuncLog , "Tempo total de análise realizada pelo Monitor de PDVs: " + AllTrim(Str(Seconds() - nSeconds) ) + " segundos.")
110 LjGrvLog( cFuncLog , "ID_FIM" )
111
112 Return Nil
113
114
115 //-----
116 /*{Protheus.doc} STFFormtMail
117 Formata email para envio
118
119 @param   aMD3 - Array com as informacoes colhidas nos PDVs
120 @return  cMsgEmail - Email Formatado
121 @obs
122 @sample
123 */
124 //-----
125 Static Function STFFormtMail(aMD3)
126
127 Local cMsgEmail      := "" //Retorna Email formatado
128 Local cMsgAux        := "" //Msg aux p compor email principal
129 Local nX             := 0 //Contador
130 Local cData          := AllTrim(DTOC( date() ) ) //Guarda data e hora formatada
131 Local cTime          := Time() //Guarda data e hora formatada
132 Local cTipoExec      := STR0002 //Tipo execucao // "Menu"
133 Local cStatusPdv     := "" //Msg aux p compor email principal
134
135 Default aMD3         := {} //array de configuracoes
136
137 //Verifica se é Schedule/Job
138 If IsBlind()
139     cTipoExec := STR0003 //Schedule/Job
140 EndIf
141
142 For nX := 1 To Len(aMD3)
143
144     If aMD3[nX][7] // 7 - Status conexao
145         cStatusPdv := STR0004 + aMD3[nX][6] + STR0006 + aMD3[nX][5] //"Conectado Ip: " // " Porta: "
146     Else
147         cStatusPdv := STR0005 + aMD3[nX][6] + STR0006 + aMD3[nX][5] //"Não Conectado Ip: " // " Porta: "
148     EndIf
149
150     //Se tem divergencia ou nao conseguiu conectar
151     If aMD3[nX][8] .OR. (!aMD3[nX][7] .AND. aCfg[CFG_SHOWPDVOFF] == "S") // 8 - Divergencia encontrada // 7 - Status conexao
152
153         cMsgAux += '<tr style="height: 30px;">'+;
154                 '<td style="text-align: center; background-color: #d0cece;">&nbsp;' + aMD3[nX][2] + '</td>'+; // 'Código da Filial'
155                 '<td style="text-align: center; background-color: #d0cece;">&nbsp;' + FWFilialName(,aMD3[nX][2],2) + '</td>'+; // Nome da Filial
156                 '<td style="text-align: center; background-color: #d0cece;">&nbsp;' + aMD3[nX][11] + '</td>'+; //'Código do Pdv'

```



```

157         '<td style="text-align: center; background-color: #d0cece;">&nbsp;' + cStatusPdv + '</td>'+; //'Status da Conexão'
158         '<td style="text-align: center; background-color: #d0cece;">&nbsp;' + Alltrim(STR(aMD3[nX][9]+aMD3[nX][10])) + '</td>'+; //'Vendas pendentes no PDV'
159         '</tr>'
160     EndIf
161
162 Next nX
163
164 //Se Encontrou mensagem de mail a enviar formata
165 //cabecalho junto do corpo do email
166 If !Empty(cMsgAux)
167
168     //Add Cabecalho principal
169     cMsgEmail := '<table style="border-color: black; float: left;" border="0" width="800">'+;
170     '<tbody>'+;
171     '<tr style="height: 30px;">'+;
172     '<td style="text-align: center; background-color: #0070c0;" colspan="2"><span style="color: #ffffff;"><strong> ' + STR0007 + ' </strong></span></td>'+;
173     // "Análise de Processamento de Vendas"
174     '</tr>'+;
175     '<tr style="height: 30px;">'+;
176     '<td style="width: 200px; background-color: #d0cece;">&nbsp;' + STR0008 + '</td>'+; // "Data de Análise:"
177     '<td style="background-color: #d0cece;">&nbsp;' + cData + '</td>'+;
178     '</tr>'+;
179     '<tr style="height: 30px;">'+;
180     '<td style="width: 200px; background-color: #d0cece;">&nbsp;' + STR0009 + '</td>'+; // "Hora de Análise:"
181     '<td style="background-color: #d0cece;">&nbsp;' + cTime + '</td>'+;
182     '</tr>'+;
183     '<tr style="height: 30px;">'+;
184     '<td style="width: 200px; background-color: #d0cece;">&nbsp;' + STR0010 + '</td>'+; // "Tipo de Execução"
185     '<td style="background-color: #d0cece;">&nbsp;' + ctipoExec + '</td>'+;
186     '</tr>'+;
187     '</tbody>'+;
188     '</table>'+;
189     '<br>'+;
190     '<table style="width: 800px;">'+;
191     '<tbody>'+;
192     '<tr style="height: 30px;">'+;
193     '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;' + STR0011 + '</strong></span></td>'+;
194     // "Código da Filial"
195     '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;' + STR0012 + '</strong></span></td>'+;
196     // "Nome da filial"
197     '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;' + STR0013 + '</strong></span></td>'+;
198     // "Código do Pdv"
199     '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;' + STR0014 + '</strong></span></td>'+;
200     // "Status da Conexão"
201     '<td style="text-align: center; background-color: #0070c0;"><span style="color: #ffffff;"><strong>&nbsp;' + STR0015 + '</strong></span></td>'+;
202     // "Vendas pendentes no PDV"
203     '</tr>'
204
205     cMsgEmail += cMsgAux +;
206     '</tbody>'+;
207     '</table>'+;
208     '<br>'+;

```

```

203      IIf(!Empty(aCfg[CFG_STATUS]), STR0016 + aCfg[CFG_STATUS], STR0017 ) //"Verifique nestes PDVs vendas com status cujo campo L1_SITUA = " ### //"Verifique as
      vendas nestes PDVs "
204
205 EndIf
206
207 Return cMsgEmail
208
209
210 //-----
211 /*{Protheus.doc} STFTestRpc
212 Realiza Analise diretamente nos PDVs via RPC
213
214 @param aConfigPdv // Array de configuracoes
215 @param lConect // Retorno de RPC fez comunicação
216 @param lDiverg // Encontrou divergencias
217 @param nSalesNoSend // Quantidade de vendas nao enviadas
218 @param nSalesErro // Quantidade de vendas com erros
219 @param cPDV // Numero do PDV
220 @param nQtdDias // Quantidade de dias a analisar
221
222 @return lConect - Retorna se conectou ou nao
223 @obs
224 @sample
225 */
226 //-----
227 Static Function STFTestRpc( aConfigPdv , lConect , lDiverg , nSalesNoSend ,;
228                          nSalesErro , cPDV , nQtdDias )
229
230 Local aArea := GetArea() // Armazena alias corrente
231 Local oServer := Nil // Objeto que chama Classe de RPC
232 Local cEmpresa := "" // Empresa
233 Local cFilPDV := "" // Filial do PDV
234 Local nPorta := 0 // Porta Rpc do PDV
235 Local cAmb := "" // Ambiente PDV
236 Local cIPPdv := "" // IP do PDV
237 Local aStruSL1Pdv := {} // Estrutura da Tabela SL1 do PDV
238 Local nPosL1Fil := 0 // Posicao Filial do SL1
239 Local nPosL1Num := 0 // Posicao Num do SL1
240 Local nPosL1Pdv := 0 // Posicao PDV do SL1
241 Local nPosL1Situa := 0 // Posicao Situa do SL1
242 Local nPosL1Doc := 0 // Posicao Doc do SL1
243 Local nPosL1Serie := 0 // Posicao Serie do SL1
244 Local nPosL1Cliente := 0 // Posicao Cliente do SL1
245 Local nPosL1VlrLiq := 0 // Posicao Valor Liquido do SL1
246 Local cL1Serie := "" // serie da Venda do PDV
247 Local cL1Doc := "" // Doc da Venda do PDV
248 Local cL1Num := "" // Num da Venda do PDV
249 Local cL1Situa := "" // Situa da Venda do PDV
250 Local cL1Cliente := "" // Cliente da Venda do PDV
251 Local nL1VlrLiq := 0 // Valor Liquido da Venda do PDV
252 Local aL1Error := {} // Array vendas com erros marcar p reprocessar
253 Local aL1Pendentes := {} // Array vendas pendentes

```

```

254 Local nX           := 0           // Contador
255 Local nY           := 0           // Contador
256 Local cSituaReproc := "00"       // Situa para Reprocessar
257 Local cMsgAux      := ""         // Mensagem auxiliar para compor Email principal
258 Local nSalesReproc := 0           // Quantidade de vendas marcadas para reprocessar
259 Local cPictVlrLiq   := PesqPict("SL1","L1_VLRLIQ")//Picture do valor liquido
260 Local cMoedaSimb    := SuperGetMV( "MV_SIMB" + Str(1,1) ) //Simbolo da Moeda
261 Local aTables       := {"SL1"}    // Tabelas abertas por RPC
262 Local aSL1          := {}         // Campos da SL1 que ira trazer na busca
263 Local aRetServer    := {}         // Retorno do Servidor
264 Local cNso           := ""         // Usamos o campo L1_NS0 para gravar qtd de tentativas de reprocessamento
265 Local nMaxReproc    := Val(aCfg[CFG_MAXREPROC]) // Numero maximo de tentativas de reprocessar venda
266 Local cStatusVenda := aCfg[CFG_STATUS] // Estatus de vendas que se quer buscar no PDV
267 Local cValid        := ""         // Valores da validacao opcional
268 Local cData         := ""         // Data a ser analisada
269
270 Default aConfigPdv   := {}         // Array de configuracoes
271 Default lConect       := .F.       // Retorno de RPC fez comunicação
272 Default lDiverg      := .F.       // Encontrou divergencias
273 Default nSalesNoSend := 0           // Quantidade de vendas nao enviadas
274 Default nSalesErro   := 0           // Quantidade de vendas com erros
275 Default cPDV         := ""         // Numero do PDV
276 Default nQtdDias     := 0           // Quantidade de Dias para Analise
277
278
279 cEmpresa := aConfigPdv[01]
280 cFilPDV  := aConfigPdv[02]
281 nPorta   := Val( aConfigPdv[05] )
282 cAmb     := aConfigPdv[04]
283 cIPPdv    := AllTrim(aConfigPdv[06])
284
285 //Valida o periodo recebido
286 If nQtdDias < 1 .OR. nQtdDias > 90
287     nQtdDias := Val(aCfg[CFG_QTDDIAS]) //quantidade padrao do Param
288     //Garante que o usuario nao preencheu parametro correto
289     If nQtdDias < 1 .OR. nQtdDias > 90
290         nQtdDias := 1
291     EndIf
292 EndIf
293
294 //Campos da SL1 que serao retornados dos PDVs
295 aSL1 := {"L1_FILIAL","L1_NUM","L1_DOC","L1_SERIE","L1_PDV","L1_SITUA","L1_CLIENTE","L1_VLRLIQ","L1_NS0"}
296
297 //Abre Conexao com o PDV destino
298 oServer := TRPC():New( cAmb )
299 lConect := oServer:Connect( cIPPdv, nPorta )
300
301 If lConect
302
303     LjGrvLog( cFuncLog ,"[V] Conectou na Filial: " + cFilPDV + ", Ambiente: " + cAmb )
304
305     If !Empty(cStatusVenda)

```

```

306         cValid := "SL1->L1_SITUA $ "+ "" + cStatusVenda + ""
307     EndIf
308
309     //Prepara Amb diretamente no PDV Conectado
310     oServer:CallProc("RPCSetType", 3 )
311     oServer:CallProc("RPCSetEnv", cEmpresa, cFilPDV,,,'FRT',,{"SL1"})//aTables
312
313     If oServer:CallProc("ExistFunc", "STFPesqTab" )
314         //Busca vendas
315         For nY := nQtdDias To 1 Step -1
316
317             cData := DTOS( date() - (nY-1) )
318             aRetServer := oServer:CallProc("STFPesqTab","SL1", 7 , "SL1->L1_FILIAL + DTOS(SL1->L1_EMISSAO)",xFilial("SL1") + cData , aSL1 , .F. , cValid )
319
320             If ValType(aRetServer) == "A" .AND. Len(aRetServer) > 0
321
322                 dbSelectarea("SL1")
323                 dbSetorder(2)//L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
324
325                 For nX := 1 To Len(aRetServer)
326
327                     cFilPDV      := aRetServer[nX][Ascan( aRetServer[1] , {|x| x[1] == "L1_FILIAL"})][2]
328                     cL1Serie     := aRetServer[nX][Ascan( aRetServer[1] , {|x| x[1] == "L1_SERIE"})][2]
329                     cL1Doc       := aRetServer[nX][Ascan( aRetServer[1] , {|x| x[1] == "L1_DOC"})][2]
330                     cL1Num       := aRetServer[nX][Ascan( aRetServer[1] , {|x| x[1] == "L1_NUM"})][2]
331                     cL1Situa     := aRetServer[nX][Ascan( aRetServer[1] , {|x| x[1] == "L1_SITUA"})][2]
332                     cL1Cliente  := aRetServer[nX][Ascan( aRetServer[1] , {|x| x[1] == "L1_CLIENTE"})][2]
333                     nL1VlrLiq   := aRetServer[nX][Ascan( aRetServer[1] , {|x| x[1] == "L1_VLR LIQ"})][2]
334                     cPDV        := aRetServer[nX][Ascan( aRetServer[1] , {|x| x[1] == "L1_PDV"})][2]
335                     cNso        := AllTrim(aRetServer[nX][Ascan( aRetServer[1] , {|x| x[1] == "L1_NS0"})][2])
336
337                     If cL1Situa <> "07"
338
339                         //Se Nao encontrar venda
340                         If !SL1->( DbSeek( cFilPDV + cL1Serie + cL1Doc ) ) //Nao considera cancelamento "07"
341
342                             lDiverg := .T. //Marca que encontrou divergencias
343
344                             //Guarda vendas com erro para remarcar para reprocessar
345                             If cL1Situa == "ER"
346                                 nSalesErro ++
347                                 Aadd( aL1Error ,;
348                                     { cFilPDV ,; // 1
349                                       cL1Num ,; // 2
350                                       cL1Serie ,; // 3
351                                       cL1Doc ,; // 4
352                                       cL1Situa ,; // 5
353                                       cL1Cliente ,; // 6
354                                       nL1VlrLiq ,; // 7
355                                       cNso } ) // 8
356                             Else
357

```

```

358             nSalesNoSend ++
359
360             Aadd(   aLlPendentes      ,;
361                   {   cFilPDV          ,; // 1
362                       cLlNum           ,; // 2
363                       cLlSerie         ,; // 3
364                       cLlDoc           ,; // 4
365                       cLlSitua         ,; // 5
366                       cLlCliente       ,; // 6
367                       nLlVlrLiq        } ) // 7
368
369             EndIf
370         EndIf
371
372         EndIf
373
374         Next nX
375
376         Else
377             //Nao Conectou no ambientes especificado
378             LjGrvLog( cFuncLog , "Não existem informações na data solicitado " + cData )
379         EndIf
380
381         Next nY
382     Else
383         Conout("PDV não preparado para executar função STFPesqTab. Atualizar. ")
384         LjGrvLog( cFuncLog , "PDV não preparado para executar função STFPesqTab. Atualizar. " )
385     EndIf
386 Else
387     LjGrvLog( cFuncLog , "[X] Não Conectou na Filial: " + cFilPDV + ", Ambiente: " + cAmb )
388 EndIf
389
390 // Vendas com Erro Marca para reprocessar diretamente no PDV
391 If Len(aLlError) > 0 .AND. nMaxReproc > 0
392
393     cMsgAux := ""
394
395     For nX := 1 To Len(aLlError)
396
397         cNso := aLlError[nX][8]
398         IIf( Empty(cNso) , cNso := "1" , cNso := Soma1(cNso) )
399
400         //Busca a venda no PDV e alterar para reprocessar
401         If Val(cNso) <= nMaxReproc .AND. oServer:CallProc("STFAltTab", "SL1" , 2 , aLlError[nX][1] + aLlError[nX][3] + aLlError[nX][4] , { {"L1_SITUA", "00"}, {"L1_NS0", cNso}
402     } )
403
404         nSalesReproc ++
405         If Val(cNso) < 1
406             nSalesErro --
407         EndIf
408
409         //Mensagem apenas para o Log

```

```

409         cMsgAux += "A venda de número: " + aLError[nX][2] + " da Filial: " + aLError[nX][1] +;
410         " Serie: " + aLError[nX][3] + " Doc: " + aLError[nX][4] +;
411         " Cliente: " + aLError[nX][6] +;
412         " Valor: " + cMoedaSimb+" "+AllTrim(Transform(aLError[nX][7],cPictVlrLiq))+;
413         " . L1_SITUA Alterado de " + aLError[nX][5] + " para 00." + CRLF
414
415     EndIf
416
417 Next nX
418
419 //Se encontrou vendas no PDV que não existem
420 //na retaguarda add msg de email
421 If nSalesReproc > 0
422     LjGrvLog( cFuncLog , cMsgAux )
423 EndIf
424
425 EndIf
426
427 If lConect
428
429     oServer:CallProc("RpcClearEnv") //Limpa Thread do server conectado
430     oServer:Disconnect()//Disconecta do server
431     FreeObj(oServer)//Limpa Obj
432
433 //Se nao houve divergencia
434 If !lDiverg
435     LjGrvLog( cFuncLog , "PDV Filial: " + cFilPDV + " Sem divergência." )
436 EndIf
437
438 EndIf
439
440 RestArea(aArea)
441
442 Return aLlPendentes
443
444
445 //-----
446 /*/{Protheus.doc} STFLoadMD3
447 Retorna Configuracoes de comunicacao Alias MD3
448
449 @param
450 @return aRet - Retorna configuracao dos PDVs disponiveis
451 @obs
452 @sample
453 */
454 //-----
455 Static Function STFLoadMD3()
456
457 Local aArea := GetArea() // Armazena alias corrente
458 Local aRet := {} // Retorno
459
460 LjGrvLog( cFuncLog , "Carrega PDVs: MD4,MD3")

```

```

461
462 DbSelectArea( "MD4" )
463 DbSetOrder(1)//MD4_FILIAL+MD4_CODIGO
464 If MD4->(DbSeek(xFilial("MD4")))
465
466     While MD4->MD4_FILIAL == xFilial( "MD4" ) .AND. MD4->(!EOF())
467
468         If !Empty(MD4->MD4_AMBPAI)
469             DbSelectArea( "MD3" )
470             DbSetOrder( 1 )//MD3_FILIAL+MD3_CODAMB+MD3_TIPO
471             If MD3->(DbSeek( xFilial( "MD3" ) + MD4->MD4_CODIGO ))
472                 While MD3->(!EOF()) .AND. MD3->MD3_CODAMB == MD4->MD4_CODIGO
473                     If AllTrim(MD3->MD3_TIPO) == "R" //Somente ambientes RPC
474
475                         Aadd( aRet, { MD3->MD3_EMP      ,; // 1 - Empresa
476                                     MD3->MD3_FIL        ,; // 2 - Filial
477                                     MD3->MD3_CODAMB      ,; // 3 - Cod Ambiente
478                                     AllTrim(MD3->MD3_NOMAMB),; // 4 - Nome ambiente
479                                     MD3->MD3_PORTA       ,; // 5 - Porta RPC
480                                     AllTrim(MD3->MD3_IP)  ,; // 6 - Ip
481                                     .F.                 ,; // 7 - Status conexao
482                                     .F.                 ,; // 8 - Divergencia encontrada
483                                     .F.                 ,; // 9 - Divergencia encontrada
484                                     .F.                 ,; // 10 - Divergencia encontrada
485                                     ""                  ,; // 11 - Numero do PDV
486                                     {}                  } ) // 12 - Array de vendas Retornadas do PDV
487
488                             EndIf
489                             MD3->(DbSkip())
490                         End
491                     EndIf
492                 EndIf
493             MD4->(DbSkip())
494         End
495     End
496 Else
497     LjGrvLog( cFuncLog , "Registros não encontrados na tabela MD4")
498 EndIf
499
500
501 If Len(aRet) > 0
502     LjGrvLog( cFuncLog , "Registros encontrados nas tabelas MD3,MD4" , aRet)
503 Else
504     LjGrvLog( cFuncLog , "Registros não encontrados na tabela MD3,MD4")
505 EndIf
506
507 RestArea(aArea)
508
509 Return aRet
510
511
512 //-----

```

```

513 /*/{Protheus.doc} STFMonitCfg
514 Retorna Configuracoes gerais do Monitor
515
516 @param
517 @return aRet - Retorna configuracao gerais
518 @obs
519 @sample
520 */
521 //-----
522 Static Function STFMonitCfg()
523
524 Local nTamArray := 4 // Define o tamanho do array de configuracoes, deixar essa como primeira variavel
525 Local aRet := Array(nTamArray) // Retorno
526 Local cCfg := AllTrim(SuperGetMv("MV_LJMOCFG", "")) // Configuracoes do Monitor tamanho max param 250 caracteres
527 Local aParam := {} // Guarda informacoes do parametro quebrado em array por ponto e virgula
528 Local lParamOK := .F. // Valida se os parametros estao integros
529
530 LjGrvLog( cFuncLog , "Configurações MV_LJMOCFG: ", cCfg)
531
532 If Len(aRet) >= nTamArray //Respeitar sempre o tamanho da declaração nTamArray
533
534 If !Empty(cCfg)
535     aParam := STRTOKARR(cCfg, ";")
536     lParamOK := ValType(aParam) == "A" .AND. Len(aParam) >= nTamArray
537 EndIf
538
539 If !lParamOK .OR. Empty(cCfg)
540     LjGrvLog( cFuncLog , "MV_LJMOCFG não reipeitou tamanho minimo de " + Alltrim(STR(nTamArray)) + "parametros será redefinido para 0;S;ER|DU;1")
541 EndIf
542
543 //Carrega Parametros Padroes
544 aRet[CFG_MAXREPROC] := IIf(lParamOK, aParam[CFG_MAXREPROC] , "0") //01 - Quantidade maxima de tentativa de reprocessar a venda
545 aRet[CFG_SHOWPDVOFF] := IIf(lParamOK, aParam[CFG_SHOWPDVOFF] , "S") //02 - Mostra Pdvs Off-Line ou nao preparados no email
546 aRet[CFG_STATUS] := IIf(lParamOK, aParam[CFG_STATUS] , "ER|DU") //03 - Avalia vendas no PDV com estes Status
547 aRet[CFG_QTDDIAS] := IIf(lParamOK, aParam[CFG_QTDDIAS] , "1") //04 - Quantidade de dias a avaliar a partir da data do dia decrescente
548
549 EndIf
550
551
552 Return aRet
553
554
555 //-----
556 /*/{Protheus.doc} SchedDef
557 Retorna as perguntas definidas no schedule.
558 @return aReturn Array com os parametros
559 */
560 //-----
561 Static Function SchedDef()
562
563 Local aParam := {}
564

```



```
565 aParam := { "P",;           //Tipo R para relatorio P para processo
566             "ParamDef",;      //Pergunte do relatorio, caso nao use passar ParamDef
567             ,;                //Alias
568             ,;                //Array de ordens
569             }                  //Titulo
570
571 Return aParam
```

Framework/Payment/STFPayment.prw

41 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STFUseCmc7
6 Retorna se utiliza leitura de CMC7
7
8 @param
9 @return lRet - Retorna se utiliza leitura de CMC7
10 @obs
11 @sample
12 */
13 //-----
14 Function STFUseCmc7()
15
16 Local lRet := .F. //Retorno
17
18 aRet := STFFireEvent( ProcName(0), "STCMC7Use", {} )
19 lRet := Len(aRet) > 0 .AND. ValType(aRet[1]) == "L" .AND. aRet[1]
20
21 Return lRet
22
23
24 //-----
25 /*{Protheus.doc} STFUsePrtCh
26 Retorna se utiliza impressao de cheque
27
28 @param
29 @return lRet - Retorna se utiliza impressao de cheque
30 @obs
31 @sample
32 */
33 //-----
34 Function STFUsePrtCh()
35
36 Local lRet := .F. //Retorno
37
38 aRet := STFFireEvent( ProcName(0), "STCheckUse", {} )
39 lRet := Len(aRet) > 0 .AND. ValType(aRet[1]) == "L" .AND. aRet[1]
40
41 Return lRet

```

Framework/Profile/STFProfile.PRW

597 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STFPROFILE.CH"
4
5 /*/
6     Defines para tratamento do Array aRet
7 /*/
8 #DEFINE RETACCESS    01
9 #DEFINE SUPNAME      02
10
11 Static lNoAccess := .F. //Indica se o não tem permissao (icone vermelho na SLF)
12
13 //-----
14 /*/{Protheus.doc} STFPROFILE
15 Function Desconto no Total da Venda
16
17 @param1    nAccess      Numero do Acesso Desejado conforme permissoes do caixa da Tabela SLF
18 @param2    cCash        Caixa a ser verificado
19 @param3    nPercDesc     Percentual de desconto a verificar permissao
20 @param4    nValueDesc    Valor do desconto a verificar permissao
21 @param5    cTypeDesc     Tipo de desconto. Se "I" - Item, se "T" - Total
22 @param6    lCashSup      Indica que o caixa passado por parâmetro já é o superior
23 @param7    lShowMessage  Indica se exibe mensagem caso acesso negado
24 @return    aRet[01]-Tem Permissao,
25 @return    aRet[02]-Nome Superior(Se precisou de autorização do superior, retorna o próprio ID do usuário)
26 @obs
27 @sample
28 /*/
29 //-----
30 Function STFPROFILE( nAccess, cCash, nPercDesc, nValueDesc, cTypeDesc, lCashSup, lShowMessage, nTotSale )
31
32 Local    lRet          := .F.          // Retorna se tem permissao
33 Local    aRet          := {}          // Array de retornos
34 Local    cAllAccess    := ""          // Conteudo do campo LF_ACESSO
35 Local    cAccess       := ""          // Armazena o acesso da posicao solicitada
36 Local    aArea         := GetArea()    // Armazena area
37 Local    aAreaSLF      := SLF->(GetArea()) // Armazena Area SLF
38 Local    cFormDesc     := SuperGetMV("MV_LJFORMD",.F., "1") // Valida so por um ou pelos dois
39 Local    cTipoDesc     := SuperGetMV("MV_LJTIPOD",.F., "1") // Valida 1- pelo % 2- pelo val
40 Local    nVlMaxDisc    := 0           // Valor Maximo de desconto permitido
41
42 Default nAccess        := 0
43 Default nPercDesc      := 0
44 Default nValueDesc     := 0
45 Default cCash          := ""
46 Default cTypeDesc      := ""
47 Default lCashSup       := .F.
48 Default lShowMessage   := .F.
49 Default nTotSale       := 0
50
51
52 If Empty( cCash )

```

```

53     cCash := xNumCaixa()
54 EndIf
55
56 DbSelectArea("SLF")
57 SLF->( DbSetOrder(1) )
58 If SLF->( DbSeek(xFilial("SLF")+cCash) )
59
60     cAllAccess := SLF->LF_ACESSO
61
62     cAccess := SubStr( cAllAccess , nAccess , 1)
63
64     // Sendo "N" eh status: Acesso Negado
65     lRet := cAccess <> "N"
66
67     If !lRet
68         lNoAccess := .F.
69     Else
70         lNoAccess := .T.
71     EndIf
72
73 Else
74
75     lRet:= .F.
76     STFMessage("STFPROFILE","STOP",STR0001) //"Caixa nao encontrado"
77
78 EndIf
79
80 // Nao sendo Acesso Negado
81 If lRet
82
83     Do Case
84
85         Case Empty( cAccess )
86
87             If nAccess == 15 .OR. nAccess == 45 // Reabertura de Caixa ou Recebimento de titulo offline
88
89                 lRet := .T.
90
91             ElseIf !( lCashSup )
92
93                 // Funcao que valida a autorizacao superior por Cartao ou Senha
94                 aRet := STFWLDSUP(      nAccess      , cCash , nPercDesc , nValueDesc      , ;
95                                     cTypeDesc
96                                     )
97
98                 lRet := aRet[RETACCESS]
99
100             EndIf
101
102         Case cAccess == "X" .AND. nAccess <> 3
103
104             If !(lCashSup)

```

```

105         // Funcao que valida a autorizacao superior por Cartao ou Senha
106         aRet := STFWLDSUP( nAccess , cCash , nPercDesc , nValueDesc , ;
107                           cTypeDesc
108                           )
109         lRet:= aRet[RETACCESS]
110
111         ElseIf nAccess == 7 .OR. nAccess == 8 .OR. nAccess == 4 .OR. nAccess == 5//7-Cancelamento de Item/8- Cancelamento de Cupom/4-Entrada e Saida de Operador/5-Sangria
e suprimento
112             // Caso o aprovador for supervisor (lCashSup == .T.), o mesmo precisa da permissão, ou seja, cAccess == S".
113             LjGrvLog( NIL, "Supervisor " + cCash + " Sem permissão para "+ STFWPermis( nAccess )[2] )
114             lRet:= .F.
115         EndIf
116
117
118         Case cAccess == "X" .AND. nAccess == 3
119
120             lRet := .F.
121
122         Case cAccess == "S"
123
124             If nAccess == 11 // Desconto
125
126                 If cTypeDesc == "I" // ITEM
127
128                     If cFormDesc == "1" // Valida ou % ou Valor
129                         If cTipoDesc == "1" // Valida %
130                             lRet := nPercDesc <= SLF->LF_DESCPER
131                         ElseIf cTipoDesc == "2" // Valida Valor
132                             lRet := nValueDesc <= SLF->LF_DESCVAL
133                         EndIf
134                     ElseIf cFormDesc == "2" // Valida os dois % e Valor
135                         lRet := ( nPercDesc <= SLF->LF_DESCPER ) .AND. ( nValueDesc <= SLF->LF_DESCVAL )
136                     EndIf
137
138                     If !lRet
139                         If !(lCashSup)
140
141                             // Funcao que valida a autorizacao superior por Cartao ou Senha
142                             aRet := STFWLDSUP( nAccess , cCash , nPercDesc , nValueDesc , ;
143                                                 cTypeDesc
144                                                 )
145                             lRet := aRet[RETACCESS]
146                         EndIf
147                     EndIf
148
149                 ElseIf cTypeDesc == "T" // TOTAL
150
151                     If cFormDesc == "1" // Valida ou % ou Valor
152                         If cTipoDesc == "1" // Valida %
153                             lRet := nPercDesc <= SLF->LF_TOTDESP
154                         EndIf
155                     If !lRet .AND. nTotSale > 0 .AND. nValueDesc > 0

```

```

156             // Verifica se a diferença do valor esta dentro na margem permitida de 1 centavo
157             nVlMaxDisc := nTotSale * ( SLF->LF_TOTDESP / 100 )
158             If nValueDesc - nVlMaxDisc <= 0.01
159                 lRet := .T.
160             EndIf
161         EndIf
162
163         ElseIf cTipoDesc == "2" // Valida Valor
164             lRet := nValueDesc <= SLF->LF_TOTDESV
165         EndIf
166     ElseIf cFormDesc == "2" // Valida os dois % e Valor
167         lRet := ( nPercDesc <= SLF->LF_TOTDESP ) .AND. ( nValueDesc <= SLF->LF_TOTDESV )
168     EndIf
169
170     If !lRet
171         If !(lCashSup)
172
173             // Funcao que valida a autorizacao superior por Cartao ou Senha
174             aRet := STFWLDSUP( nAccess , cCash , nPercDesc , nValueDesc , ;
175                             cTypeDesc
176                             )
177             lRet := aRet[RETACCESS]
178         EndIf
179     EndIf
180
181     Else
182
183         STFMessage( "STFPROFILE" , "STOP" , STR0002 ) //"Parâmetro inválido. Para permissao de desconto, passar parâmetro se TOTAL ou ITEM"
184         lRet := .F.
185     Endif
186
187     Else
188         lRet := .T.
189     EndIf
190 EndIf
191
192 EndCase
193
194 EndIf
195
196 //-----
197 // TODO:
198 // Verificar nAccess 17 - XX-> mesmo com "N"
199 // Ele faz um MsgAlert("Usuário sem autorização para efetuar Cancelamento Manual de TEF","Atenção") logo após verificar o "N".
200 // Suposição: Onde chama(LOJXTEF) nao passa .T. para mostrar mensagem, por isso dá msgalert. vEr com o TEF 2.0 como vai ser, pq no meu caso o default é .T. pra mostrar
    mensagem
201 //-----
202
203 If !lRet .AND. !IsInCallStack("LoadConfig")
204     STFMessage( "STFPROFILE" , "STOP" , STR0003 + " " + STR0004 + cCash + ; //"Acesso Negado." // "0 caixa "
205                 STR0005 + ; //" não tem permissão para executar a seguinte operação: "
206                 STFWPermis( nAccess ) [2] )

```

```

207 EndIf
208
209 /*/
210     Caso nao venha o aRet, nao precisou de superior. Apenas adiciona o retorno.
211 /*/
212 If Len(aRet) <= 0
213
214     AADD( aRet , lRet      )
215     AADD( aRet , cUserName )
216     AADD( aRet , ""       )
217
218 EndIf
219
220 If lShowMessage
221     STFSHOWMESSAGE("STFPROFILE")
222 EndIf
223
224 RestArea(aAreaSLF)
225 RestArea(aArea)
226
227 Return aRet
228
229
230 //-----
231 /*/{Protheus.doc} STFWLDSUP
232 Validação do Usuário Superior
233
234 @param  nAccess      Numero do Acesso Desejado conforme permissões do caixa da Tabela SLF
235 @param  cCash        Caixa a ser verificado
236 @param  nPercDesc    Percentual de desconto a verificar permissão
237 @param  nValueDesc    Valor do desconto a verificar permissão
238 @param  cTypeDesc    Tipo de desconto. Se "I" - Item, se "T" - Total
239 @return aRet[01]-Tem Permissão,
240 @return aRet[02]-Nome Superior(Se precisou de autorização ao superior, retorna o próprio ID do usuário)
241 @obs
242 @sample
243 /*/
244 //-----
245 Function STFWLDSUP(    nAccess      , cCash , nPercDesc , nValueDesc , ;
246                      cTypeDesc      )
247
248 Local lRet           := .F.                // Retorna se tem permissão
249 Local aRet           := {}                 // Array de retorno.
250 Local aArea          := GetArea()          // Area atual
251 Local aAreaSLF       := SLF->(GetArea())    // Area SLF
252 Local aAreaSA6       := SA6->(GetArea())    // Area SLF
253 Local cTypeVal       := "1"               // Tipo de Validacao: 1-Senha, 2-Cartao, 3-Ambos
254 Local cCardSup       := ""                // Numero do Cartao do Superior
255 Local cCodSup        := ""                // Codigo do Supervisor
256 Local aUsers         := {}                // Usuarios superiores
257 Local aCodSup        := LjRetSup(1,"",@aUsers) // Codigos dos Caixas Superiores
258 Local cSuperSel      := ""                // Nome do Superior selecionado (caso tenha mais de um)

```

```

259 Local cNumCart      := ""                // Numero do cartão recebido pelo PinPad
260 Local lRetRegra      := .T.              // Retorno da politica de liberação do varejo
261 Local lUsaTef        := STFGetCfg("lUsaTef") // Controla se a estação esta configurada com TEF
262 Local oTef          := STBGetTEF()        // Armazena o Objeto do TEF
263 Local cSupDig        := ""                // digital criptograda do supervisor
264 Local lExistPPad     := .F.              // Existe Pinpad disponivel para leitura de cartão
265 Local llJValDig      := ExistFunc("LjValDig") .AND. ExistFunc("LjUsaBiom") .AND. LjUsaBiom()// Verifica se existe a função LjValDig() no RPO
266
267 Default nAccess      := 0
268 Default cCash        := ""
269 Default nPercDesc    := 0
270 Default nValueDesc   := 0
271 Default cTypeDesc    := ""
272
273 //Seleciona o superior do caixa. Se houver mais de 1 superior,
274 //o caixa seleciona qual superior fara a liberacao
275 cCodSup := LjSelSup( aCodSup , @cSuperSel, aUsers )
276
277 lExistPPad := (lUsaTef .AND. GetApoInfo("LOJA1906.PRW")[4] >= CTOD("05/12/2018") .AND. ValType(oTef) == "0" .AND. ValType(oTef:oConfig:oComSitef) == "0" .AND.
oTef:oConfig:oComSitef:verPinPad() == 1)
278
279 If !Empty(cCodSup)
280   dbSelectArea("SLF")
281   SLF->(dbSetOrder(1)) //Filial + Caixa
282   If SLF->(dbSeek(xFilial("SLF") + cCodSup))
283
284     If SLF->LF_TPVAL $ "23" .AND. lExistPPad // Guarda o Tipo de Validacao
285       cTypeVal := SLF->LF_TPVAL //alimenta a variável somente se está como cartão/ambos e existe pinpad, senão usa o default
286       cCardSup := IIF(ExistFunc("LjEncrypt"),LjEncrypt(1,SLF->LF_NUMCART),"") // Recupera o numero do Cartao cadastrado
287     ElseIf llJValDig .AND. SLF->LF_TPVAL=="4" .AND. SLF->(ColumnPos("LF_FINGER")) > 0 // biometria
288       cSupDig := SLF->LF_FINGER
289       cTypeVal := SLF->LF_TPVAL //alimenta a variável somente se está como biometria e existe campo, senão usa o default
290     EndIf
291   EndIf
292
293   RestArea(aAreaSLF)
294
295 EndIf
296
297 If cTypeVal $ "23" //Tipo 2 - Cartão Magnetico, 3 - Cartão Magnetico e Senha
298
299   //Comunica com o PinPad e recebe o numero do Cartão
300   //Obs: Apenas comunicação com CliSitef
301   cNumCart := STFGetCart(2)
302
303   If cCardSup $ cNumCart
304     // Chamada para validar o acesso do superior
305     If lRetRegra := STFPFILE( nAccess , cCodSup , nPercDesc , nValueDesc , cTypeDesc , .T. )[1]
306
307       lRet := .T.
308
309     EndIf

```



```

310
311     If !lRetRegra .AND. STFGetCfg("lUsaDisplay")
312         DisplayEnv(StatDisplay(), "1C"+ "Acesso Superior negado" )
313     EndIf
314
315 EndIf
316 ElseIf cTypeVal=='4' //Liberação do Supervisor por Biometria
317
318     LjMsgrun(STR0104 + cSuperSel ,, {|lRet:= LjValDig(.T., cSupDig)}) //"Valide a biometria do Superior - "
319
320 EndIf
321
322 If ((!lRet .AND. cTypeVal == "3" ) .OR. cTypeVal == "1" ) .AND. lRetRegra //Tipo 1 - Senha, 3 - Cartão Magnetico e Senha
323     //Me retorna o usuario autenticado
324     If FWAAuthSuper(, @cSuperSel)
325
326         //Busco o Codigo do Caixa passando o usuario recebido
327         cCodSup := POSICIONE("SA6",2,xFilial("SA6") + PADR(UPPER(cSuperSel),TamSX3("A6_NOME")[1]),"A6_COD")
328         lRet     := STFPFILE( nAccess , cCodSup , nPercDesc , nValueDesc , cTypeDesc , .T. )[1]
329
330     If !lRet .AND. STFGetCfg("lUsaDisplay")
331         DisplayEnv(StatDisplay(), "1C"+ "Acesso Superior negado" )
332     EndIf
333
334 EndIf
335 EndIf
336
337 If !lRet
338     STFMessage( ProcName(), "POPUP", STR0095 )//"Acesso do Superior negado, verifique permissões"
339     STFShowMessage(ProcName())
340 EndIf
341
342 AADD( aRet , lRet )
343 AADD( aRet , cSuperSel ) // Caso seja negado acesso, nao retorna o nome do superior
344 AADD( aRet , cCodSup ) // Codigo do Supervisor
345
346 RestArea(aAreaSA6)
347 RestArea(aAreaSLF)
348 RestArea(aArea)
349
350 Return aRet
351
352
353 //-----
354 /*{Protheus.doc} STFWPermis
355 Busca Descrição da Permissão de Acesso
356
357 @param  nAccess      Numero do Acesso Desejado conforme permissoes do caixa da Tabela SLF
358 @return aRet  Retorna Todos os registros ou apenas o solicitado
359 @obs    Se passado o acesso desejado o retorno será o registro do acesso. Caso não seja passado, o retorno será todos os registros.
360 @sample
361 */

```

```

362 //-----
363 Function STFWPermis( nAccess )
364
365 Local aRet      := {}           // Retorno da função
366 Local lFinServ  := AliasIndic("MG8") .AND. SuperGetMV("MV_LJCSF",,.F.) // Valida implementação do serviço financeiro
367
368 Default nAccess      := 0
369
370 ParamType 0 var  nAccess      As Numeric      Default 0
371
372 AADD( aRet , { 1 , STR0006 , STR0007 , STR0008 } ) //Verificacao do caixa se tem analise de credito"
373 AADD( aRet , { 2 , STR0009 , STR0010 , STR0011 } ) //Caixa tem acesso a utilizar o sistema de TEF"
374 AADD( aRet , { 3 , STR0012 , STR0013 , STR0014 } ) //Caixa tem acesso a utilizar Impressora Fiscal"
375 AADD( aRet , { 4 , STR0015 , STR0016 , STR0017 } ) //Acesso para abrir e fechar o caixa"
376 AADD( aRet , { 5 , STR0018 , STR0019 , STR0020 } ) //Acesso para fazer sangria e entrada de troco"
377 AADD( aRet , { 6 , STR0021 , STR0022 , STR0023 } ) //Acesso para fazer Redução Z"
378 AADD( aRet , { 7 , STR0024 , STR0025 , STR0026 } ) //Acesso para cancelar item"
379 AADD( aRet , { 8 , STR0027 , STR0028 , STR0029 } ) //Acesso para cancelar o cupom"
380 AADD( aRet , { 9 , STR0030 , STR0031 , STR0032 } ) //Acesso para alterar parcelas na venda balc,,o"
381 AADD( aRet , { 10 , STR0033 , STR0034 , STR0035 } ) //Acesso para sair da tela de Atendimento"
382 AADD( aRet , { 11 , STR0036 , STR0037 , STR0038 } ) //Acesso para efetuar descontos"
383 AADD( aRet , { 12 , STR0039 , STR0040 , STR0041 } ) //Acesso para acessar a tecla de funções (F12)"
384 AADD( aRet , { 13 , STR0042 , STR0043 , STR0044 } ) //Acesso para abrir Gaveta"
385 AADD( aRet , { 14 , STR0045 , STR0046 , STR0047 } ) //Acesso para realizar atualizacao geral no Check-Out"
386 AADD( aRet , { 15 , STR0048 , STR0049 , STR0050 } ) //Acesso para liberar financiamento"
387 AADD( aRet , { 16 , STR0051 , STR0052 , STR0053 } ) //Acesso para Reabrir o Caixa"
388 AADD( aRet , { 17 , STR0054 , STR0055 , STR0056 } ) //Acesso para Cancelar Transação TEF"
389 AADD( aRet , { 19 , STR0057 , STR0058 , STR0059 } ) //Acesso para liberar vale compra"
390 AADD( aRet , { 20 , STR0060 , STR0061 , STR0062 } ) //Acesso para Liberar Cancelamento Automatico"
391 AADD( aRet , { 21 , STR0063 , STR0064 , STR0065 } ) //Acesso para fazer Leitura X"
392 AADD( aRet , { 22 , STR0066 , STR0067 , STR0068 } ) //Acesso para Liberar sem Senha do Superior na NCC"
393 AADD( aRet , { 23 , STR0069 , STR0070 , STR0071 } ) //Acesso para inativar Vale-Presente"
394 AADD( aRet , { 24 , STR0072 , STR0073 , STR0074 } ) //Acesso para reprocessar orcamentos em aberto na Reducao Z"
395 AADD( aRet , { 25 , STR0075 , STR0076 , STR0077 } ) //Acesso para efetuar o fechamento de caixa simplificado"
396 AADD( aRet , { 26 , STR0078 , STR0079 , STR0080 } ) //Acesso para Alterar Numero do Formulario"
397 AADD( aRet , { 27 , STR0081 , STR0082 , STR0083 } ) //Acesso a Cancelamento de Credito da Lista de Presentes"
398 AADD( aRet , { 28 , STR0084 , STR0085 , STR0086 } ) //Acesso para realizar venda sem impressao"
399 AADD( aRet , { 29 , STR0087 , STR0088 , "" } ) //Acesso para estonar uma venda"
400 AADD( aRet , { 30 , STR0089 , STR0090 , STR0091 } ) //Acesso a Impressora não fiscal para Australia"
401 AADD( aRet , { 31 , STR0092 , STR0093 , "" } ) //Acesso à rotina de aprovação da análise de crédito manual"
402
403 If lFinServ
404     AADD( aRet , { 38 , STR0094 , STR0094 , "" } ) //Acesso a venda de serviço financeiro"
405 EndIf
406
407 AADD( aRet , { 41 , STR0096 , STR0096 , "" } ) //Acesso para limpar Formas de Pagamento - TotvsPdv"
408 aAdd( aRet , { 44 , STR0097 , STR0097 , STR0097 } ) //Altera Parcelas(TEF) de um orçamento importado no TOTVS PDV"
409 aAdd( aRet , { 45 , STR0103 , STR0103 , STR0103 } ) //Permite o recebimento de títulos offline no TOTVS PDV"
410
411 If ( nAccess > 0 ) .AND. aScan( aRet , { |x| x[1] == nAccess } ) > 0
412     aRet := aRet[aScan( aRet , { |x| x[1] == nAccess } )]
413 EndIf

```

```

414
415
416 Return aRet
417
418
419 //-----
420 /*{Protheus.doc} STFUseFiscalPrinter
421 Verifica se utiliza impressora fiscal
422
423 @param
424 @return lUse          Retorna se utiliza impressora fiscal
425 @obs
426 @sample
427 */
428 //-----
429 Function STFUseFiscalPrinter()
430
431 Local lUse := .F.          // Retorna se utiliza impressora fiscal
432 Local aRet := {}          // Array de retornos
433
434 /*
435     Importante esta rotina deve ser chamada após a abertura do periférico na rotina OpenDevice
436 */
437
438 If STGetCfg("lUseECF")
439     aRet := STFFireEvent( ProcName(0)                                ,;      // Nome do processo
440                          "STIsTaxPrinter"                          ,;      // Nome do evento
441                          {"" } )
442
443     If (Len(aRet) > 0 .AND. ValType(aRet[1]) == "L" .AND. aRet[1]) // Verifica permissão se usuário é caixa.
444         lUse := .T.
445     Else
446         lUse := .F.
447     EndIf
448
449 Else
450     lUse := .F.
451 EndIf
452
453 Return lUse
454
455
456 //-----
457 /*{Protheus.doc} STFUserProfInfo
458 Retorna informacoes do usuario
459
460 @param      uCampos - Informa o campo ou os campos que deseja retornar o valor da SLG
461 @return      uReturn - Retorna o valor do campo ou dos campos que foi solicitado na funcao
462 @obs
463 @sample
464 */
465 //-----

```

```

466 Function STFUserProfInfo(uCampos)
467
468 Local aArea      := GetArea()           //Salva area
469 Local uReturn    := ""                 //Retorno da funcao
470 Local nI         := 0                  //Variavel de loop
471 Local xRet       := ""                 //Variavel que armazena o nome do campo
472 Local cUserPDV   := ""                 //Usuario
473 Local IsFly01    := ExistFunc("ISFly01") .AND. ISFly01() //Versao Fly01?
474 Local lMobile    := STFGetCfg("lMobile", .F.) //Smart Client Mobile Pdv Mobile?
475
476 Default uCampos := Nil
477
478 lMobile := ValType(lMobile) == "L" .AND. lMobile
479
480 //Posiciona na SA6
481 xNumCaixa()
482
483 //Versao Fly busca usuario especifico
484 If lMobile .AND. IsFly01
485     cUserPDV := "USR"
486 Else
487     cUserPDV := SA6->A6_COD
488 EndIf
489
490 DbSelectArea("SLF")
491 SLF->(DbSetOrder(1))//LF_FILIAL+LF_COD
492
493 If SLF->(DbSeek( xFilial("SLF")+ cUserPDV ))
494
495     If ValType(uCampos) == "A"
496
497         uReturn := {}
498
499         For nI := 1 To Len(uCampos)
500
501             xRet := FieldGet( FieldPos( If(! Left(uCampos[nI],3) == "LF_", "LF_", "") + uCampos[nI]))
502
503             If ValType(xRet) == "C"
504                 xRet := xRet
505             EndIf
506
507             AAdd(uReturn,xRet)
508
509         Next nI
510
511     Else
512
513         uReturn := FieldGet( FieldPos( If(! Left(uCampos,3) == "LF_", "LF_", "") + uCampos))
514
515         If Valtype(uReturn) == "C"
516             uReturn := uReturn
517         EndIf

```

```

518
519     EndIf
520
521 EndIf
522
523 RestArea(aArea)
524
525 Return uReturn
526
527 //-----
528 /*{Protheus.doc} STFAccess
529 Retorna se o usuario nao tem acesso
530
531 @param
532 @return      lNoAccess - Se retornar .F. o usuario nao tem acesso
533 @obs
534 @sample
535 */
536 //-----
537 Function STFAccess()
538 Return lNoAccess
539
540 //-----
541 /*{Protheus.doc} STFGetCart
542 Funcao responsavel por realizar a comunicação com Pinpad devolvendo criptografada ou não a trilha do cartão
543
544 @type function
545 @param nEncrypt, Numérico, Determina a ação da função aonde:
546 0 Encrypt, Descripta, 1 incripta e 2 Apenas Limpa a String
547
548 @return cNumCart, Retorna o numero do cartão criptografado ou
549 apenas o numero do cartão dependendo do parametro recebido pelo nEncrypt
550 */
551 //-----
552
553 Function STFGetCart(nEncrypt)
554
555 Local cNumCart      := ""           //Numero do Cartão retornado pelo PinPad
556 Local nHandle       := -1          //Variavel de controle de retorno da DLL
557 Local cTrilha1      := Space(150)  //Variavel Obrigatoria para a DLL
558 Local cTrilha2      := Space(150)  //Variavel Obrigatoria para a DLL
559 Local lTentativa     := .T.         //Variavel para controle de tentativas.
560 Local lUsaTef        := STFGetCfg("lUsaTef") //Indica se usa TEF
561 Local oTef           := STBGetTEF()  //Retorna objeto Tef
562
563 Default nEncrypt := 1               //Tipo de tratamento a ser feito, aonde: 0 Encrypt, Descripta, 1 incripta e 2 Apenas Limpa a String
564
565     /* Verifica se o Pinpad esta configurado. */
566 If lUsaTef .AND. GetApoInfo("LOJA1906.PRW")[4] >= CTOD("05/12/2018") .AND. ValType(oTef:oConfig:oComSitef) == "0" .AND. oTef:oConfig:oComSitef:verPinPad() == 1
567
568     /* Le o cartao do superior, e verifica se confere
569     com o cartao cadastrado no SLF. */

```

```

570
571   While lTentativa
572
573       LjMsgrun(STR0098,, {|| nHandle := oTef:oConfig:oComSitef:LeCartao(STR0099, @cTrilha1, @cTrilha2) } ) //"Passe o cartão superior no PinPad." ### "Passe o Cartao"
574
575       If nHandle == 0
576           lTentativa := .F.
577           cNumCart   := cTrilha1 + cTrilha2
578           cNumCart   := LjEncrypt(nEncrypt,cNumCart)
579       Else
580           STFMessage("STFGetCart","YESNO",STR0100) //"Não foi possível receber a senha do usuario pelo PinPad, deseja tentar novamente?"
581           If STFShowMessage("STFGetCart")
582               lTentativa := .T.
583           Else
584               lTentativa := .F.
585           EndIf
586       EndIf
587   End
588 ElseIf lUsaTef .AND. GetApoInfo("LOJA1906.PRW")[4] >= CTOD("05/12/2018") .AND. !ValType(oTef:oConfig:oComSitef) == "0"
589     STFMessage("STFGetCart","ALERT", STR0102) //"Verifique sua configuração do TEF, funcionalidade apenas disponível para CLISITEF"
590     STFShowMessage("STFGetCart")
591 Else
592     STFMessage("STFGetCart","ALERT", STR0101) //"Sitef ou Pinpad não configurado. Verifique suas configurações de estação da aba TEF"
593     STFShowMessage("STFGetCart")
594 EndIf
595
596
597 Return cNumCart

```

Framework/RemoteExecute/STBRemoteExecute.PRW

588 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "TBICONN.CH"
2 #INCLUDE "PROTHEUS.CH"
3 #INCLUDE "PARMTYPE.CH"
4 #INCLUDE "STBRemoteExecute.CH"
5
6 #DEFINE RET_PENDENTE    "00"    // Não processado
7 #DEFINE RET_OK          "OK"    // Processamento OK
8 #DEFINE RET_ERRO        "ER"    // Falha no processamento
9
10
11 Static aRotinas        := {}      // Rotinas da dicionario XXY ([1] - Processo / [2] - Descricao / [3] - Funcao)
12 Static aPrioridade     := {}      // Array com a lista de rotinas na ordem de prioridade
13
14
15 //-----
16 /*{Protheus.doc} STBRemoteExecute()
17 Executa remotamente uma função na retaguarda. Se não houver conexão pode-se gravar localmente para execução posterior (contingência).
18 @param cProcess        ID do processo (XXY_ID)
19 @param aParams          Parametros da rotina a ser executada
20 @param aReturnOK        Array de retornos OK da rotina a ser executada
21 @param lContingency     .T. para gravacao da contingencia caso nao haja comunicacao / .F. para nao gravar a contingencia (passando parametro por referencia retorna se foi
    utilizada a gravacao da contingencia no processo)
22 @param uResult          Resultado da execucao da rotina (referencia)
23 @param cType            Tipo da operacao
24 @param cKeyOri          Chave de origem
25 @param nCodRet         Codigo de retorno do componente FWCallFunctionality
26 @param cDescError       Descricao do erro caso ocorra. passar por referencia
27 @param lPPdvOnCPdv      lógico - Indica que processo executado pelo STWUpdate se trata de registro do PDV Online com Central de PDV
28 @return lRetorno        .T. Quando gravado com sucesso / .F. Quando erro na gravacao
29 */
30 //-----
31 Function STBRemoteExecute( cProcess    , aParams , aReturnOK , lContingency , ;
32                           uResult     , cType   , cKeyOri   , nCodRet    , ;
33                           cDescError   , lPdvOn  , lPPdvOnCPdv )
34
35
36 Local aArea           := GetArea()    // Save da area de trabalho
37 Local lRetorno         := .F.        // Retorno função
38 Local oRetornoCall     := Nil        // Retorno da execucao remota (FW)
39 Local aRetornoCall     := {}         // Retorno da execucao remota (FW)
40 Local uRetorno         := .F.        // Resultado da execucao da funcao (oRetornoCall:uResult)
41 Local nSeconds         := 0          // Armazena segundos inicial
42 Local lSemaforo        := ExistFunc("STILoadSem") .AND. !IsBlind()
43 Local lPDVOnline       := ExistFunc("STFGetOnPdv") .AND. STFGetOnPdv()
44 Local lConnHost        := Nil        // -- Variavel de controle que indica se tem conexão com a retaguarda
45 Local lAutomato        := IIf(Type("lAutomatoX") <> "L", .F., lAutomatoX) //Teste Automatizado
46 Local lPdvOnCPdv       := LjGetCPDV()[1] .AND. STFGetOnPdv() // TOTVS PDV ONLINE COM CENTRAL DE PDV.
47
48 Default nCodRet        := 0          // Apenas Indica se teve conec
49 Default cProcess       := ""         // ID do processo (XXY_ID)
50 Default aParams        := {}         // Parametros da rotina a ser executada
51 Default aReturnOK      := {}         // Array de retornos OK da rotina a ser executada

```

```

52 Default lContingency := .F. // para gravacao da contingencia caso nao haja comunicacao
53 Default uResult := Nil // Resultado da execucao da rotina (referencia)
54 Default cType := "" // Tipo da operacao
55 Default cKeyOri := "" // Chave de origem
56 Default cDescError := "" // Descricao do erro caso ocorra. passar por referencia
57 Default lPdvOn := .F.
58 Default lPPdvOnCPdv := .F. // Indica que processo executado pelo STWUpdate se trata de registro do PDV Online com Central de PDV
59
60
61 ParamType 0 Var cProcess As Character Default ""
62 ParamType 1 Var aParams As Array Default {}
63 ParamType 2 Var aReturnOK As Array Default {}
64 ParamType 3 var lContingency As Logical Default .F.
65 ParamType 5 Var cType As Character Default ""
66 ParamType 6 Var cKeyOri As Character Default ""
67
68
69 If lPPdvOnCPdv
70 // ENTRA AQUI SOMENTE QUANDO O PROCESSO É EXECUTADO PELO STWUDATA PARA SUBIR REGISTRO DO PDVONLINE COM CENTRAL PDV
71 lPdvOnCPdv := lPPdvOnCPdv
72 Endif
73
74 If !Empty(cProcess)
75
76 If !lPDVOnline
77 lPDVOnline := lPdvOn
78 EndIf
79 // Ajusta o tamanho para 10
80 cProcess := Upper(SubStr(cProcess,1,10))
81
82 LjGrvLog( "RemoteExecute","Antes da chamada FWCallFunctionality: " + cProcess )
83
84 nSeconds := Seconds() // Armazena segundos inicial
85
86 If !lPDVOnline .OR. (lPdvOnCPdv .AND. (!cProcess$"FR271HARQ|STBIMPORTR|STBISGETOP"))
87 // -- Varifica se existe comunicação com a retaguarda
88 GetGlbVars( "lConnHost", @lConnHost)
89
90 If ValType( lConnHost ) <> "L" .OR. ( ValType( lConnHost ) == "L" .AND. lConnHost )
91 // Executa rotina pela funcao do FW, e trata o objeto de retorno
92 oRetornoCall := FWCallFunctionality(cProcess, aParams)
93 Else
94 oRetornoCall := STFExecOn():New()
95 oRetornoCall:nStatusCode := -105 // -- -105 : Não foi possível conectar ao host superior
96 oRetornoCall:cStatusDescription := STR0001 // -- "Não foi possível conectar ao host superior"
97 EndIf
98
99 Else
100 lContingency := .F.
101
102 If lAutomato
103 aRetornoCall := StartJob("STFExecOn",getenvserver(),.T.,cProcess,aParams,cEmpAnt,cFilAnt,.T.)//PDV Online (PDV diretamente na retaguarda)

```



```

104     Else
105         aRetornoCall := STFExecOn(cProcess,aParams)
106     EndIf
107
108     oRetornoCall := STFExecOn():New()
109     If ValType(aRetornoCall)=="A" .AND. len(aRetornoCall) > 0
110         oRetornoCall:nStatusCode      := aRetornoCall[1]
111         oRetornoCall:cStatusDescription := aRetornoCall[2]
112         oRetornoCall:uResult          := aRetornoCall[3]
113         oRetornoCall:oStatusError     := aRetornoCall[4]
114     EndIf
115 Endif
116
117 LjGrvLog( "RemoteExecute","Depois da chamada FWCallFunctionality: " + cProcess + " | Tempo: " + AllTrim(Str(Seconds() - nSeconds) ) )
118
119 /*
120 //-----
121 Status e Erros do Componente de Comunicacao
122 0 : Ok
123 -101 : Funcionalidade não encontrada no componente de comunicacao
124 -102 : Função não encontrada no RPO
125 -103 : Ocorreu um erro na execução da funcionalidade
126 -104 : Protocolo de comunicação não configurado no servidor
127 -105 : Não foi possível conectar ao host superior
128 -106 : Não foi possível deserializar os parametros (JSON)
129 -107 : Ocorreu um erro durante a execução da funcionalidade no host superior
130 -108 : Não existe host responsável pela execução da funcionalidade
131 //-----
132 */
133 nCodRet := 0
134 If Valtype(oRetornoCall) == "0"
135     nCodRet := oRetornoCall:nStatusCode
136     LjGrvLog( "RemoteExecute","Retorno nCodRet:",nCodRet )
137     If nCodRet < 0
138         If nCodRet == -106 // -106 : 'Nao foi possivel deserializar os parametros (JSON)
139             lContingency := .F.
140             cDescError := AllTrim(oRetornoCall:cStatusDescription)
141             LjGrvLog( "RemoteExecute","Retornou erro deserializar objeto",cDescError )
142         Else
143             If lContingency
144                 LjGrvLog( "RemoteExecute","sera gravado como contingência ")
145                 lRetorno := STBRecordLocalContingency( cProcess, aParams, aReturnOK , cType , ;
146                                                         cKeyOri                                     )[[1]
147                 lContingency := .F.
148             Else
149                 //Exibe mensagem com erro de comunicacao
150                 Conout( "cStatusDescription : " + AllTrim(oRetornoCall:cStatusDescription) )
151                 LjGrvLog( "RemoteExecute", "cStatusDescription : " + AllTrim(oRetornoCall:cStatusDescription) )
152
153                 // Se houve erro na execucao no servidor mostra erro no console
154                 If nCodRet == -103 .AND. ValType(oRetornoCall:oStatusError) == "0"
155                     cDescError := Left( oRetornoCall:oStatusError:ErrorStack , 254 )

```

```

156         Conout( "RemoteExecute: " + cDescError )
157         LjGrvLog( "RemoteExecute","Retornou erro execucao ",cDescError )
158     EndIf
159 EndIf
160 EndIf
161
162 //Atualiza semaforo de conexao com Retaguarda
163 If lSemaforo .AND. IsInCallStack("STIPOSMAIN")
164     If nCodRet == -104 .Or. nCodRet == -105
165         STILoadSem(.F., .F.) //Nao conectado
166     Else
167         STILoadSem(.F., .T.) //Conectado
168     EndIf
169 EndIf
170 Else
171     // Armazena status de contingencia para acesso por referencia
172     lContingency := .F.
173     // Armazena retorno da funcao para acesso por referencia
174     uResult := oRetornoCall:uResult
175
176     If Len(aReturnOK) > 0
177         lRetorno := (Ascan(aReturnOK, {|x| ValType(x) == ValType(uResult) .AND. x == uResult}) > 0)
178         LjGrvLog( "RemoteExecute","Retorno específico ")
179     Else
180         lRetorno := .T.
181         LjGrvLog( "RemoteExecute","Retornou corretamente ")
182     EndIf
183     //Atualiza semaforo de conexao com retaguarda
184     If lSemaforo .AND. IsInCallStack("STIPOSMAIN")
185         STILoadSem(.F., .T.) //Conectado
186     EndIf
187 EndIf
188 EndIf
189 Else
190     lRetorno := .F.
191     LjGrvLog( "RemoteExecute","Não retornou um objeto valido" )
192 EndIf
193
194 RestArea(aArea) // Restaura area
195
196 LjGrvLog( "RemoteExecute","Retorno:" , lRetorno )
197
198 Return lRetorno
199
200
201 //-----
202 /*{Protheus.doc} STBRecordLocalContingency()
203 Grava os dados na tabela de contingencia
204 @param cProcess ID do processo (XXY_ID)
205 @param aParams Parametros da rotina a ser executada
206 @param aReturnOK Array de retornos OK da rotina a ser executada
207 @param cType Tipo da operacao

```

```

208 @param cKeyOri      Chave de origem
209 @return lRetorno     .T. Quando gravado com sucesso / .F. Quando erro na gravacao
210 */
211 //-----
212 Function STBRecordLocalContingency( cProcess      , aParams      , aReturnOK , cType      , ;
213                                     cKeyOri
214 )
215 Local aArea          := GetArea()      // Save da area de trabalho
216 Local lRetorno       := .F.           // Retorno da gravacao da tabela MES
217 Local aRet           := {.F.}         // Retono da funcao
218
219 Default cProcess      := ""           // ID do processo (XXY_ID)
220 Default aParams       := {}           // Parametros da rotina a ser executada
221 Default aReturnOK     := {}           // Array de retornos OK da rotina a ser executada
222 Default cType         := ""           // Tipo da operacao
223 Default cKeyOri       := ""           // Chave de origem
224
225 ParamType 0 Var      cProcess As Character Default ""
226 ParamType 1 Var      aParams As Array Default {}
227 ParamType 2 Var      aReturnOK As Array Default {}
228 ParamType 3 Var      cType As Character Default ""
229 ParamType 4 Var      cKeyOri As Character Default ""
230
231
232 DbSelectArea("MES")
233 If Reclock("MES", .T.)
234     REPLACE MES->MES_FILIAL WITH xFilial("MES") // xFilial da tabela
235     REPLACE MES->MES_SEQ WITH GetSXEnum("MES", "MES_SEQ") // Numeracao automatica
236     REPLACE MES->MES_ESTACAO WITH STFGetStation("LG_CODIGO") // xFilial da tabela
237     REPLACE MES->MES_DATA WITH dDataBase // Data base do sistema
238     REPLACE MES->MES_HORA WITH SubStr(Time(), 1, 5) // Horario
239     REPLACE MES->MES_PROCESS WITH cProcess // Nome da rotina a ser executada
240     REPLACE MES->MES_PARAMS WITH FwJsonSerialize(aParams) // Parametros da rotina a ser executada
241     REPLACE MES->MES_RETOK WITH FwJsonSerialize(aReturnOK) // Possiveis retornos OK da rotina a ser executada
242     REPLACE MES->MES_SITUA WITH RET_PENDENTE // Status de execucao da rotina
243
244     REPLACE MES->MES_TIPO WITH cType // Status de execucao da rotina - "01" = Reserva
245
246
247 // Guarda Filial e Chave de origem de quem chamou contingencia para posterior
248 // gravacao do resultado na origem apos execucao
249 REPLACE MES->MES_FILORI WITH cFilAnt // Status de execucao da rotina
250 REPLACE MES->MES_KEYORI WITH cKeyOri
251
252 ConfirmSX8()
253 MES->(MsUnlock())
254 lRetorno := .T.
255
256 aRet[1] := lRetorno // Retorno da funcao
257 Add(aRet, MES->(RecNo())) // recno da MES
258
259 EndIf

```

```

260
261 MES->(DbCloseArea())
262 RestArea(aArea)
263
264
265 Return aRet
266
267
268 //-----
269 /*{Protheus.doc} STBRemoteExecute
270 Job de processamento de contingencias executado na retaguarda
271 @return Nil
272 @obs
273 @sample
274 */
275 //-----
276 Function STWJOBContingency( cEmp , cFil , cPdv )
277
278 Local lPrepEnv      := .F.           // Verifica se deve preparar o Ambiente
279 Local lConnect      := .F.           // Se conectou no server
280
281
282 Default cEmp        := ""            // Empresa para processamento
283 Default cFil        := ""            // Filial para processamento
284 Default cPdv        := "001"         // Conteudo do terceiro parametro (Parm3 do mp8srv.ini)
285
286
287 //Aguarda para evitar erro de __CInternet
288 Sleep(5000)
289
290 lPrepEnv := !Empty(cEmp) .AND. !Empty(cFil)
291
292
293 If lPrepEnv
294     RPCSetType(3)
295     // "FRT" > Liberacao de acesso PDV cTree para o modulo FrontLoja
296     RpcSetEnv(cEmp,cFil,Nil,Nil,"FRT")
297 Else
298     ConOut("Não foram informados os parametros do processo no arquivo INI")
299 EndIf
300
301 //Testa a comunicacao atraves da STFTTestRemoteExecute
302 lConnect := STFTTestRemoteExecute()
303
304 If lConnect
305     ConOut("Com conexao com o Host, o sera realizado processo de contingencia - ProcessContingency")
306     ProcessContingency()
307     ConOut("Final do processo de contingencia - ProcessContingency")
308 Else
309     ConOut("Sem conexao com o Host, o nao sera realizado processo de contingencia - ProcessContingency")
310 EndIf
311

```

```

312 FWCloseFunctionality() // Desconecta do Host Superior para liberar memoria alocada
313
314
315 If lPrepEnv
316     RESET ENVIRONMENT
317 EndIf
318
319
320 Return Nil
321
322
323 //-----
324 /*{Protheus.doc} ProcessContingency()
325 Processa as contingencias podendo ser por lotes, respeitando as prioridades caso existam
326 @return Nil
327 @obs
328 @sample
329 */
330 //-----
331 Static Function ProcessContingency()
332
333 Local aArea           := GetArea()           // Save da area de trabalho
334 Local nMaxPacote      := SuperGetMv("MV_MAXUPL", Nil, 50) // Maximo de execucao de rotinas por lote
335 Local nCount          := 0                  // Contador de rotinas executadas
336 Local aParams         := {}                 // Array de parametros da rotina
337 Local aRetornosOK     := {}                 // Array de retornos ok da rotina a ser executada
338 Local lOK             := .F.                // Status de execucao da rotina .T. OK / .F. Erro
339 Local nX              := 0                  // Contador
340 Local uRet            := NIL                //Retorno da Rotina
341 Local nTamRet         := 0                  //Tamanho do array de retorno
342 Local cSitua          := ""                 //Situação da Contingência
343 Local cChave          := ""                 //Chave do Registro
344 Local cCmpChave       := "Chave: MES_PROCESS/MES_SEQ/MES_ESTACA0: " //Campos da Chave
345 Local aMesRec         := {} //Array de Registros da MES
346 Local nC              := 0 //Contador de registro
347
348 MES->(DbSetOrder(2) ) // MES_FILIAL+MES_SITUA
349
350 cChave := xFilial("MES") + AllTrim(RET_PENDENTE)
351 MES->(DbSeek(cChave))
352 While MES->(!Eof()) .AND. MES->( MES_FILIAL + MES->MES_SITUA == cChave)
353     aAdd( aMesRec, MES->(Recno()))
354     MES->(DbSkip(1))
355     If ++nCount >= nMaxPacote
356         Exit
357     EndIf
358 EndDo
359
360 For nC := 1 to Len(aMesRec)
361
362     MES->(DbGoTo(aMesRec[nC]))
363     aRetornosOK := {}

```

```

364     aParams      := {}
365
366     cSitua := ""
367     cChave := MES->(MES_PROCESS+"/"+MES_SEQ+"/"+MES_ESTACA0)
368
369     If FWJsonDeserialize(MES->MES_RETOK, @aRetornosOK) .AND. FWJsonDeserialize(MES->MES_PARAMS, @aParams)
370
371         If ValType(aRetornosOK) == "A" .AND. (nTamRet := Len(aRetornosOK)) > 0
372             lOK := .F.
373             uRet := DynamicCall(AllTrim(MES->(MES_PROCES)), aParams)[2]
374
375             //Trata o tipo do retorno, pois a rotina de recebimento em contingencia offline tem 2 tipos de retorno: lógico e caractere
376             For nX := 1 to nTamRet
377                 If ValType(uRet) == ValType(aRetornosOK[nX])
378                     lOK := uRet == aRetornosOK[nX]
379                     If lOK
380                         Exit
381                     EndIf
382             EndIf
383
384             Next nX
385
386             LjGrvLog( "ProcessContingency", "ProcessContingency Retorno de execução " + cCmpChave + cChave + "{lOk, uRet} ", {lOk, uRet})
387             If lOK
388                 cSitua := RET_OK
389             Else
390                 cSitua := RET_ERRO
391             EndIf
392         Else
393             lOk := DynamicCall(AllTrim(MES->(MES_PROCES)), aParams)[1]
394             LjGrvLog( "ProcessContingency", "ProcessContingency Retorno de execução " + cCmpChave + cChave + "{lOk} ", {lOk})
395
396             If lOk
397                 cSitua := RET_OK
398             EndIf
399         EndIf
400
401     Else
402         LjGrvLog( "ProcessContingency", "ProcessContingency Erro ao desserializar parâmetro")
403         cSitua := RET_ERRO
404     EndIf
405
406     If !Empty(cSitua)
407         RecLock("MES", .F.)
408         REPLACE MES->MES_SITUA WITH cSitua
409         MES->(MsUnlock())
410     EndIf
411
412 Next nC
413
414
415 MES->(DbCloseArea())

```

```

416
417 RestArea(aArea)
418
419 Return Nil
420
421
422 //-----
423 /*{Protheus.doc} LoadPriority
424 Carrega o array estatico de ordem de prioridades de execucao (tabela MET)
425 @return Nil
426 @obs
427 @sample
428 */
429 //-----
430 Static Function LoadPriority()
431
432 Local cAliasProcess := "" // Guardara Alias temporario da query executada
433 Local aArea := GetArea() // Save da area de trabalho
434 Local nTamSX3 := TamSx3("MET_PROCES")[1] // Tamanho do campo
435
436
437 cAliasProcess := GetNextAlias() // Alias temporario da query executada
438
439 BeginSQL alias cAliasProcess
440     SELECT
441         MET_PROCES PROCES,
442         MET_PRIORI PRIORI
443     FROM
444         %table:MET% MET
445     WHERE
446         MET.%notDel%
447     ORDER BY MET_PRIORI ASC
448 EndSql
449
450 aPrioridade := {}
451
452 While (cAliasProcess)->(!EOF())
453     Aadd(aPrioridade, PadR((cAliasProcess)->(PROCES), nTamSX3))
454     (cAliasProcess)->(DbSkip())
455 End Do
456
457 (cAliasProcess)->(DbCloseArea())
458
459
460 RestArea(aArea)
461
462 Return Nil
463
464
465 //-----
466 /*{Protheus.doc} DynamicCall
467 Executa rotinas de forma dinamica, passando o nome do processo e parametros

```

```

468 @return Aret[1] Retorna se executou a funcao no server
469 @return Aret[1] uRetorno Variante
470 @obs
471 @sample
472 */
473 //-----
474 Static Function DynamicCall( cProcess, aParams )
475
476 Local nI           := 0           // Contador
477 Local uRetorno     := Nil         // Retorno da funcao
478 Local cParametros  := ""         // Parametros da funcao
479 Local lRet         := .F.        // Retorno
480
481 Default cProcess   := ""
482 Default aParams    := {}
483
484 ParamType 0 Var    cProcess   As Character   Default ""
485 ParamType 1 Var    aParams    As Array      Default {}
486
487 For nI := 1 To Len(aParams)
488     cParametros += IIF(nI <> 1, ", ", "") + "aParams[" + AllTrim(Str(nI)) + "]"
489 Next nI
490
491 //uRetorno := Eval("&{| " + cProcess + "(" + cParametros + ")"}")
492
493 //Tenta executar novamente o processo que esta em contingencia
494 lRet := STBRemoteExecute( cProcess , aParams , Nil , .F. , @uRetorno )
495
496 Return ({lRet, uRetorno})
497
498
499 //-----
500 /*/{Protheus.doc} GetFunction()
501 Retorna o nome da funcao de acordo com o ID (XXY_ID)
502 @param cProcess XXY_ID da rotina
503 @return cRetorno - nome da funcao
504 @obs
505 @sample
506 */
507 //-----
508 Static Function GetFunction( cProcess )
509
510 Local cRetorno     := ""         // Retorno da funcao
511 Local nPos         := 0         // Posicao do array, retorno do Ascan
512
513 Default cProcess   := ""
514
515 ParamType 0 Var    cProcess   As Character   Default ""
516
517 nPos := Ascan(aRotinas, {|x| Upper(Alltrim(x[1])) == Upper(AllTrim(cProcess))})
518 If nPos > 0
519     cRetorno := aRotinas[nPos][3]

```



```

520 EndIf
521
522 Return cRetorno
523
524
525 //-----
526 /*{Protheus.doc} LoadArrayFunctions
527 Carrega o array de funcoes estatico
528 @return Nil
529 @obs
530 @sample
531 */
532 //-----
533 Static Function LoadArrayFunctions()
534
535 Local aArea := GetArea()    // Save da area de trabalho
536
537 aRotinas := {}
538
539 FwOpenXXY()
540 DbSetOrder(1)
541
542 While XXY->(EOF())
543     Aadd(aRotinas, {XXY->XXY_ID, XXY->XXY_DESC, XXY->XXY_FUNC})
544     XXY->(DbSkip())
545 End Do
546
547 XXY->(DbCloseArea())
548 RestArea(aArea)
549
550 Return Nil
551
552 //-----
553 /*{Protheus.doc} STTMTT
554 Função chamada pela lib e executada no host MASTER sem a necessidade de funcionalidade cadastrada ou vinculo com perfil.
555
556 @type      Function
557
558 @param oX, Object, Objeto enviado pela camada de framework com dados referente a requisição
559 @param cFunction, Caracter, Função que será executada no Host superio
560 @param aParams, Array, Contem os parametros da função (cFunction)
561 @return uResult, Indefinido, Retorna para camada inferior o retorno da função (cFunction)
562 */
563 //-----
564
565 Function STCOMDIR(oX,cFunction,aParams)
566 Local lVirgula := .F.
567 Local nParams := 0
568 Local uResult := Nil
569
570 Default cFunction := ""
571 Default aParams := {}

```

```
572
573 If !Empty(cFunction)
574     cFunction += "("
575     For nParams := 1 To len(aParams)
576         If lVirgula
577             cFunction += ", "
578         EndIf
579         lVirgula := .T.
580         cFunction += "aParams[" + cValtoChar(nParams) + "]"
581     Next
582     cFunction += ")"
583
584     uResult := &(amp;cFunction)
585
586 EndIf
587
588 return uResult
```

Framework/RemoteExecute/STFExecOn.PRW

132 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "TBICONN.CH"
2 #INCLUDE "PROTHEUS.CH"
3
4 //-----
5 /*{Protheus.doc} STFExecOn
6 Classe de controle de execução online
7 @param
8 @return Self
9 @obs
10 @sample
11 */
12 //-----
13 Class STFExecOn
14
15     Data nStatusCode
16     Data cStatusDescription
17     Data uResult
18     Data oStatusError
19
20     Method New()//Construtor
21
22 EndClass
23
24
25 //-----
26 /*{Protheus.doc} STFExecOn
27 Metodo construtor
28 @param
29 @return Self
30 @obs
31 @sample
32 */
33 //-----
34 Method New() Class STFExecOn
35
36     Self:nStatusCode := 0
37     Self:cStatusDescription := ""
38     Self:uResult := Nil
39     Self:oStatusError := Nil
40
41 Return Self
42 //-----
43 /*{Protheus.doc} STFExecOn
44 Faz execução da rotina na própria base(Local/Online) pois no PDV Online
45 não depende da FWCallFunctionality(cProcess, aParams)
46
47 @type function
48 @param lValue
49 @return lIsPOS - Retorna se é PDV
50 @obs
51 @sample
52 */

```

```

53 //-----
54 Function STFExecOn(cProcess, aParams,cEmpresa,cFilExec,lTestAuto)
55
56 Local nI      := 0
57 Local bBloco  := { || }
58 Local cParams := ""
59 Local aErroAuto := {}
60 Local nCount  := 1
61 Local lAmbOk  := .T.
62 Local aObjRet := {0,"",Nil,Nil}
63
64 Default cProcess := ""
65 Default aParams  := {}
66 Default cEmpresa := ""
67 Default cFilExec := ""
68 Default lTestAuto := .F.
69
70 If lTestAuto
71   RPCSetType(3) //Nao faz consumo de licença
72
73   //realizamos a abertura do ambiente
74   lAmbOk := RPCSetEnv(cEmpresa, cFilExec,Nil,Nil, "LOJA")
75
76   //Seta nModulo para 23 para garantir execucao via Loja
77   nModulo := 23
78 EndIf
79
80 If lTestAuto .AND. !lAmbOk
81   LjGrvLog( Nil, "Falha ao abrir o ambiente - Empresa: " + cEmpresa + " Filial: " + cFilExec )
82   aObjRet[1] := -105//Não foi possível conectar ao host superior
83 Else
84   LjGrvLog( Nil, "Aberto o ambiente - Empresa: " + cEmpAnt + " Filial: " + cFilAnt )
85
86   If ExistFunc(cProcess)
87
88     //Quando é execução local (Totvs PDV Online) sem passar pelo FWCallFunctionality, então o primeiro parametro da função não é o
89     //o objeto com os dados da conexão por isso foi necessario fazer um ajuste incluir o conteúdo LjGetLTT na segunda posição.
90     If !lTestAuto .AND. cProcess = 'STCOMDIR' .AND. Len(aParams) == 1 .AND. aParams[1] == "LjGetLTT"
91       aParams := {}
92       aParams := Array(2)
93       aParams[1] := Nil
94       aParams[2] := "LjGetLTT"
95     EndIf
96
97     For nI := 1 To Len(aParams)
98       cParams += IIF(nI == 1,"",",")
99       cParams += " aParams["+ AllTrim(STR(nI)) +"] "
100     Next nI
101
102     bBloco := { || &(cProcess + "(" + cParams + ")") }
103
104     BEGIN SEQUENCE

```

```

105         aObjRet[3] := Eval(bBloco)
106     RECOVER
107     aErroAuto := GetAutoGRLog()
108     For nCount := 1 To Len(aErroAuto)
109         aObjRet[2] += StrTran(StrTran(aErroAuto[nCount], "<", ""), "-", "") + (" ")
110     Next nCount
111     aObjRet[1] := -103//Ocorreu um erro na execução da funcionalidade
112     DisarmTransaction()
113     END SEQUENCE
114
115     If ValType(aObjRet[3])=="C"
116         aObjRet[1] := 0//Ocorreu um erro na execução da funcionalidade
117         aObjRet[2] := aObjRet[3]
118     ElseIf ValType(aObjRet[3])=="L" .AND. !aObjRet[3]
119         aObjRet[1] := -103//Ocorreu um erro na execução da funcionalidade
120         aObjRet[2] := "Falha durante processo executado pela retaguarda!"
121     Endif
122 Else
123     aObjRet[1] := -102//Função não encontrada no RP0
124     aObjRet[2] := "Função não encontrada no RP0"
125 Endif
126 Endif
127
128 If !TestAuto
129     RESET ENVIRONMENT//Fecha o ambiente
130 EndIf
131
132 Return aObjRet

```

Framework/RemoteExecute/STFTTestRemoteExecute.prw

60 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STFTTestRemoteExecute
6 Verifica se há comunicação com a Retaguarda
7
8 @param lComCPDV - Se executa a funcao na central de PDV
9 @return lCommuOk      Retorna se a tem comunicação com a Retaguarda
10 @obs
11 @sample
12 */
13 //-----
14 Function STFTTestRemoteExecute( lComCPDV )
15
16 Local lCommuOk := .F.
17
18 Default lComCPDV := .F. //Usa comunicacao com a Central de PDV
19
20 If lComCPDV
21     //Para central de PDV executa outra funcao para testar comunicacao
22     STBRemoteExecute("STFCOMMUCP", NIL , NIL, .F. , @lCommuOk )
23 Else
24     STBRemoteExecute("STFCOMMUOK", NIL , NIL, .F. , @lCommuOk )
25 EndIf
26
27 Return lCommuOk
28
29
30 //-----
31 /*{Protheus.doc} STFCommuOk
32 Função executada na retaguarda
33
34 @param
35 @return
36 @obs
37 @sample
38 */
39 //-----
40 Function STFCommuOk()
41
42 // Se chegou aqui a comunicação está ok, retorna .T.
43
44 Return .T.
45
46
47 //-----
48 /*{Protheus.doc} STFCommuCP
49 Função executada na Central de PDV para verificar se esta Ativa
50 @param
51 @return
52 @obs

```

```
53 @sample
54 /*
55 //-----
56 Function STFCommuCP()
57
58 // Se chegou aqui a comunicação está ok, retorna .T.
59
60 Return .T.
```

Framework/Restart/STFRestart.prw

321 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2
3 //-----
4 /*{Protheus.doc} STFRestart
5 Funcao responsável em reiniciar para próxima venda
6
7 @param      Nil
8 @return     Nil
9 @obs
10 @sample
11 */
12 //-----
13 Function STFRestart(lCncCupom)
14
15 Local lFinServ      := AliasIndic("MG8") .AND. SuperGetMV("MV_LJCSF",,.F.) // Define se habilita o controle de servicos financeiros
16 Local lLimSang      := SuperGetMV( "MV_LJLISAN",, .F.) // Utiliza controle para limite de sangria
17 Local lSelNcc       := ExistBlock("STSelNcc") //Existe ponto de entrada para selecao da NCC?
18 Local lCRdesItTt    := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMV("MV_LJCRDPT",,"0") == "1" .AND.
    FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal", .T.) // Verifica se o calculo do desconto por item esta sendo feito no
    final da venda
19
20 Default lCncCupom  := .F.
21 /*/
22     Motivo de Venda Perdida
23 /*/
24 STDRLSFree()
25
26 /*
27     Limpa regras de bonificacao
28 */
29 STDBS7Rules( )
30
31 /*/
32     Motivo de Desconto
33 /*/
34 STDRFDSFree()
35
36 /*/
37     Desconto na próxima venda
38 /*/
39 STDNSDFree()
40
41 /*
42     Reinicio a variavel que controla se a tela do CPF ja foi apresentada.
43 */
44 STI7InfCPF(.F.)
45
46 /*
47     Reinicializo o modelo de dados e o preenchimento com as informacoes iniciais basicas.
48 */
49
50 If !lCncCupom

```



```
51     STDPBRestart()
52 Endif
53 STDInitPBasket()
54
55
56 // Limpa as variaveis static das NCCs
57 If ProcName(1) <> "STIRECESEL" .OR. !lSelNcc
58     STINCCClearStatic()
59 EndIf
60
61 /*/
62     Zera funções fiscais
63 /*/
64 STBTaxEnd()
65
66 /*/
67     Inicia Função fiscais
68 /*/
69 STBStartTax()
70
71 /*/
72     Seta Default cupom aberto
73 /*/
74 STWSetIsOpenReceipt( .F. )
75
76 /*/
77     Set na variavel lCard
78 /*/
79 STISetCard()
80
81 /*/
82     Set no array de ID de TEF
83 /*/
84 If FindFunction("STBSetIDTF")
85     STBSetIDTF()
86 EndIf
87
88 /*/
89     Totalizador
90 /*/
91 STFTotRestart()
92
93 /*/
94     FreeObj no objeto do TEF
95 /*/
96 STBSetRetTef()
97
98 /*/
99     Limpa variavel dos dados do cheque
100 /*/
101 STWSetCkRet()
102
```

```

103 /*
104     Limpa dados do Cupom de Vale Troca
105 */
106 STICVTClear()
107
108 /*
109     Limpa dados do Cupom de Vale Presente / Credito
110 */
111 STBSetCodVP()
112 STBSetVlrVP()
113
114 /*
115     Limpa os dados das parcelas
116 */
117 STBSetParc()
118
119 /*/
120     Limpa msg da tela
121 /*/
122 STFCleanInterfaceMessage()
123
124 /* Desabilita os botoes da tela */
125 STIBtnDeActivate()
126
127 //Venda nao esta mais ativa
128 STBSaleAct(.F.)
129
130 /* Zerar doação Instituto Arredondar */
131 STBSetInsArr(0)
132
133 /* Limpa informacoes do objeto de Garantia Estendida */
134 If ExistFunc("STWItemGarEst")
135     STWItemGarEst(3) //(1=Set - 2=Get - 3=Clear)
136 EndIf
137
138 /* Limpa informacoes do objeto de Serviços Financeiros */
139 If lFinServ
140     STWItemFin(3) //(1=Set - 2=Get - 3=Clear)
141 EndIf
142
143 /* Limpa CPF/CNPJ utilizado no Panel de Recebimento do Cliente selecionado */
144 If FindFunction("STISCnpjRec")
145     STISCnpjRec("")
146 EndIf
147
148 /* Reseta a Operação em execução atual */
149 If FindFunction("STIPoSS0pc")
150     STIPoSS0pc("")
151 EndIf
152
153 /* Verificar Limite de Sangria */
154 If lLimSang .AND. AliasIndic("MGW") .AND. AliasIndic("MGX") .AND. FindFunction("STDVerLimSan")

```

```

155     STDVerLimSan()
156 EndIf
157
158 /*
159     Limpa a variavel que define se os pagamentos serão Somente Leitura
160     (quando o Caixa não tem permissão de Alterar Parcelas)
161 */
162 If FindFunction("STIGetPayR0")
163     STISetPayR0(0)
164 EndIf
165
166 /* Reseta a variavel que define que os pagamentos de um orçamento importado foram utilizados */
167 STISetPayImp(.F.)
168
169 /* Reseta as variavel de recebimento de titulo*/
170 STISetRecTit(.F.)
171
172 /* Reseta o botao responsavel por finalizar a venda (STIItemRegister) */
173 If FindFunction("STISetBtIR")
174     STISetBtIR()
175 EndIf
176
177 /* Limpa a variavel de vendedor selecionado na venda */
178 If FindFunction("STISelVend")
179     STISelVend(.F.)
180 EndIf
181
182 /* Limpa a variaveis dos pagamentos da venda */
183 If ExistFunc("STBClearPay")
184     STBClearPay()
185 EndIf
186
187 /* Limpa a variavel do valor de entrada da venda */
188 If ExistFunc("STBSetEnt")
189     STBSetEnt()
190 EndIf
191
192 /* Limpa a variavel model dos pagamentos */
193 If ExistFunc("STIPayMdlc")
194     STIPayMdlc()
195 EndIf
196
197 /* Limpa a variaveis do codigo do cliente e loja */
198 If ExistFunc("STWScliLoj")
199     STWScliLoj()
200 EndIf
201
202 /* Limpa variavel estatica de importacao de orcamento, para nao limpar desconto em vendas sem importacao */
203 If ExistFunc("STIClrVar")
204     STIClrVar()
205 EndIf
206

```

```

207 //Volta ao estado original da variavel que indica se houve alteração em alguma informação do orçamento do tipo FI
208 If ExistFunc("STISFiAltImp")
209     STISFiAltImp()
210 EndIf
211
212 /* Limpa a variavel de recuperação de venda */
213 STB7Recovered(.F.)
214
215 /* Limpa o array aCustSele com o código do cliente e a loja utilizado no Recebimen de Título do fonte STDCustLis.prw */
216 If ExistFunc("STDSCustSele")
217     STDSCustSele()
218 EndIf
219
220 /* Essa funcao limpa o conteudo do array com os precos da ultima tabela de preco */
221 MaReleTabPrc()
222
223 /*Limpa Variavel lZeraPayImport presente no STINCCSelection */
224 If ExistFunc("STISetZrPg")
225     STISetZrPg()
226 EndIf
227
228 //Limpar a variavel da multi-negociacao
229 If ExistFunc("STISetMult")
230     STISetMult(.F.)
231 EndIf
232
233 //Limpar as variáveis de pergunta se imprime CPF/CNPJ após importar orçamento, caso se MV_LJDCCLI for 0 ou 1. Estas funções estão em STBImportSale.prw
234 If ExistFunc("STBSetPgtCpf") .AND. ExistFunc("STBSetCpfRet")
235     STBSetPgtCPF(.F.)
236     STBSetCPFRet(.F.)
237 EndIf
238
239 //Inicializa a variavel static lDescOk com valor default.
240 If ExistFunc("STBIDSet")
241     STBIDSet(.T.)
242 EndIf
243
244 If ExistFunc("STBGetDPgChck")
245     STBGetDPgChck()
246 EndIf
247
248 /* Limpa as variáveis para Análise de Crédito SIGACRD */
249 If ExistFunc("STBSetCrdIdent")
250     STBSetCrdIdent()
251 EndIf
252
253 If ExistFunc("STBSetAcresFin")
254     STBSetAcresFin( 0, 0 )
255 EndIf
256
257 //Limpa variável de tributação dos produtos para impressão SAT
258 If ExistFunc("LjSetTriSat")      //LOJSAT.PRW

```

```

259     LjSetTriSat()
260 EndIf
261
262 //Limpa variável de desconto do total da venda dos orçamentos importados
263 If ExistFunc("STBSetDscImp")    //STBImportSale.prw
264     STBSetDscImp(0)
265 EndIf
266
267 // Limpa Variavel que indica se tem desconto no total da venda
268 If ExistFunc("STBSetDiscTotPDV")
269     STBSetDiscTotPDV( .F. )
270 EndIf
271
272 // Reseta a variavel lDescGrupo relacionado a regra de desconto
273 If ExistFunc("STDSetGrp")
274     STDSetGrp(.F.)
275 EndIf
276
277 // Reseta a variavel lDescReg relacionado a regra de desconto
278 If ExistFunc("STBSetDesc")
279     STBSetDesc(.F.)
280 EndIf
281
282 // Reseta a variavel lFrmDesc relacionado a regra de desconto
283 If ExistFunc("STDSetFrm")
284     STDSetFrm(.F.)
285 EndIf
286
287 // Reseta array com os pagamentos do orçamento importado.
288 If ExistFunc("SetOrcP0ri")
289     SetOrcP0ri({"",0,0})
290 EndIf
291
292 If ExistFunc("STDSetDTSPg")
293     STDSetDTSPg(.F.)
294 Endif
295
296 // Reseta a variavel lApTPD relacionado ao Pagamento digital
297 If ExistFunc("STBSetTPD")
298     STBSetTPD(.F.)
299 EndIf
300
301 // Reseta a variavel lEndQr relacionado ao Qr-Code do PIX no PinPad
302 If ExistFunc("STBSetEnd")
303     STBSetEnd(.F.)
304 EndIf
305
306 // Reseta a variavel lIsImpDes relacionado ao desconto no total na importação do orçamento
307 If ExistFunc("STISetImp")
308     STISetImp(.F.)
309 EndIf
310

```

```
311 //Reinicia variaveis Static do fonte do LOJR600A
312 If ExistFunc("Lj600ACln")
313     Lj600ACln()
314 EndIf
315
316 // Reseta o array ajDescProdRegDescTotal, utilizado na tela de descontos, da Regra de Desconto pelo total de produtos na venda
317 If lCRdesItTt
318     totvs.protheus.retail.desconto.RegraDescProdutoTotal.SetDescProdRegDesTotal({})
319 EndIf
320
321 Return Nil
```

Framework/Rms/STFCrmRms.prw

117 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "TBICONN.CH"
3 #INCLUDE "STFCRMMS.CH"
4
5 //-----
6 /*{Protheus.doc} STFCrmRms
7 Funcao para pesquisar o credito do cliente no ws da RMS
8
9 */
10 //-----
11 Function STFCrmRms(cType,nVal,cCardConve,cCpfCheck)
12
13 Local oDados := Nil //Classe do ws de credito
14 Local aRet := Array(4) //Array de retorno
15
16 Default cType := ''
17 Default nVal := 0
18 Default cCardConve := ''
19 Default cCpfCheck := ''
20
21 If (cType $ 'CH|CO') .AND. (nVal > 0) .AND. (FindFunction('U_NXTWPAR'))
22     // Foi retirado o trecho desse If, pois a função WSCreditoService
23     // utilizada para preencher o objeto oDados, foi descontinuada.
24     aRet[1] := .T.
25     aRet[2] := 0
26     aRet[3] := ''
27     aRet[4] := ''
28 Else
29     aRet[1] := .T.
30     aRet[2] := 0
31     aRet[3] := ''
32     aRet[4] := STR0003 //"Forma de pagamento não contempla o CRM!"
33 EndIf
34
35 Return aRet
36
37
38 //-----
39 /*{Protheus.doc} STFRmsWs
40 Retorna o caminho do web service de credito
41
42 */
43 //-----
44 Static Function STFRmsWs()
45
46 Local cPatchWs := '' //Caminho do webservice da rsm
47
48 If !Empty(AllTrim(SuperGetMv("MV_LJWSCPO",,"")))
49     cPatchWs := 'http://' + AllTrim(SuperGetMv("MV_LJWSCRm",,"")) + ':' + AllTrim(SuperGetMv("MV_LJWSCPO",,"")) + '/RMS.WS.Credito/Services/CreditoService.svc'
50 Else
51     cPatchWs := 'http://' + AllTrim(SuperGetMv("MV_LJWSCRm",,"")) + '/RMS.WS.Credito/Services/CreditoService.svc'
52 EndIf

```

```

53
54 Return cPatchWs
55
56
57 //-----
58 /*{Protheus.doc} STFDePFili
59 Funcao para pesquisar a filial de/para
60
61 */
62 //-----
63 Function STFDePFili(cFilPrt)
64
65 Local cFilialRms := '' //Variavel de retorno
66
67 Default cFilPrt := ''
68
69 If !Empty(cFilPrt)
70
71     dbSelectArea('MGL')
72     MGL->(dbSetOrder(2)) //MGL_FILIAL+MGL_FILPRT
73     If MGL->(dbSeek(xFilial('MGL')+cFilPrt))
74         cFilialRms := MGL->MGL_FILRMS
75     EndIf
76
77 EndIf
78
79 Return cFilialRms
80
81 //-----
82 /*{Protheus.doc} STFDePFili
83 Funcao para pesquisar a filial de/para
84
85 */
86 //-----
87 Function STFCpfCli()
88
89 Local cRet := '' //Retorno do documento do cliente
90 Local aArea := GetArea() //Guarda area
91 Local cCodCli := STDGPBasket( "SL1" , "L1_CLIENTE" ) //Retorna o cliente que esta na cesta de vendas
92 Local cCodLoj := STDGPBasket( "SL1" , "L1_LOJA" ) //Retorna a loja do cliente que esta na cesta de vendas
93
94 dbSelectArea("SA1")
95 SA1->(dbSetOrder(1)) //A1_FILIAL+A1_COD+A1_LOJA
96 If SA1->(dbSeek(xFilial("SA1")+cCodCli+cCodLoj))
97     cRet := SA1->A1_CGC
98 EndIf
99
100 RestArea(aArea)
101
102 Return cRet
103
104 //-----

```



```
105 /*/{Protheus.doc} STFSetAuto
106 Set da autorizacao para o CRM da RMS
107
108 */
109 //-----
110 Function STFSetAuto()
111
112 //bloco de IF e respectivas variáveis excluídos devido trabalho de débito técnico da Engenharia de Funções não Compiladas
113 //WSCREDITOSERVICE
114
115 STISetChRms()
116
117 Return .T.
```

Framework/Round/STBRounding.prw

326 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static cImpressora := "" //Impressora utilizada
5
6 //-----
7 /*{Protheus.doc} STBRound
8 Arredonda a parte decimal do valor especificado no parametro de
9 acordo com a quantidades de casas decimais solicitadas, ou
10 se não for passado usa padrao do sistema
11
12 @param nValue      Valor para arredondar
13 @param nDecimal    Quantidade de casas decimais para arredondar
14 @return nValue - Retorna um valor passado por parametro arredondando
15 @obs
16 @sample
17 */
18 //-----
19 Function STBRound( nValue , nDecimal )
20
21 Default nValue := 0 //Valor para arredondar
22 Default nDecimal := MsDecimais(STBGetCurrency()) //Quantidade de casas decimais para arredondar
23
24 ParamType 0 Var nValue AS Numeric Default 0
25 ParamType 1 Var nDecimal AS Numeric Default MsDecimais(STBGetCurrency())
26
27 nValue := Round( nValue , nDecimal )
28
29 Return nValue
30
31
32 //-----
33 /*{Protheus.doc} STBNoRound
34 Retorna um valor, truncando a parte decimal do valor especificado no
35 parametro de acordo com a quantidade de casas decimais solicitadas
36
37 @param nValue      Valor para arredondar
38 @param nDecimal    Quantidade de casas decimais para arredondar
39 @return nValue - Retorna um valor passado por parametro arredondando
40 @obs
41 @sample
42 */
43 //-----
44 Function STBNoRound( nValue , nDecimal )
45
46 Default nValue := 0 //Valor para arredondar
47 Default nDecimal := MsDecimais(STBGetCurrency()) //Quantidade de casas decimais para arredondar
48
49 ParamType 0 Var nValue AS Numeric Default 0
50 ParamType 1 Var nDecimal AS Numeric Default MsDecimais(STBGetCurrency())
51
52 nValue := NoRound( nValue , nDecimal )

```

```

53
54 Return nValue
55
56
57 //-----
58 /*{Protheus.doc} STBRoundNoRound
59 Trunca a parte decimal do valor especificado no parametro de
60 acordo com a quantidade de casas decimais solicitadas e depois
61 arredonda o valor truncado.
62
63 @param  nValue      Valor para arredondar
64 @param  nRDecimal   Quantidade de casas decimais para arredondar
65 @param  nTDecimal   Quantidade de casas decimais para truncar
66 @return nValue - Retorna um valor passado por parametro arredondando
67 @obs
68 @sample
69 */
70 //-----
71 Function STBRoundNoRound( nValue , nRDecimal , nTDecimal )
72
73 Default  nValue      := 0                      //Valor para arredondar
74 Default  nRDecimal   := MsDecimais(STBGetCurrency()) //Quantidade de casas decimais para arredondar
75 Default  nTDecimal   := MsDecimais(STBGetCurrency()) //Quantidade de casas decimais para truncar
76
77 ParamType 0 Var nValue      AS Numeric      Default 0
78 ParamType 1 Var nRDecimal   AS Numeric      Default MsDecimais(STBGetCurrency())
79 ParamType 2 Var nTDecimal   AS Numeric      Default MsDecimais(STBGetCurrency())
80
81 // Trunca
82 nValue := NoRound( nValue , nTDecimal )
83
84
85 // Arredonda o valor ja truncado
86 nValue := Round( nValue , nRDecimal )
87
88 Return nValue
89
90
91 //-----
92 /*{Protheus.doc} STBRoundCurrency
93 Trunca a parte decimal do valor especificado no parametro de
94 acordo com a quantidade de casas decimais solicitadas e depois
95 arredonda o valor truncado.
96
97 @param nValue      Valor a ser arredondado
98 @param nMoedaIni    Moeda de partida
99 @param nMoedaFinal  Moeda de destino
100 @param dData       Data base da Conversao
101 @param nCDecimal    Decimal para truncamento da Moeda
102 @param nTaxap       Taxap
103 @param nTaxad       Taxad
104 @param nRDecimal    Decimal para arredondamento final da Moeda

```

```

105
106 @return nValue - Retorna um valor passado por parametro arredondando
107 @obs
108 @sample
109 /*/
110 //-----
111 Function STBRoundCurrency( nValue      , nMoedaIni      , nMoedaFinal      , dData      ,;
112                          nCDecimal      , nTaxap      , nTaxad      , nRDecimal )
113
114
115 Default nValue      := 0 // Valor a ser arredondado
116 Default nMoedaIni    := 1 // Moeda de partida
117 Default nMoedaFinal  := 1 // Moeda de destino
118 Default dData        := dDataBase // Data base da Conversao
119 Default nCDecimal    := MsDecimais(STBGetCurrency()) // Decimal para truncamento da Moeda
120 Default nTaxap       := 0 // Taxap
121 Default nTaxad       := 0 // Taxad
122 Default nRDecimal    := MsDecimais(STBGetCurrency()) // Decimal para arredondamento final da Moeda
123
124
125 ParamType 0 Var nValue      AS Numeric      Default 0
126 ParamType 1 Var nMoedaIni    AS Numeric      Default 1
127 ParamType 2 Var nMoedaFinal  AS Numeric      Default 1
128 ParamType 3 Var dData        AS Date        Default dDataBase
129 ParamType 4 Var nCDecimal    AS Numeric      Default MsDecimais(STBGetCurrency())
130 ParamType 5 Var nTaxap       AS Numeric      Default 0
131 ParamType 6 Var nTaxad       AS Numeric      Default 0
132 ParamType 7 Var nRDecimal    AS Numeric      Default MsDecimais(STBGetCurrency())
133
134
135 // Converte valor na moeda atual para moeda solicitada
136 nValue      := xMoeda( nValue      , nMoedaIni      , nMoedaFinal      , dData      ,;
137                      nCDecimal      , nTaxap      , nTaxad      , nRDecimal )
138
139 // Arredonda o valor ja convertido anteriormente
140 nValue      := Round( nValue      , nRDecimal )
141
142 Return nValue
143
144
145 //-----
146 /*/{Protheus.doc} STBArred
147 Arredonda ou trunca o valor de acordo com o campo e a configuracao
148
149 @param nValue      Valor a ser arredondado
150 @param nDecimal     Quantidade de casas decimais para arredondar
151 @param cField       Campo de referencia para arredondamento
152 @param cRound       Define se Arredonda ou Trunca valor
153 @return nValue - Retorna um valor passado por parametro arredondando
154 @obs
155 @sample
156 /*/

```

```

157 //-----
158 Function STBArred( nValue , nDecimal , cField , cRound )
159 Local aArea      := GetArea()                // Guarda area
160 Local aAreaSX3   := SX3->(GetArea())          // Guarda area SX3
161 Local nDecimal   := 0                        // Quantidade de casas decimais para arredondar
162 Local cRound     := ""                      // Armazena configuração se Arredonda ou trunca valores
163
164 Default nValue    := 0                      // Valor para arredondar
165 Default nDecimal  := 0                      // Quantidade de casas decimais para arredondar
166 Default cField    := ""                     // Campo de referencia para arredondamento
167 Default cRound    := ""                     // Define se Arredonda ou Trunca valor
168
169 If nDecimal <= 0
170     If Empty(cField)
171         nDecimal := MsDecimais(STBGetCurrency())
172     Else
173         nDecimal := TamSX3(cField)[2]
174     EndIf
175 EndIf
176
177 If Empty(cRound)
178     cRound := STBRuleArred()
179 EndIf
180
181 //Valida se a impressora irá tratar a regra
182 If STBImpRnd5()
183     //Efetua arredondamento ou truncamento de acordo com o numero 5
184     cRound := STBRound5(nValue,cRound)
185 EndIf
186
187 If cRound == "R"
188     nValue := STBRound( nValue , nDecimal )
189 ElseIf cRound == "T"
190     nValue := STBNoRound( nValue , nDecimal )
191 EndIf
192
193 //Retorna o estado de entrada
194 RestArea(aAreaSX3)
195 RestArea(aArea)
196
197 LjGrvLog("STBArred", "cRound ==", cRound )
198
199 Return nValue
200
201 //-----
202 /*{Protheus.doc} STBRuleArred
203 Determina configuração se Arredonda ou Trunca valores
204
205 @param
206 @return cRound - Retorna configuração se trunca ou arredonda valores
207 @obs
208 @sample

```

```

209 /*/
210 //-----
211 Function STBRuleArred()
212
213 Local cRound      := ""      // Retorna se arredonda ou trunca valores
214 Local aDados      := {"13", space(20)} // Pega Estatus 13 do ECF para saber se trunca ou arredonda
215
216 /*/
217   Se utiliza impressora fiscal, utiliza o status da impressora
218   Senão, utiliza parametro MV_ARREFAT
219 /*/
220 If STFUseFiscalPrinter()
221
222   aRet := STFFireEvent( ProcName(0)      ,; // Nome do processo
223                        "STPrinterStatus" ,; // Nome do evento
224                        aDados            )
225
226   // 0-Arredonda 1-Trunca
227   If aRet[1] == 1
228     LjGrvLog( , "ECF-Trunca")
229     cRound := "T"
230   Else
231     LjGrvLog( , "ECF-Arredonda")
232     cRound := "R"
233   EndIf
234
235 Else
236
237   /*/
238   Dependendo do parametro Arredonda ou Trunca o valor
239   /*/
240   If ( SuperGetMv("MV_ARREFAT") == "S" )
241     cRound := "R"
242   Else
243     cRound := "T"
244   EndIf
245
246 EndIf
247
248 Return cRound
249
250 //-----
251 /*/{Protheus.doc} STBRound5
252 Rotina responsável por realizar tratamento quando o numero 5 for de decisão.
253 Casos:
254 - Se o algarismo decimal seguinte for menor que 5, o anterior não se modifica.
255 - Se o algarismo decimal seguinte for maior que 5, o anterior incrementa-se em uma unidade.
256 - Se o algarismo decimal seguinte for igual a 5, deve-se verificar o anterior, se ele for par não se modifica,
257   se ele for ímpar incrementa-se uma unidade.
258
259 Exemplos:
260 0 número 12,652 seria arredondado para 12,65

```

```

261 0 número 12,658 seria arredondado para 12,66
262 0 número 12,865 seria arredondado para 12,86
263 0 número 12,875 seria arredondado para 12,88
264
265 @param    nValue
266 @return    cRound - Se arredonda ou trunca
267 */
268 //-----
269 Function STBRound5(nValue,cCfgRndECF)
270 Local cRound      := "R"           //Tipo Arredonda ou Trunca
271 Local nUltVal     := 0             //Ultimo valor
272 Local nAntVal     := 0             //Penultimo Valor
273 Local cValue      := ""           //Numero convertido para caracter
274
275 Default nValue     := 0
276 Default cCfgRndECF := "" //Deve ser deixado o default vazio pra não interferir, caso param não enviado
277
278 /*Deve considerar a regra de arredondamento/truncamento da impressora */
279 If !Empty(cCfgRndECF)
280     cRound := cCfgRndECF
281 EndIf
282
283 If nValue > 0
284     nValue := NoRound(nValue,3)           //Trunca o valor para 3 casas
285     cValue := AllTrim(STR(nValue))
286     nUltVal := Val(SubStr(cValue,Len(cValue),1)) //Ultimo
287     nAntVal := Val(SubStr(cValue,Len(cValue)-1,1)) //Penultimo
288     //Valida se o ultimo numero é 5
289     If nUltVal == 5
290         //Valida se o numero anterior ao 5 é par, se for trunca
291         If nAntVal%2 == 0
292             cRound := "T"
293         EndIf
294     EndIf
295 EndIf
296
297 Return cRound
298
299 //-----
300 /*{Protheus.doc} STBImpRnd5
301 Verifica se a impressora irá utilizar a regra para arredondar ou truncar o valor
302 conforme definido na função STBRound5
303
304 @return    lRet
305 */
306 //-----
307 Function STBImpRnd5()
308 Local lRet := .F.
309 Local lPos := STFIsPOS() // Pos?
310
311 If Empty(AllTrim(cImpressora))
312     If lPos

```

```
313         cImpressora := STFGetStation("IMPFISC") //modelo da impressora configurada
314     Else
315         cImpressora := LjGetStation("IMPFISC") //modelo da impressora configurada
316     EndIf
317 EndIf
318
319 cImpressora := Upper(cImpressora)
320
321 If 'BEMATECH' $ cImpressora .And. 'FI' $ cImpressora .And. ('4000' $ cImpressora .Or. '4200' $ cImpressora)
322     LjGrvLog("STBImpRnd5()", "ECF-BEmatech")
323     lRet := .T.
324 EndIf
325
326 Return lRet
```


Framework/SaveTable/STFSaveTab.prw

325 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STFSAVETAB.CH"
4
5 //-----
6 /*{Protoheus.doc} STFSaveTab
7 Realiza a gravacao dos campos passado em array no Alias correspondente
8
9 @param      cAlias      Alias para gravacao
10 @param  aArray      Array com Campos e valores a serem gravados
11 @param  lAppend      Indica se cria novo registro na tabela
12 @param  lUnLock      Indica se trava o registro para alteracao
13
14 @return  lRet      Retorna se conseguiu gravar
15 @obs
16 @sample
17 */
18 //-----
19 Function STFSaveTab( cAlias , aArray , lAppend , lUnLock )
20
21 Local aArea      := GetArea()      // Guarda area
22
23 Local lRet      := .T.      // Retorno
24
25 Default cAlias      := ""      // Alias para gravacao
26 Default aArray      := {}      // array com Campos e valores a serem gravados
27 Default lAppend      := .F.      // Indica se cria novo registro na tabela
28 Default lUnLock      := .T.      // Indica se trava o registro para alteracao
29
30 ParamType 0 Var      cAlias      As Character      Default      ""
31 ParamType 1 Var      aArray      As Array      Default      {}
32 ParamType 2 var      lAppend      As Logical      Default      .F.
33 ParamType 3 var      lUnLock      As Logical      Default      .T.
34
35 If AliasIndic(cAlias)
36   DbSelectArea(cAlias)
37   CoNout(STR0001 + cAlias)      //'TABELA: '
38
39   If !lAppend .And. !lUnLock
40     // Verifica se o arquivo esta disponivel para alteracao
41     lRet := MsRLock()
42     CoNout(STR0002 + cAlias)      //'MsRLock: '
43   Else
44     // Trava o registro para alteracao
45     lRet := RecLock(cAlias, lAppend)
46     CoNout(STR0003 + cAlias)      //'RecLock: '
47   EndIf
48
49   If lRet
50     // Insere os registros do array
51     AEval(aArray, {|x| FieldPut(FieldPos(x[1]), x[2])})
52

```

```

53      // Se travou registro, confirma a transacao e destrava o registro
54      If lUnLock
55          dbCommit()
56          MsUnLock()
57          CoNout(STR0004 + cAlias)      //'dbCommit: '
58          LjGrvLog(STR0005, 'MsUnLock ' + cAlias) //'MsUnLock '
59      EndIf
60
61      Else
62          ConOut(STR0006+cAlias+STR0007+AllTrim(Str(Recno()))+".") //"Impossível travar arquivo" ### "Registro: "
63          LjGrvLog("STFSaveTab", STR0006+cAlias+STR0007+AllTrim(Str(Recno()))+".")      //"Impossível travar arquivo" ### "Registro: "
64      EndIf
65  Else
66      ConOut(STR0008+cAlias+STR0009) //"A tabela " ### " não existe na retaguarda!"
67  EndIf
68
69  RestArea(aArea)
70
71  Return lRet
72
73
74  //-----
75  /*{Protheus.doc} STFSLICreate
76  Realiza a gravacao dos campos da tabela de Monitoramento das Estacoes(SLI)
77
78  @param cStation      Estacao
79  @param cType          Tipo operacao
80  @param cMsg           Mensagem
81  @param cChoice       Decisao
82  @param lUnLock       Trava o registro
83  @param cOperName     Usuario
84  @param dDate         Data
85  @param cHour         Hora
86
87  @param cAlias        Alias a subir ao server
88  @param nRecno        Numero do recno a subir para server
89  @param cFunc         Opcional - Funcao a ser executada no server para subir registro
90
91  @return lOpenCash    Retorna se o caixa esta aberto
92  @obs
93  @sample
94  */
95  //-----
96
97  Function STFSLICreate( cStation      , cType          , cMsg          , cChoice      , ;
98                      lUnLock , cOperName      , dDate      , cHour      , ;
99                      cAlias      , nRecno          , cFunc          )
100
101  Local aSLI          := {}      // Array Usado para gravar SLI
102  Local lRet          := .F.      // Retorno
103  Local lAppend       := .F.      // Cria novo registro
104  Local cLstSeq       := "0"     //Ultima Sequencia

```

```

105 Local lLi_Seq      := SLI->(ColumnPos("LI_SEQ")) > 0 //Existe campo sequencia?
106 Local cChave       := "" //Chave de Busca
107 Local nTamLiSeq    := 0 //Tamanho do campo LI_SEQ
108
109 Default cStation    := STFGetStat("CODIGO") // Estacao
110 Default cType       := "" // Tipo operacao
111 Default cMsg        := "" // Mensagem
112 Default cChoice     := "ABANDONA" // Decisao
113 Default lUnLock     := .T. // Trava o registro
114 Default cOperName   := cUserName // Usuario
115 Default dDate       := dDataBase // Data
116 Default cHour       := Time() // Hora
117 Default cAlias      := "" // Alias a subir ao server
118 Default nRecno      := 0 // Numero do recno a subir para server
119 Default cFunc       := "" // Opcional - Funcao a ser executada no server para subir registro
120
121 ParamType 0 Var cStation As Character Default STFGetStat("CODIGO")
122 ParamType 1 Var cType As Character Default ""
123 ParamType 2 Var cMsg As Character Default ""
124 ParamType 3 Var cChoice As Character Default "ABANDONA"
125 ParamType 4 Var lUnLock As Logical Default .T.
126 ParamType 5 Var cOperName As Character Default cUserName
127 ParamType 6 Var dDate As Date Default dDataBase
128 ParamType 7 Var cHour As Character Default Time()
129 ParamType 8 Var cAlias As Character Default ""
130 ParamType 9 Var nRecno As Numeric Default 0
131 ParamType 10 Var cFunc As Character Default ""
132
133 If lLi_Seq
134     nTamLiSeq := TamSx3("LI_SEQ")[1]
135     cLstSeq := Replicate("0", nTamLiSeq)
136     cLstSeq := Somal(cLstSeq, nTamLiSeq)
137 EndIf
138
139 SLI->(DbSetOrder(1)) //LI_FILIAL+LI_ESTACAO+LI_TIPO+LI_SEQ
140 cChave := xFilial("SLI")+PadR(cStation,TamSx3("LI_ESTACAO")[1])+PadR(cType, TamSx3("LI_TIPO")[1])
141 lAppend := !SLI->(DbSeek(cChave))
142
143 If !lAppend // Se registro ja existe
144
145     If cChoice == "ABANDONA"
146         lRet := .F.
147     ElseIf cChoice == "SOBREPOE"
148         lAppend := .F.
149         lRet := .T.
150     ElseIf cChoice == "NOVO"
151         lAppend := .T.
152         lRet := .T.
153     EndIf
154 Else
155     lRet := .T.
156 EndIf

```

```

157
158 //Procura a última sequencia enviada
159 If lAppend .AND. lRet .AND. lLi_Seq .AND. SLI->(Found())
160
161     //Artifício para ir até o último registro rapidamente
162     //Pesquisa DbSeek cuja chave termina com 9999999999, para depois voltar o dbskip um registro anterior
163     SLI->(DbSeek(cChave + Replicate("9", nTamLiSeq),.T.)) //com Softseek
164     If SLI->(EOF())
165         SLI->(DbGoBottom())
166     Else
167         SLI->(DbSkip(-1))
168     EndIf
169     cLstSeq := SLI->LI_SEQ
170
171     Do While SLI->(DbSeek(cChave+cLstSeq))
172         cLstSeq := Somal(cLstSeq, nTamLiSeq)
173     EndDo
174 EndIf
175
176 If lRet
177     aSLI := {{ "LI_FILIAL" , xFilial("SLI") } , ;
178             { "LI_ESTACAO" , cStation } , ;
179             { "LI_TIPO" , cType } , ;
180             { "LI_USUARIO" , cOperName } , ;
181             { "LI_DATA" , dDate } , ;
182             { "LI_HORA" , cHour } , ;
183             { "LI_MSG" , cMsg } , ;
184             { "LI_ALIAS" , cAlias } , ;
185             { "LI_UPREC" , nRecno } , ;
186             { "LI_FUNC" , cFunc } } }
187     If lLi_Seq
188         aAdd(aSLI, {"LI_SEQ", cLstSeq})
189     EndIf
190
191     lRet := STFSaveTab("SLI", aSLI, lAppend, lUnLock)
192 EndIf
193
194 Return lRet
195
196
197 //-----
198 /*{Protheus.doc} STFPesqTab
199 Realiza a pesquisa na tabela e retorna as informacoes solicitadas em parametros
200
201 @param cTabela    Alias para gravacao
202 @param nOrder     Ordem de busca
203 @param cChave     Chave com campos da busca
204 @param cBusca     Valores da busca
205 @param aArray     Array com Campos e valores a serem retornados
206 @param lFirst     Considera apenas o primeiro registro encontrado - Opcional
207 @param cValid     Valores da validacao - Opcional
208

```

```

209 @return aRet          Retorna informacoes solicitadas
210 @obs
211 @sample
212 /*/
213 //-----
214 Function STFPesqTab( cTabela , nOrder , cChave , cBusca ;
215                   aArray , lFirst , cValid )
216
217 Local aRet      := {}          // Retorno
218 Local aAux      := {}          // Array auxiliar
219 Local nI        := 0           // Contador
220 Local xRet      := Nil         // Retorno de valor dos campos
221 Local lValid    := .F.         // Verifica se recebeu validacao especifica
222
223
224 Default cTabela  := ""         // Alias para gravacao
225 Default nOrder   := 0          // Ordem de busca
226 Default cChave   := ""         // Chave com campos da busca
227 Default cBusca   := ""         // Chave com campos da busca
228 Default aArray   := {}         // array com Campos e valores a serem retornados
229 Default lFirst   := .F.        // Busca apenas primeiro registro que encontrar
230 Default cValid   := ""         // Valores da validacao opcional
231
232 ConOut(STR0010 + cTabela )      //"Consulta de informações na base. Alias: "
233 LjGrvLog("STFPesqTab", STR0010 +cTabela ) //"Consulta de informações na base. Alias: "
234
235 ConOut(cTabela + "," + cChave + "," + cBusca)
236
237 // Verifica se recebeu validacao especifica
238 lValid := ! Empty(cValid)
239
240 If ValType(aArray) == "A" .AND. Len(aArray) > 0
241   DbSelectArea(cTabela)
242   DbSetOrder(nOrder)
243   If DbSeek(cBusca)
244
245     ConOut(STR0011 +cTabela )      //"Encontrou informações em sua busca. Alias: "
246     LjGrvLog("STFPesqTab", STR0011 +cTabela ) //"Encontrou informações em sua busca. Alias: "
247
248     While &(cChave) == cBusca .AND. !EOF()
249
250       If !lValid .OR. (lValid .AND. &(cValid) )
251         aAux := {}
252         For nI := 1 To Len(aArray)
253
254           xRet := FieldGet( FieldPos( aArray[nI]) )
255
256           AADD(aAux,{aArray[nI],xRet})
257
258         Next nI
259
260       AADD( aRet, aAux )

```

```

261
262         If lFirst //Busca apenas primeiro registro e retorna
263             Exit
264         EndIf
265
266     EndIf
267
268     &(cTabela)->(DbSkip())
269
270 EndDo
271
272 EndIf
273 EndIf
274
275 Return aRet
276
277
278 //-----
279 /*{Protoheus.doc} STFAltTab
280 Altera os campos da tabela de acordo com os parametros recebidos
281 @param cTabela  Alias para gravacao
282 @param nOrder   Ordem de busca
283 @param cBusca   Valores da busca
284 @param aCampos - Informa o campo ou os campos que deseja atualizar
285 @return lSet - Retorna se o campo foi gravado ou não
286 @obs
287 @sample
288 */
289 //-----
290 Function STFAltTab( cTabela , nOrder , cBusca , aCampos )
291
292 Local aArea      := GetArea()    //Salva area
293 Local lRet       := .F.         //Retorno da função
294 Local nI         := 0           //Contador
295
296 Default cTabela  := ""          // Alias para gravacao
297 Default nOrder   := 0           // Ordem de busca
298 Default cBusca   := ""          // Chave com campos da busca
299 Default aCampos  := {}          // Campos a alterar
300
301 ConOut(STR0012 + cTabela ) //"Alterar informações da base. Alias: "
302 LjGrvLog("STFAltTab",STR0012 + cTabela ) //"Alterar informações da base. Alias: "
303
304 DbSelectArea(cTabela)
305 DbSetOrder(nOrder)
306 If DbSeek(cBusca)
307
308     If RecLock(cTabela, .F.)
309
310         For nI := 1 To Len(aCampos)
311             FieldPut( FieldPos( aCampos[nI, 1]), aCampos[nI, 2])
312         Next nI

```

```
313
314      &(cTabela)->(MsUnLock())
315      lRet := .T.
316      ConOut(STR0013 + cTabela )      //"Informações alteradas com sucesso. Alias "
317      LjGrvLog("STFAltTab",STR0013 + cTabela  )  //"Informações alteradas com sucesso. Alias "
318
319      EndIf
320
321 EndIf
322
323 RestArea(aArea)
324
325 Return lRet
```

Framework/ScreenColor/STFScreenColor.prw

112 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "STFScreenColor.CH"
3
4 //-----
5 /*{Protheus.doc} STFScreenColor
6 Função para alterar a cor da Janela do POS
7 @obs
8 @sample
9 */
10 //-----
11 Function STFScreenColor()
12
13 Local oDlg      := NIL      //Janela
14 Local oColorT   := NIL      //Color da Janela
15 Local oButton1  := NIL      //botao 1
16 Local oButton2  := NIL      //Botão 2
17 Local nColor    := 0        //Codigo da Cor
18
19 DEFINE MSDIALOG oDlg FROM 0,0 TO 430,600 PIXEL TITLE STR0001 //"Cores"
20
21     oColorT := tColorTriangle():New(1, 1, oDlg, 300, 200)
22     oColorT:SetColorIni( 2 )
23     nColor  := oColorT:RetColor()
24
25     oButton1:=tButton():New(200,1,STR0002,oDlg,{||nColor := STFScrRec(@oColorT,oDlg)},100,20,,,,.T.)    //"Gravar Cor"
26     oButton2:=tButton():New(200,200,STR0003,oDlg,{||oDlg:End()},101,20,,,,.T.)    //"Fechar"
27     oButton3:=tButton():New(200,100,STR0004,oDlg,{||nColor := STFSetDefault(@oColorT,oDlg)},100,20,,,,.T.)    //"Padrão"
28
29
30 ACTIVATE MSDIALOG oDlg CENTERED
31
32 Return
33
34 //-----
35 /*{Protheus.doc} STFScrRec
36 Carrega e Grava a Cor
37 @param oColor Objeto Paleta
38 @param oDlg Janela
39 @return oColorT:RetColor() Codigo da Cor
40 @obs
41 @sample
42 */
43 //-----
44 Static Function STFScrRec(oColorT, oDlg)
45 Local nAzul      := 0        //Codigo da Cor Azul
46 Local nRestoAzul := 0        //Resto do Codigo da Cor Azul
47 Local nVerde     := 0        //Codigo da Cor Verde
48 Local nVermelho  := 0        //Codigo da Cor Vermelho
49 Local cRet       := ""       //Retorno da função
50
51 DEFAULT oColorT := NIL
52 DEFAULT oDlg    := NIL

```



```

53
54 nColor := oColorT:RetColor()
55
56 nAzul      := Int(nColor/65536)
57 nRestoAzul := Mod(nColor,65536)
58 nVerde     := Int(nRestoAzul/256)
59 nVermelho  := Int(Mod(nRestoAzul,256))
60
61 cRet := decToHex(nVermelho,2) + decToHex(nVerde,2) + decToHex(nAzul,2)
62
63 PutMv("MV_LJCOLOR" , cRet)
64
65 If !Empty(cRet)
66     MsgAlert (STR0005) //"Cor gravada com sucesso"
67 EndIf
68
69
70 Return oColorT:RetColor()
71
72
73 //-----
74 /*/{Protheus.doc} STFSetDefault
75 Carrega a Cor Default
76 @param oColor Objeto Paleta
77 @param oDlg Janela
78 @return oColorT:RetColor() Codigo da Cor
79 @obs
80 @sample
81 */
82
83 //-----
84 Static Function STFSetDefault(oColorT, oDlg)
85 Local nAzul      := 0 //Codigo da Cor Azul
86 Local nRestoAzul := 0 //Resto do Codigo da Cor Azul
87 Local nVerde     := 0 //Codigo da Cor Verde
88 Local nVermelho  := 0 //Codigo da Cor Vermelho
89 Local cRet       := "" //Retorno da função
90 Local cDefCor    := "07334C"
91
92
93 DEFAULT oColorT := NIL
94 DEFAULT oDlg    := NIL
95
96 nColor := oColorT:RetColor()
97
98 nAzul      := Int(nColor/65536)
99 nRestoAzul := Mod(nColor,65536)
100 nVerde     := Int(nRestoAzul/256)
101 nVermelho  := Int(Mod(nRestoAzul,256))
102
103 cRet := decToHex(nVermelho,2) + decToHex(nVerde,2) + decToHex(nAzul,2)
104

```

```
105 PutMv("MV_LJCOLOR" , cDefCor)
106
107 If !Empty(cRet)
108     MsgAlert (STR0005) //"Cor gravada com sucesso"
109 EndIf
110
111
112 Return oColorT:RetColor()
```

Framework/SendMail/STFSendMail.prw

313 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'Protheus.ch'
2 #INCLUDE "AP5MAIL.CH"
3 #INCLUDE "STFSENDMAIL.CH"
4 #INCLUDE "POSCSS.CH"
5
6 //-----
7 /*{Protheus.doc} STFSendMail
8 Envia e-mail ao Superior
9
10 @param
11 @return Nil
12 @obs
13 @sample
14 */
15 //-----
16 Function STFSendMail(cParamText, cAssunto, lFormatText, lEnviaConf)
17
18 Local cError      := "" // Bloco de erro
19 Local lOk         := .T. // Se conseguiu conectar no servidor SMTP
20 Local lSendOk     := .T. // Se o e-mail foi enviado com sucesso
21 Local cMailConta  := SuperGetMv("MV_RELFROM", "") // Conta
22 Local cMailServer := SuperGetMv("MV_RELSERV", "") // Servidor SMTP
23 Local cMailSenha  := SuperGetMv("MV_RELPSW", "") // Senha
24 Local cMailToSup  := SuperGetMv("MV_LJEMSUP", "") // Mensagem customizada do cliente
25 Local aTo         := UsrSuper() // Retorna dados do superior
26 Local lRet        := .T. // Retorno se existe e-mail do superior
27 Local cCaixa      := STDNumCash() // Retorna o código do Caixa que está logado
28 Local cTexto      := "" // Texto a ser enviado no e-mail
29 Local aFormatTxt  := {} // Armazena a mensagem formatada
30 Local nX          := 0 // Contador
31
32 Default lFormatText := .F. // Se deverá formatar o texto
33 Default cParamText  := "" // Corpo do e-mail
34 Default cAssunto    := STR0003 // "Info. Limite Sangria"
35 Default lEnviaConf  := .T. // Envio confirmação para a tela se o e-mail foi enviado
36
37 If Len(aTo) >= 1
38     If Len(aTo[1]) >= 14
39         If Empty(aTo[1][14]) // Se não tiver e-mail cadastrado do superior não tenta enviar
40             lRet := .F.
41         EndIf
42     Else
43         lRet := .F.
44     EndIf
45 Else
46     lRet := .F.
47 EndIf
48
49
50 If lFormatText
51
52     aFormatTxt := STRTOKARR(cParamText, Chr(10)+Chr(13))

```

```

53     cParamText := ""
54
55     For nX = 1 to Len(aFormatTxt)
56         cParamText += aFormatTxt[nX] + "<br>"
57     Next nX
58
59 EndIf
60
61 If lRet
62
63     If Empty(cMailToSup)
64         cTexto := cParamText
65     Else
66         If ExistFunc(cMailToSup)
67             cTexto := &(cMailToSup) //Macro executa
68         Else
69             cTexto := cParamText
70         EndIf
71     EndIf
72
73     CONNECT SMTP SERVER cMailServer ACCOUNT cMailConta PASSWORD cMailSenha RESULT lOk
74
75     SEND MAIL FROM AllTrim(cMailConta) to aTo[1][14] SUBJECT cAssunto BODY cTexto RESULT lSendOk
76
77     If !lSendOk
78         Conout (STR0004 + cError) //"Erro no envio do email"
79     Else
80         If lEnviaConf
81             MsgAlert(STR0005) // "E-mail enviado com sucesso"
82         EndIf
83     EndIf
84
85     If lOk
86         DISCONNECT SMTP SERVER
87     EndIf
88 Else
89     If IsInCallStack("STDVERLIMSAN") //Só posso emitir alerta se veio do controle de limite de sangria
90         MsgAlert("Favor cadastrar com urgência o código e o e-mail do supervisor. Avise a gerência!") //"Favor cadastrar com urgência o código e o e-mail do supervisor. Avise
          a gerência!"
91     EndIf
92
93 Endif
94
95 Return
96
97
98 //-----
99 /*{Protheus.doc} UsrSuper
100 Busca informações do Superior do Caixa logado
101
102 @param
103 @return aUserSup

```

```

104 @obs
105 @sample
106 */
107 //-----
108 Static Function UsrSuper()
109
110 Local nI          := 0    // Contador
111 Local cSuperiores := ""    // Armazena o Superior
112 Local aReturn     := {}   // Retorno da função do Frame
113 Local aUserSup    := {}   // Array com todos os dados do Superior
114
115 aReturn := FWSFUsrSup(__UserID)
116
117 For nI := 1 to Len(aReturn)
118     cSuperiores := aReturn[nI]
119 Next nI
120
121 If PswSeek(cSuperiores)
122     aUserSup := PswRet()
123 EndIf
124
125 Return aUserSup
126
127
128 //-----
129 /*{Protheus.doc} STFSendMail
130 Envia e-mail a partir das configuracoes realizadas
131 no SIGACFG Ambiente/E-mail-Proxy/Configurar
132
133 @param    cAssunto    Assunto do email
134 @param    cMsgEmail   Mensagem de email
135 @param    lFormatText Formata o texto do email com quebra de linhas
136 @param    cDestinat   Destinatarios separado por ;
137 @param    cErro       Controle de erro do envio usar por referencia ;
138 @return   lSendOk     Email enviado
139 @obs
140 @sample
141 */
142 //-----
143 Function STFMail( cAssunto , cMsgEmail, lFormatText , cDestinat ,;
144                 cErro      )
145
146 Local  cError      := ""    // Bloco de erro
147 Local  lOk         := .F.   // Se conseguiu conectar no servidor SMTP
148 Local  lSendOk     := .F.   // Se o e-mail foi enviado com sucesso
149 Local  cMailFrom   := AllTrim(SuperGetMv("MV_RELFROM", "")) // Mail From
150 Local  cMailConta  := AllTrim(SuperGetMv("MV_RELACNT", "")) // Conta
151 Local  cMailServer := AllTrim(SuperGetMv("MV_RELSERV", "")) // Servidor SMTP
152 Local  cMailSenha  := AllTrim(SuperGetMv("MV_RELPSW", "")) // Senha
153 Local  lAutentic   := SuperGetMv("MV_RELAUTH", .F.) // Usa Autenticacao
154 Local  cTexto      := ""    // Texto a ser enviado no e-mail
155 Local  aFormatTxt  := {}    // Armazena a mensagem formatada

```

```

156 Local  nX          := 0           // Contador
157 Local  cUser       := ""          // Usuario para tentativa de atenticacao
158 Local  nA          := 0           // Separador
159
160 Default cAssunto    := ""          // Assunto do email
161 Default cMsgEmail   := ""          // Corpo do e-mail
162 Default lFormatText := .F.         // Se deverá formatar o texto
163 Default cDestinat   := ""          // Destinatario
164 Default cErro       := ""          // Controla erro de envio do email uar por referencia
165
166 If lFormatText
167
168     aFormatTxt := STRTOKARR(cMsgEmail,Chr(10)+Chr(13))
169
170     For nX = 1 to Len(aFormatTxt)
171         cTexto += aFormatTxt[nX] + "<br>"
172     Next nX
173 Else
174     cTexto := cMsgEmail
175 EndIf
176
177
178 CONNECT SMTP SERVER cMailServer ACCOUNT cMailConta PASSWORD cMailSenha RESULT lOk
179
180 If lOk
181
182     //Caso utiliza autenticacao
183     If lAutentic
184         lOk := MailAuth(cMailConta, cMailSenha)
185         //Se nao conseguiu fazer a Autenticacao usando o E-mail completo,
186         //tenta fazer a autenticacao usando apenas o nome de usuario do E-mail
187         If lOk
188             LjGrvLog( "STFMail" , "Usuario autenticado." )
189         Else
190             nA := At("@",cMailConta)
191             cUser := If(nA>0,Subs(cMailConta,1,nA-1),cMailConta)
192             lOk := MailAuth(cUser, cMailSenha)
193         Endif
194     Endif
195
196
197 If lOk
198     SEND MAIL FROM cMailFrom to cDestinat SUBJECT cAssunto BODY cTexto RESULT lSendOk
199
200     If lSendOk
201         LjGrvLog( "STFMail" , "Email Enviado com sucesso." )
202     Else
203         GET MAIL ERROR cErro
204         LjGrvLog( "STFMail" , "Não conseguiu enviar email. " + cErro )
205         LjGrvLog( "STFMail" , "Caso tenha dificuldades no envio de e-mail tente setar a porta do smtp. " + ;
206             "Exemplo: smtp.suaempresa.com.br:587 ou smtp.suaempresa.com.br:465" )
207     Endif
208 EndIf

```

```

208         EndIf
209
210         DISCONNECT SMTP SERVER
211     Else
212         LjGrvLog( "STFMail" , "Usuario não autenticado." )
213     EndIf
214
215
216 Else
217     GET MAIL ERROR cErro
218     LjGrvLog( "STFMail" , "Não conseguiu conectar no serviço SMTP. " + cErro )
219 EndIf
220
221 Return lSendOk
222
223
224 //-----
225 /*{Protheus.doc} STFMailTest
226 Tela para teste de envio de email
227
228 @param
229 @return      Nil
230 @obs
231 @sample
232 */
233 //-----
234 Function STFMailTest()
235
236 Local aRes           := GetScreenRes() // Recupera Resolução atual
237 Local nWidth         := aRes[1]       // Largura
238 Local nHeight        := aRes[2]       // Altura
239 Local oModal         := Nil           //Dialog Modal
240 Local oContainer     := Nil           //Container
241 Local oPanel         := Nil           //Painel
242 Local oFont          := TFont():New("Verdana",,014,,.T.,,,,F.,.F.)//Fontes
243 Local oGetMail       := Nil           // Obj end email
244 Local cEmail         := Space(250)    //Destinatário email
245 Local nColl         := 20             //Coluna Esquerda padrao
246
247 oModal := FWDIALOGModal():New()
248
249 oModal:SetBackground(.F.)
250 oModal:SetTitle(STR0007) //"Teste de Envio de email"
251
252 //Monta tela com 30% do tamanho Total
253 nWidth := (nWidth/2) * 0.3
254 nHeight := (nHeight/2) * 0.3
255
256 oModal:SetFreeArea(nWidth,nHeight)
257 oModal:createDialog()
258 oModal:createFormBar()
259

```

```

260 oPanel := oModal:getPanelMain()
261
262 oModal:AddButton( STR0012 , {|| STFTstSendMail(cEmail) } , ; //"Enviar E-mail"
263                "" , , .T., .F., .T., {||.T.} )
264
265 oModal:addCloseButton({|oModal:Deactivate()}, STR0013 )//"Sair"
266
267 @ 20,nColl SAY oSayNomeParam PROMPT STR0014 SIZE (nWidth * 0.8) ,50 FONT oFont OF oPanel PIXEL//"Digite o destinatário de E-mail"
268
269 //"Get email"
270 @ 40,nColl MSGET oGetMail VAR cEmail SIZE (nWidth * 0.8) , 20 OF oPanel PIXEL
271 oGetMail:SetCSS( POSCSS(CSS_GET_FOCAL) )
272
273 oModal:Activate()
274
275 Return Nil
276
277
278 //-----
279 /*{Protheus.doc} STFValidEmail
280 Realiza teste de envio de email
281
282 @param      cEmail - Email a ser validado
283 @return      lRet - Retorno do teste de email
284 @obs
285 @sample
286 */
287 //-----
288 Static Function STFTstSendMail(cEmail)
289
290 Local lRet      := .F.  //Retorno
291 Local cError    := ""   //Bloco de erro
292
293 Default cEmail := ""
294
295 cEmail := AllTrim(cEmail)
296
297 If !Empty(cEmail) .AND. ISEMAIL(cEmail)
298
299     If STFMail( STR0007 , STR0007 , .F. , cEmail, ;//"Teste de envio de E-mail"
300              @cError ) //"Enviar E-mail"
301
302         FWAlertSuccess("",STR0005) //"E-mail enviado com sucesso"
303         lRet           := .T.
304     Else
305         FWAlertError( STR0008 + CRLF + ; //"Verifique se o destinatário existe e revise as configurações de E-mail no Módulo SIGACFG."
306                     STR0009 + cError ,STR0004) //" Erro: ""Erro no envio do email"
307     EndIf
308
309 Else
310     FWAlertError(STR0010,STR0011) //"Verifique o E-mail digitado, informar apenas um e-mail para o teste." "E-mail Inválido"
311 EndIf

```



```
312  
313 Return lRet
```

Framework/Start/STFCfg.prw

497 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "AUTODEF.CH"
5
6 //STFCFG - carrega configuracoes globais no inicio do sistema
7
8 Static lMobilePDV := Nil           // Verifica se é PDV Mobile
9 Static lPDVOnLine := .F.          // Verifica se é PDV On-Line
10 //-----
11 /*{Protheus.doc} STFCnfAmb
12 Classe de Inicialização do Sistema, carrega configuracoes
13 @param
14 @return Self
15 @obs
16 @sample
17 */
18 //-----
19 Class STFCnfAmb
20
21     Data lMultCoin
22     Data lPafEcf
23     Data lUseECF
24     Data lUsesNotFiscal
25     Data lTEFMex
26     Data lConfCash
27     Data lEcfArg
28     Data lImpExtChq
29     Data lCredDF
30     Data lSaveCmdFch
31     Data lCanChoose
32     Data lFiCanCancel
33     Data lCanActSale
34     Data lVldCanLastSale
35     Data lseDocChoose
36     Data lGlobFact
37     Data lCtrlFol
38     Data cRFDSHOWObs
39     Data cRFDObrigat
40     Data lRuleDiscShow
41     Data lSugRuleDisc
42     Data nVlrRngSugRD
43     Data cCtrlDesc
44     Data lUsaDisplay
45     Data lUsaTef
46     Data lOnlyMenuFiscal
47     Data lUseSAT
48     Data lMobile
49     Data cIntegration
50     Data lNFCETSS
51     Data cTypeOperation
52     Data lChangePay

```

```

53 Data lUseNFCE
54 Data lPAFNFCe
55
56 Method New()           //Construtor
57 Method LoadConfig()    //Carrega configuracoes gerais
58
59 EndClass
60
61
62 //-----
63 /*{Protheus.doc} STFConfAmb
64 Metodo construtor
65 @param
66 @return Self
67 @obs
68 @sample
69 */
70 //-----
71 Method New() Class STFConfAmb
72 Return Self
73
74
75 //-----
76 /*{Protheus.doc} LoadConfig
77 Carrega as configuracoes
78 @param
79 @return Self
80 @obs
81 @sample
82 */
83 //-----
84 Method LoadConfig() Class STFConfAmb
85
86 // Trabalha com multimoeda
87 Self:lMultCoin := cPaisLoc <> "BRA"
88
89 // Configuracao para a NFC-e
90 If IIF(STFGetStation("NFCE") == Nil,.F.,STFGetStation("NFCE"))
91   Self:lUseECF := .F.
92 Else
93   // Trabalha com Impressora Fiscal ECF
94   Self:lUseECF := STFProFile(3)[1]
95 EndIf
96
97 //Configuração para SAT
98 If Iif(STFGetStation("USESAT") == NIL, .F., STFGetStation("USESAT"))
99   Self:lUseECF := .F.
100 EndIf
101
102 // Trabalha com Impressora nao Fiscal
103 Self:lUsesNotFiscal := STWVldEcFptg()
104

```

```

105 // Trabalha com PAFECE
106 Self:lPafEcf := STBIsPAF()
107
108 // Trabalha com TEF do Mexico
109 Self:lTEFMex := cPaisLoc == "MEX"
110
111 // Trabalha com conferencia de caixa
112 Self:lConfCash := SuperGetMV( "MV_LJCONFF",,.F. )
113
114 // Trabalha com ECF da argentina
115 Self:lEcfArg := cPaisLoc == "ARG"
116
117 // Trabalha com impressao extensa do cheque
118 Self:lImpExtChq := cPaisLoc <> "BRA"
119
120 // Trabalha com programa de credito para DF
121 Self:lCredDF := cPaisLoc $ "BRA" .AND. SM0->M0_ESTCOB $ "DF"
122
123 // Trabalha com armazenamento do comando de fechamento
124 Self:lSaveCmdFch := cPaisLoc $ "EUA|POR"
125
126 // Verifica se pode escolher a venda a ser cancelada
127 Self:lCanChoose := cPaisLoc <> "BRA"
128
129 // Verifica se pode cancelar vendas ja finalizadas
130 Self:lFiCanCancel := cPaisLoc $ "CHI|COL"
131
132 // Verifica se pode cancelar venda atual em andamento
133 Self:lCanActSale := cPaisLoc $ "CHI|COL"
134
135 // Valida se pode cancelar somente a ultima venda
136 Self:lVldCanLastSale := cPaisLoc == "BRA"
137
138 // Verifica se escolhe venda a ser Cancelada a Partir do doc
139 Self:lseDocChoose := cPaisLoc == "MEX"
140
141 // Valida se faz verificacao de factura global
142 Self:lGlobFact := cPaisLoc == "MEX"
143
144 // Trabalha com controle de formularios
145 Self:lCtrlFol := SuperGetMV("MV_CTRLFOL",,.F.) .AND. cPaisLoc $ "CHI|COL"
146
147 // Motivo de desconto Exibe observação na interface
148 // "N" - Não Aparece      "B" - Aparece e é obrigatório   "P" - Aparece e é opcional
149 Self:cRFDShowObs := "N"
150
151 // Motivo de desconto Exibe observação na interface obrigatoriamente
152 // "N" - Não Aparece      "B" - Aparece e é obrigatório   "P" - Aparece e é opcional
153 Self:cRFDObriगत := "B"
154
155 // Mostra regras de descontos aplicadas
156 Self:lRuleDiscShow := .T.

```

```

157
158 // Exibe sugestao de regra de desconto
159 Self:lSugRuleDisc := .F.
160
161 // Valor do range para exibicao da sugestao de regra de desconto
162 Self:nVlrRngSugRD := 50
163
164 // Fazer Facilitador para configuração se usa desconto via usuário, via regra ou ambos
165 // "U" - Usuário      "R" - Regra de Desconto "A" - Ambos
166 Self:cCtrlDesc := SuperGetMV("MV_LJCFDES",.T.,"A")
167
168 // Verifica se a estacao possui Display
169 Self:lUsaDisplay := !Empty(STFGetStation("DISPLAY"))
170
171 // Verifica se usa TEF
172 Self:lUsaTef := STFProFile(2,,,,.T.)[1]
173
174 // Verifica se habilita somente o menu fiscal na tela
175 // Padrao sempre falso, alterado na entrada do sistema
176 Self:lOnlyMenuFiscal := .F.
177
178 // Verifica se utiliza SAT
179 If SLG->(ColumnPos("LG_USESAT")) > 0
180     Self:lUseSAT := IIF(STFGetStation("USESAT") == Nil,.F.,STFGetStation("USESAT"))
181 Else
182     Self:lUseSAT := .F.
183 Endif
184
185 //Utilização do PDV atraves de dispositivos Móveis
186 Self:lMobile := STFIsMobile()
187
188 // Usa NFC-e com TSS
189 Self:lNFCETSS := !Self:lMobile
190
191 // Define o tipo de integração usada DEFAULT = "Protheus"
192 Self:cIntegration := STFCfgIntegration()
193
194 //Tipos de Transacao: "VENDA|TROCADEVOLUCAO|DEMONSTRACAO|"
195 Self:cTypeOperation := STFTypeOperation() //Seta o mode de operacao
196
197 //habilita troca de pagamento para orcamentos
198 Self:lChangePay := STFCChangePay()
199
200 //Verifica se usa NFCE
201 Self:lUseNFCE := Iif(ExistFunc("LjEmitNFCE"), LjEmitNFCE(), .F.) // Sinaliza se utiliza NFC-e
202
203 Self:lPAFNFCe := ExistFunc("LjTipoPAF") .AND. LjTipoPAF(STFGetStation("CODIGO")) $ TIPO_PAF_NFCE
204
205 Return Self
206
207
208 //-----

```

```

209 /*{Protheus.doc} STFIsFrs
210 Verifica se está integrando com o FIRST
211 @param
212 @return STFIsMobile()
213 @obs
214 @sample
215 */
216 //-----
217 Function STFIsFrs() //Se integra com first
218 Local cIntegration := Upper(SuperGetMv("MV_LJRETIN", .F., "DEFAULT")) //Parâmetro que indica a integração com retaguarda
219 Local lRet := .F. //Retorno da rotina
220
221 lRet := cIntegration == "FIRST"
222
223 Return lRet
224
225
226 //-----
227 /*{Protheus.doc} STFCfgIntegration
228 Verifica se está integrando com o FIRST ou com a RMS
229 @param
230 @return cIntegration
231 @obs
232 @sample
233 */
234 //-----
235 Function STFCfgIntegration()
236
237 Local cIntegration := "DEFAULT" // Nome da integração "DEFAULT" = Protheus
238
239 Do Case
240
241     Case STFIsFrs()
242         cIntegration := "FIRST"
243
244     Case SuperGetMv("MV_LJRMS", .F.)
245         cIntegration := "RMS"
246
247     Otherwise
248         cIntegration := "DEFAULT" // Protheus Padrão
249
250 EndCase
251
252 Return cIntegration
253
254
255 //-----
256 /*{Protheus.doc} STFIsMobile
257 Verifica a informação do Mobile
258 @param
259 @return STFGetMobile()
260 @obs

```

```

261 @sample
262 */
263 //-----
264 Function STFIsMobile()
265
266 Local aInfoRmt := {}           // Informações do remote
267 Local lMobile := .F.          // Verificação do mobile
268
269 If STFGetMobile() = Nil .AND. GetSrvProfString("FLY01","2") $ "01"
270
271     If FindFunction("GetRmtInfo")
272         aInfoRmt := GetRmtInfo()
273     EndIf
274
275     If ValType(aInfoRmt) == "A" .AND. Len(aInfoRmt) >= 2
276         lMobile := Upper(aInfoRmt[2]) == "ANDROID" .OR. (Upper(aInfoRmt[2]) == "WINDOWS RUNTIME") //Windows Mobile
277     EndIf
278
279     //Temporario.
280     //Usado apenas para testes no Windows onde nao se tem aparelho mobile disponivel
281     lMobile := lMobile .OR. GetSrvProfString("PDVMOBILE","0") == "1"
282
283     STFSetMobile(lMobile)
284
285 Else
286     If GetSrvProfString("FLY01","2") $ "01"
287         lMobile := STFGetMobile()
288     Else
289         lMobile := .F.
290     EndIf
291 EndIf
292
293 Return lMobile
294
295
296 //-----
297 /*/{Protheus.doc} STFGetMobile
298 Retorna a flag se for Mobile
299 @param
300 @return lMobilePDV
301 @obs
302 @sample
303 */
304 //-----
305 Function STFGetMobile()
306
307 Return lMobilePDV
308
309
310 //-----
311 /*/{Protheus.doc} STFSetMobile
312 Atribui a flag se for Mobile

```

```

313 @param lValue
314 @return lMobilePDV
315 @obs
316 @sample
317 */
318 //-----
319 Function STFSetMobile(lValue)
320
321 Default lValue := .F.
322
323 lMobilePDV := lValue
324
325 Return lMobilePDV
326
327
328 //-----
329 /*{Protheus.doc} STFIniMobile
330 Faz alteracoes nos ini do PDV mobile
331
332 @param
333 @return Nil
334 @obs
335 @sample
336 */
337 //-----
338 Main Function STFIniMobile()
339
340 Local oButton1      := Nil      //Botao
341 Local oButton2      := Nil      //Botao
342 Local oButton3      := Nil      //Botao
343 Local oSay1         := Nil      //Say
344 Local oSay2         := Nil      //Say
345 Local oSay3         := Nil      //Say
346 Local oSay4         := Nil      //Say
347 Local oSay5         := Nil      //Say
348 Local oSay6         := Nil      //Say
349 Local oDlg          := Nil      //Dialog Principal
350 Local lHideParams   := AllTrim(GetPvProfString("config","HideParamsForm","", "SMARTCLIENT.INI" )) == "1" //Esconde parametros
351 Local lLaunchAsServer := AllTrim(GetPvProfString("Local","LaunchAsServer","", "SMARTCLIENT.INI" )) == "1" //Executa server
352 Local lLogMessage   := AllTrim(GetPvProfString("General","LogMessages","",GetAdv97()) ) == "1" //habilita log de mensagem
353 Local lLicenseServer := AllTrim(GetPvProfString("LicenseServer","enable","",GetAdv97()) ) == "1" //Licence
354 Local serverLcClient := AllTrim(GetPvProfString("LicenseClient","server","",GetAdv97()) ) //Licence
355 Local portLcClient  := AllTrim(GetPvProfString("LicenseClient","port","",GetAdv97()) ) //Licence
356
357 Local bBlock1 := {| WritePProString("config" , "HideParamsForm" , IIF(lHideParams , "0", "1"), "SMARTCLIENT.INI" ), lHideParams:=!lHideParams } //Bloco de Codigo
358 Local bBlock2 := {| WritePProString("Local" , "LaunchAsServer" , IIF(lLaunchAsServer , "0", "1"), "SMARTCLIENT.INI" ), lLaunchAsServer:=!lLaunchAsServer } //Bloco de
Codigo
359 Local bBlock3 := {| WritePProString("General" , "LogMessage" , IIF(lLogMessage , "0", "1"), GetAdv97() ), lLogMessage:=!lLogMessage } //Bloco de Codigo
360 Local bBlock4 := {| WritePProString("LicenseServer" , "enable" , IIF(lLicenseServer , "0", "1"), GetAdv97() ), lLicenseServer:=!lLicenseServer } //Bloco de Codigo
361
362
363 DEFINE MSDIALOG oDlg TITLE "TOTVS FATCLIENT" FROM 000, 000 TO 500, 300 COLORS 0, 16777215 PIXEL //"TOTVS FATCLIENT"

```



```

364
365 @ 020, 009 BUTTON oButton1 PROMPT "On/Off" SIZE 062, 022 OF oDlg ACTION Eval(bBlock1) PIXEL //"On/Off"
366 @ 075, 009 BUTTON oButton2 PROMPT "On/Off" SIZE 062, 022 OF oDlg ACTION Eval(bBlock2) PIXEL //"On/Off"
367 @ 120, 009 BUTTON oButton3 PROMPT "On/Off" SIZE 062, 022 OF oDlg ACTION Eval(bBlock3) PIXEL //"On/Off"
368 @ 165, 009 BUTTON oButton4 PROMPT "On/Off" SIZE 062, 022 OF oDlg ACTION Eval(bBlock4) PIXEL //"On/Off"
369
370 @ 020, 081 SAY oSay1 PROMPT IIF(lHideParams,"On","Off") SIZE 062, 022 OF oDlg COLORS 0, 16777215 PIXEL //"On/Off"
371 @ 075, 080 SAY oSay2 PROMPT IIF(lLaunchAsServer,"On","Off") SIZE 062, 022 OF oDlg COLORS 0, 16777215 PIXEL //"On/Off"
372 @ 130, 080 SAY oSay3 PROMPT IIF(lLogMessage,"On","Off") SIZE 062, 022 OF oDlg COLORS 0, 16777215 PIXEL //"On/Off"
373 @ 185, 080 SAY oSay3 PROMPT IIF(lLicenseServer,"On","Off") SIZE 062, 022 OF oDlg COLORS 0, 16777215 PIXEL //"On/Off"
374
375 @ 005, 008 SAY oSay4 PROMPT "HideParamsForm" SIZE 136, 014 OF oDlg COLORS 0, 16777215 PIXEL
376 @ 060, 008 SAY oSay5 PROMPT "LaunchAsServer" SIZE 136, 014 OF oDlg COLORS 0, 16777215 PIXEL
377 @ 115, 008 SAY oSay6 PROMPT "LogMessage" SIZE 137, 014 OF oDlg COLORS 0, 16777215 PIXEL
378 @ 170, 008 SAY oSay6 PROMPT "LicenseServer" SIZE 137, 014 OF oDlg COLORS 0, 16777215 PIXEL
379
380
381 oButton1:SetCSS( POSCSS (GetClassName(oButton1), CSS_BTN_FOCAL ))
382 oButton2:SetCSS( POSCSS (GetClassName(oButton2), CSS_BTN_FOCAL ))
383 oButton3:SetCSS( POSCSS (GetClassName(oButton3), CSS_BTN_FOCAL ))
384 oButton4:SetCSS( POSCSS (GetClassName(oButton3), CSS_BTN_FOCAL ))
385
386 ACTIVATE MSDIALOG oDlg CENTERED
387
388 Return Nil
389
390
391 //-----
392 /*{Protheus.doc} ISFly01
393 Retorna se está com a versão Fly ativa
394
395 @param
396 @return Retorna se está com a versão Fly ativa
397 @obs
398 @sample
399 /*/
400 //-----
401 Function ISFly01()
402
403 Local lIsFly01 := .F. //Controla se é versão Fly01
404
405 lIsFly01 := GetSrvProfString("FLY01","0") == "1"
406
407 Return lIsFly01
408
409
410 //-----
411 /*{Protheus.doc} STFTYPE0operation
412 Retorna o tipo de operacao em uso, venda, troca demonstracao etc
413
414 @param
415 @return Retorna a operacao em uso no momento

```

```

416 @obs
417 @sample
418 /*/
419 //-----
420 Function STFTYPE0operation()
421
422 Local cType0operation := "VENDA" //Tipos de operação "VENDA|TROCADEVOLUCAO|DEMONSTRACAO|"
423
424
425 //Valida versao demonstrativa
426 If UPPER(AllTrim(STFGetStation("LG_TIPELA")) ) == "D" // D = versao demonstrativa
427     cType0operation := "DEMONSTRACAO"
428
429 Else
430     cType0operation := "VENDA"
431 EndIf
432
433 Return cType0operation
434
435 //-----
436 /*/{Protheus.doc} STFChangePay
437 Valida se a troca de forma de pagamentos está ativa
438 @type function
439 @param
440 @return      lRet - Retorna se a troca de pagamentos está habilitada
441 /*/
442 //-----
443 Static Function STFChangePay()
444 Local lRet      := .F.
445 Local lHabTFP   := Iif(SuperGetMV("MV_LJHBTFP",,0) == 0, .F.,.T.) //Habilita ou não a troca de forma de pagamento
446 Local cCash     := xNumCaixa()
447
448 lRet := AliasIndic("MHI")
449
450 lRet := lRet .And. lHabTFP
451
452 If lRet
453
454     DbSelectArea("SLF")
455     SLF->( DbSetOrder(1) )
456     If SLF->( DbSeek(xFilial("SLF")+cCash) )
457
458         cAllAccess := SLF->LF_ACESSO
459         cAccess := SubStr( cAllAccess , 41 , 1)
460
461         // status: N - Acesso Negado, S - Acesso Liberado, X - Liberação via Superior
462         lRet := If(cAccess <> "S", .T., .F.)
463
464     EndIf
465
466 EndIf
467

```

```
468 lRet := lRet .And. ExistFunc("STIChangePay") //Função que realiza a troca da forma de pagamento
469
470 Return lRet
471
472 //-----
473 /*{Protheus.doc} STFSetOnPdv
474 Seta se é PDV online
475 @param lValue - Determina se é PDV Online
476 @type function
477 @param
478 @return lPDVOnLine - Retorna se é PDV Online
479 */
480 //-----
481 Function STFSetOnPdv(lValue)
482 Default lValue := .F.
483
484 lPDVOnLine := lValue
485
486 Return lPDVOnLine
487
488 //-----
489 /*{Protheus.doc} STFGetOnPdv
490 Retorna se é PDV online
491 @type function
492 @param
493 @return lPDVOnLine - Retorna se é PDV Online
494 */
495 //-----
496 Function STFGetOnPdv()
497 Return lPDVOnLine
```

Framework/Start/STFIsPOS.PRW

97 linhas após remoção de metadados de autoria/data

```

1 #Include "Protheus.ch"
2
3 Static lIsPOS := Nil          // Verifica se é TOTVS PDV
4
5
6 //-----
7 /*{Protheus.doc} STFIsPOS
8 Verifica se é TOTVS PDV
9 @type function
10 @param      STFIsPOS , identifica se o módulo é FrontLoja 23
11 @return      lIsPOS - Retorna se é TOTVS PDV
12 */
13 //-----
14 Function STFIsPOS(lFront)
15 Local nMod := 0
16
17 Default lFront := .F.
18
19 If lIsPOS = Nil
20
21     If lFront
22         nMod := 23
23     Else
24         nMod := 12
25     EndIf
26
27     If Type("nModulo") == "N"
28         nMod := nModulo
29     EndIf
30
31     If nMod == 23
32
33         lIsPOS := ExistFunc("FWPosLight") .AND. FWPosLight()
34
35         If !lIsPOS
36             lIsPOS := STFPdvOn()
37         EndIf
38     Else
39         lIsPOS := .F.
40     EndIf
41
42 EndIf
43
44 Return lIsPOS
45
46
47 //-----
48 /*{Protheus.doc} STFSetPOS
49 Atribui a flag se for TOTVS PDV.
50 Esse método pode ser usado para forçar a marca que é TOTVS PDV
51 @type function
52 @param      lValue

```

```

53 @return lIsPOS - Retorna se é PDV
54 @obs
55 @sample
56 /*/
57 //-----
58 Function STFSetPOS(lValue)
59
60 Default lValue := .F.
61
62 lIsPOS := lValue
63
64 Return lIsPOS
65
66
67 //-----
68 /*/{Protheus.doc} STFPdvOn
69 Retorna se esta trabalhando com o PDV Online
70
71 @type      function
72 @return    lRet - Retorna se o PDV esta trabalhando Online
73 @obs
74 @sample
75 /*/
76 //-----
77 Function STFPdvOn(cEstacao)
78
79 Local lRet := .F.
80 Local aArea := GetArea()
81
82 Default cEstacao := ""
83
84 If SLG->(ColumnPos("LG_PDVON")) > 0
85     If Empty(cEstacao)
86         lRet := AllTrim(STFGetStat("PDVON")) == "1"
87     Else
88         SLG->(dbSetOrder(1)) //LG_FILIAL+LG_CODIGO
89         If SLG->(dbSeek(xFilial("SLG") + PadR(cEstacao,TamSX3("LG_CODIGO")[1])))
90             lRet := AllTrim(SLG->LG_PDVON) == "1"
91         EndIf
92     EndIf
93 EndIf
94
95 RestArea(aArea)
96
97 Return lRet

```

Framework/Start/STFStart.prw

678 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STFSTART.CH"
4
5 Static oSTConfig      := STFCnfAmb():New()           // Objeto de Configuracoes gerais
6 Static lShowMsg       := .T.
7
8 //-----
9 /*/{Protheus.doc} STFStart
10 Inicialização do Sistema
11
12 @param
13 @return lRet          Retorna se inicializou com sucesso
14 @obs
15 @sample
16 /*/
17 //-----
18 Function STFStart()
19 Local lRet             := .T.                        // Retorna se inicializou com sucesso
20 Local aRet             := {}
21 Local aDados           := {"23", space(SLG->(TamSx3("LG_CR0")[1])) }
22 Local cCR0             := ""
23 Local cC00             := ""
24 Local lEmptySale       := .F.                        // Caso haja um cupom aberto, porem nao haja nada gravado, evita a chamada do cancelamento de uma
   venda que nao existe.
25 Local lFiscal          := .F.                        // Usa impressora fiscal
26 Local lUsesNotFiscal:= .F.                        // Usa impresora nao fiscal
27 Local lFinServ         := SuperGetMV("MV_LJCSF",,.F.) // Define se habilita o controle de servicos financeiros
28 Local lJobConPrd       := (SuperGetMV("MV_LJAJCP",,"2") == "1") // Atualizacao da variavel global da consulta de produto será realizada por JOB 1=Sim e 2=Nao
29 Local lUpData          := ExistFunc("STWUpData")      // Verifica se existe a rotina de subida de dados
30 Local lDowndata        := ExistFunc("STWDownData")    // Verifica se existe a rotina de descida de dados
31 Local lFirst           := .F.                        // Integração com o First
32 Local lMobile          := ExistFunc("STFIsMobile") .AND. STFIsMobile() // Pdv Mobile?
33 Local lNFCETSS         := .T.                        // Envio de NFce por TSS
34 Local cIntegration     := ""                        // Tipo da Integração
35 Local lPOS             := STFIsPOS()                 // TO D0: Colocar a regra de como identificar de está sendo executado Front ou POS
36 Local lLjNfPafEcf     := .F.
37 Local uRet             := nil
38 Local lUsaSAT          := lJUseSat()                // Verifica se usa SAT
39 Local lAutomato        := If(Type("lAutomatoX")<>"L",.F.,lAutomatoX)
40 Local cNfceURL         := PadR(GetNewPar("MV_NFCEURL",""),250)
41 Local lIsMDI           := Iif(ExistFunc("LjIsMDI"),lJIsMDI(),oApp:lMDI) //Verifica se acessou via SIGAMDI
42 Local cMvRelt          := SuperGetMv("MV_RELT",.F.,"") // Validar diretório para impressao danfe A4
43 Local lFunTTd707       := ExistFunc("LjVldTTD707")
44 Local cMsg             := ""
45 /*
46      !!!!!!!!!!!   ATENCAO      !!!!!!!!!!!
47
48      NAO RETIRAR O METODO oSTConfig:LoadConfig() DO INICIO DA FUNCAO.
49      ESTE METODO TER SEMPRE QUE SER CARREGADO NO INICIO
50
51      !!!!!!!!!!!   ATENCAO      !!!!!!!!!!!

```

```

52 */
53
54 // -- Inicio Job de checagem de comunicação com a retaguarda
55 StartJob("STFCheckCo",GetEnvServer(),.F.,cEmpAnt,cFilAnt)
56
57 //Configura estação como mobile antes do oSTConfig:LoadConfig(), pois o LoadConfig utiliza essa configuração
58 If lMobile //Mobile sempre será NFC-e
59     STFSetStat({{ "NFCe", .t.}})
60     AjstSX6Mbl() //Cria o parâmetro da integração First
61 EndIf
62
63 //Desabilita a subida de dados
64 lUpData := .F.
65
66 //Carrega as configuracoes globais logo no inicio
67 oSTConfig:LoadConfig() // Carrega configuracoes gerais
68
69
70 /*Caso esteja habilitado o PAF-NFC-e, precisa estar cadastrado o IE do cliente na rotina LjVldTTD707(),
71 caso contrário não sobe a aplicação*/
72 If STFGetCfg("LPAFNFCe") .AND. lFunTTd707 .AND. !LjVldTTD707()
73     Return .F.
74 EndIf
75
76
77 /*-----
78     Avisa ao usuário sobre a desativação da NFC-e 3.10
79 -----*/
80 If LjEmitNFCe() .AND. !lUsaSAT
81     If ExistFunc("LjNfceMsg")
82         cMsg := LjNfceMsg()
83
84         If !Empty(cMsg)
85             STPosMSG( "NT 2016.002" , cMsg, .T., .F., .F.)
86         EndIf
87     EndIf
88 EndIf
89
90 /*-----
91     Avisa ao usuário sobre o vencimento do certificado digital
92 -----*/
93 If !Empty(cNfceURL) .And. !lAutomato .And. lShowMsg .And. LjEmitNFCe() .And. !lUsaSAT
94     If ExistFunc("LjTSSIDENT")
95         cIdEnt := LjTSSIDENT("65")
96         oLOJGNFCE := LOJGNFCE():New()
97         oLOJGNFCE:LjChkCert(cIdEnt)
98         lShowMsg := .F.
99     EndIf
100 EndIf
101
102 //Faz validacoes importantes para o PDV
103 If ExistFunc("STFVldCfg")

```

```

104     uRet := STFVldCfg()
105     If (ValType(uRet) = "L") .AND. !uRet    //Há versão anterior de STFVldCfg() que retorna nil
106         Return .F.
107     EndIf
108 EndIf
109
110 // verifica a propriedade do objeto verdadeiro se MDI
111 If !IsMDI
112     MsgStop(STR0001)    //"0 acesso a rotina atendimento na interface SIGAMDI não é permitido."
113     Return .F.
114 EndIf
115
116 // Protege rotina para que seja usada apenas no SIGALOJA / Front Loja
117 If !AmIIn(12,23)
118     Return .F.
119 EndIf
120
121 If !lPOS
122     MsgStop(STR0039) //"Para utilização do TotvsPdv, adicione/altere o conteúdo da chave PosLight = 1 no AppServer.ini deste ambiente"
123     Return .F.
124 EndIf
125
126 /*-----
127     Evitar errorlog durante a impressao por FWMSPRINTER
128     devido não existir o diretorio de spool
129 -----*/
130 if Empty(cMvRelt)
131     MsgStop("Para utilização do TotvsPdv, deve-se preencher o parametro MV_RELT!")
132     Return .F.
133
134 elseif !ExistDir(cMvRelt)
135     if Makedir(cMvRelt) <> 0
136         MsgStop("Para utilização do TotvsPdv, verifique o parametro MV_RELT e se o diretorio existe no protheus_data!")
137         Return .F.
138     endif
139
140 endif
141
142 lNFCETSS := STFGetCfg("lNFCETSS", .T.)
143 lNFCETSS := IIF(ValType(lNFCETSS)="U", .T., lNFCETSS)
144
145 If !lNFCETSS .AND. SL1->(FieldPos("L1_PRONFCE")) == 0
146     STFMessage("STFStart", "STOP", STR0008) // "Para transmissão de NFce direta pelo PDV, faz-se necessária a criação do campo L1_PRONFCE"
147     STFShowMessage("STFStart")
148     Return .F.
149
150 EndIf
151
152 If STFGetStation("NFCE") <> NIL .AND. STFGetStation("NFCE")
153     AjustaSX5()
154 EndIf
155

```



```

156 cIntegration := STFGetCfg("cIntegration", "DEFAULT")
157
158 If Valtype(cIntegration) <> "C"
159     cIntegration := "DEFAULT"
160 EndIf
161 lFirst := cIntegration == "FIRST"
162
163 //retirada da chamada das funções STFVldCadEmp() e STFCadEmp()
164
165 //Integração First
166 If lFirst .AND. !STFTypeOperation() == "DEMONSTRACAO"
167     AjustaSX6()
168     //Execução da rotina de carga de dados
169
170     If lUpData
171         STFMessage("UpData", "FWMSGRUN", STR0010, {|| STWUpData( , , STFGetStat("CODIGO")) }) // "Executando subida de dados"
172         STFShowMessage("UpData")
173     EndIf
174 EndIf
175
176 // Validacoes de usuario e estacao
177 If !STFVldUser(lMobile .AND. lFirst )
178     Return .F.
179 EndIf
180
181 llJnFpafEcf := STFGetCfg("lPafEcf")
182
183 If llJnFpafEcf .AND. ! LjEmitNFCe()
184     If !LjGtIsPaf()
185         MsgStop( STR0007 + " [SIGAPAF.exe]") //"Nos estados aonde é obrigatório o uso do PAF-ECF, o acesso deve ser realizado pelo executável homologado"
186         Return .F.
187     EndIf
188 EndIf
189
190 //-----
191 // Permite o uso do sistema sem impressora fiscal
192 lUsesNotFiscal := STFGetCfg("lUsesNotFiscal")
193
194 //Usa impressora fiscal
195 lFiscal := STFGetCfg("lUseECF")
196
197 If !lUsesNotFiscal
198
199     //Abertura de Dispositivos
200     lRet := STWOpenDevi()
201
202     If STFGetCfg("lUseECF")
203
204         If lRet
205             STBNSrAval() //Verifica o numero de série
206             aRet := STFFireEvent(ProcName(0),"STPrinterStatus", aDados)
207

```

```

208     If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 1 .AND. !Empty(aDados[2])
209         //Verifica o numero de série
210         cCRO := aDados[2]
211     EndIf
212
213     aDados := {space(20), nil}
214
215     aRet := STFFireEvent( ProcName(0) ,"StGetReceipt", aDados )
216
217     If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0 .AND. !Empty(aDados[1])
218         cC00 := PadL(AllTrim(aDados[1]), TamSX3("L1_DOC")[1])
219     EndIf
220
221     If !Empty(cCRO) .AND. !Empty(cC00)
222         STBSerieAlt( cC00, cCRO )
223     EndIf
224
225     //Grava no INI as configurações de Arredondamento do ECF
226     If ExistFunc("STBTaxDec") .AND. !lAutomato
227         STBTaxDec("1") // 1 = Grava no ini , 2 = Recupera informacao do ini
228     EndIf
229
230 Else
231
232     //Se for PAF ECF e nao conseguiu abrir os perifericos
233     //Abre a tela inicial so com o menu fiscal
234     If llJNfPafEcf
235         //Seta o a configuracao para somente Menu fiscal
236         STFSetCfg( "lOnlyMenuFiscal" , .T. )
237         //Retorna .T. mesmo nao abrindo os perifericos
238         lRet := .T.
239     EndIf
240 EndIf
241
242 //Chama rotina para armazenar MD-5 do arquivo paflista.txt no arquivo criptografado, para utilizar na impressao do Cupom
243 //Requisito IX: "...gravar o resultado no arquivo auxiliar criptografado"
244 If llJNfPafEcf .AND. LjGtIsPaf() .AND. !LjEmitNFCe()
245     STBGrvMd5Ls(.T.)
246     STWValGT(.T.)
247
248     //Homologacao 2017 - ER-PAF-ECF 02.05
249     If ExistFunc("STBIniPAF")
250         lRet := STBIniPAF( .T. , .F.)
251     EndIf
252 EndIf
253
254 Else
255     /* Nao utiliza impressora nao fiscal*/
256
257     STBNSrAval() //Verifica o numero de série
258     //Carrega as configuracoes globais logo no inicio
259     oSTConfig:LoadConfig() // Carrega configuracoes gerais

```

[illegible]

```

312         Else
313             MsgRun ( STR0005, STR0006, { || STDStartProd() } ) // "Inicializando! Esse processo pode levar alguns minutos..." ### "Aguarde"
314         EndIf
315     EndIf
316 EndIf
317
318 Return lRet
319
320 //-----
321 /*{Protheus.doc} STFGetCfg
322 Pega as configuracoes do parametro solicitado
323 @param
324 @return Self
325 @obs
326 @sample
327 */
328 //-----
329 Function STFGetCfg( cParam , xDefault )
330
331 Local uRet          := Nil // Parametro a ser retornado
332
333 Default cParam      := ""
334 Default xDefault    := Nil
335
336 If !Empty(cParam) .AND. Type("oSTConfig:" + cParam) <> "U"
337     uRet := &( "oSTConfig:" + cParam )
338 EndIf
339
340 // tratamento para usar o valor default
341 If ValType(uRet) == "U" .AND. ValType(xDefault) <> "U"
342     uRet := xDefault
343 EndIf
344
345 Return uRet
346
347
348 //-----
349 /*{Protheus.doc} STFSetCfg
350 Seta as configuracoes do parametro solicitado
351 @param
352 @return Self
353 @obs    Cuidado ao alterar esse conteudo pois o mesmo é refletido para todo sistema
354 @sample
355 */
356 //-----
357 Function STFSetCfg( cParam , uValue )
358
359 Local uRet          := Nil // Parametro a ser retornado
360
361 Default cParam := ""
362 Default uValue := Nil
363

```

```

364 If !Empty(cParam) .AND. Type("oSTConfig:" + cParam) <> "U" .AND. ValType(uValue) <> "U"
365
366     //Macroexecuta
367     &("oSTConfig:" + cParam) := uValue
368
369 EndIf
370
371 Return uRet
372
373
374 //-----
375 /*{Protheus.doc} STFVldUser
376 Verifica diversas atributos para liberar o acesso a venda
377 para o caixa que esta tentando entrar na rotina de venda
378 @param lMobFrst - Usuário Mobile e First (default .f.)
379 @return Retorna se permiti abrir o sistema
380 @obs
381 @sample
382 */
383 //-----
384 Function STFVldUser(lMobFrst)
385
386 Local lRet           := .T.           //Retorno
387 Local aAreaSLF       := {}           //WorkArea SLF
388 Local cStrAcesso     := ""           //Acessos do Totvs PDV, exceto gaveta
389 Local aRet           := {}
390
391 Default lMobFrst := .F.
392
393 // Se for administrador nao deixa fazer a venda
394 If __cUserID == "000000"
395
396     MsgInfo( STR0002          +; // 0 administrador nao podera efetuar vendas. Utilize a opcao
397              STR0003          +; // Senhas/Caixas no Menu Miscelanea para incluir um Caixa. Caso
398              STR0004          ) // ja exista um cadastrado, reentre no sistema com a senha de Caixa
399     lRet := .F.
400 EndIf
401
402 If lRet
403     aRet := STFProFile( 3 ) //Valida se usuario eh caixa (SLF-3 - Usuario Caixa)
404
405     If !aRet[1]
406         MsgInfo( STR0035+SA6->A6_COD+STR0036 +Chr(13)+; // "Usuário logado [Caixa:" ## "]" não possui permissão de Caixa."
407                 STR0037+Chr(13)+; // "Verifique na rotina de Caixas (LOJA120) a permissão para o acesso: Usuário Caixa."
408                 STR0038) // "(Totvs PDV não será iniciado)"
409         lRet := .F.
410     EndIf
411 EndIf
412
413 // A ocorrencia 25 (ACS), verifica se o usuario
414 // podera ou nao efetuar um Atendimento
415 If lRet

```

```

416     If !ChkPsw(25)
417         lRet := .F.
418     EndIf
419 EndIf
420
421
422 // Validacoes de estacao e caixa
423 If lRet
424
425     xNumCaixa()
426     DbSelectArea("SLF")
427     DbSetOrder(1)
428     If lMobFrst .AND. DbSeek(xFilial("SLF")+SA6->A6_COD)
429         aAreaSLF := GetArea()
430         cStrAcesso := LF_ACESSO
431
432
433         If Empty(cStrAcesso) .OR. LF_DESCPER = 0 .OR. LF_DESCVAL = 0 .OR. LF_TOTDESP = 0 .OR. LF_TOTDESP = 0 .OR. LF_TOTDESV = 0
434
435             RecLock("SLF", .F.)
436             Replace LF_ACESSO With replicate("S", SLF->(TamSX3("LF_ACESSO"))[1]))
437
438             Replace LF_DESCPER With Val(replicate("9", SLF->(TamSX3("LF_DESCPER"))[1]-1 ))/(10**SLF->(TamSX3("LF_DESCPER"))[2])
439
440             Replace LF_TOTDESP With Val(replicate("9", SLF->(TamSX3("LF_TOTDESP"))[1]-1 ))/(10**SLF->(TamSX3("LF_TOTDESP"))[2])
441
442             Replace LF_DESCVAL With Val(replicate("9", SLF->(TamSX3("LF_DESCVAL"))[1]-1 ))/(10**SLF->(TamSX3("LF_DESCVAL"))[2])
443
444             Replace LF_TOTDESV With Val(replicate("9", SLF->(TamSX3("LF_DESCVAL"))[1]-1 ))/(10**SLF->(TamSX3("LF_DESCVAL"))[2])
445
446             SLF->(MsUnLock())
447
448             //Realimenta configurações
449             oSTConfig:LoadConfig()
450
451         EndIf
452
453
454     ElseIf lMobFrst
455         Help(" ",1,"NOESTACAO")
456         lRet := .F.
457     EndIf
458
459     If Empty(cEstacao)
460         cEstacao := "001"
461     EndIf
462
463     DbSelectArea("SLG")
464     If !DbSeek(xFilial("SLG")+cEstacao)
465         Help(" ",1,"NOESTACAO")
466         lRet := .F.
467     EndIf

```

```

468
469 EndIf
470
471 Return lRet
472
473
474 //-----
475 /*/{Protheus.doc} AjstSX6Mbl
476 Criação de parâmetros se PDV for Mobile
477 @param
478 @return
479 @obs
480 @sample
481 */
482 //-----
483
484 Static Function AjstSX6Mbl()
485 Local aArea := GetArea() //WorkArea
486 Local aSX6 := {} //Parametros
487 Local nI := 0 //Contadora
488 Local nX := 0 //Contadora
489
490
491 aEstru := { "X6_FIL" , "X6_VAR" , "X6_TIPO" , "X6_DESCRIC" , "X6_DSCSPA" , "X6_DSCENG" , "X6_DESC1" , "X6_DSCSPA1",;
492            "X6_DSCENG1" , "X6_DESC2" , "X6_DSCSPA2" , "X6_DSCENG2" , "X6_CONTEUD" , "X6_CONTSPA" , "X6_CONTENG" , "X6_PROPRI" , "X6_PYME" }
493
494
495 ///
496 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJRETIN","C" , STR0011, "X6_DSCSPA1 ,X6_DSCENG1 ,X6_DESC2 ,X6_DSCSPA2 ,X6_DSCENG2 , X6_CONTEUD ,X6_CONTSPA ,X6_CONTENG ,X6_PROPRI ,X6_PYME" , "X6_DSCSPA" , "X6_DSCENG" , "X6_DESC1" , "X6_DSCSPA1",;
497 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJEMPPK","C" , STR0012, "X6_DSCSPA1 ,X6_DSCENG1 ,X6_DESC2 ,X6_DSCSPA2 ,X6_DSCENG2 , X6_CONTEUD ,X6_CONTSPA ,X6_CONTENG ,X6_PROPRI ,X6_PYME" , "X6_DSCSPA" , "X6_DSCENG" , "X6_DESC1" , "X6_DSCSPA1",;
498 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJEMPCCK","C" , STR0013, "X6_DSCSPA1 ,X6_DSCENG1 ,X6_DESC2 ,X6_DSCSPA2 ,X6_DSCENG2 , X6_CONTEUD ,X6_CONTSPA ,X6_CONTENG ,X6_PROPRI ,X6_PYME" , "X6_DSCSPA" , "X6_DSCENG" , "X6_DESC1" , "X6_DSCSPA1",;
499 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_NFCEIDT","C" , STR0018, "X6_DSCSPA1 ,X6_DSCENG1 ,X6_DESC2 ,X6_DSCSPA2 ,X6_DSCENG2 , X6_CONTEUD ,X6_CONTSPA ,X6_CONTENG ,X6_PROPRI ,X6_PYME" , "X6_DSCSPA" , "X6_DSCENG" , "X6_DESC1" , "X6_DSCSPA1",;
500 //PARA TESTES ESTE AMBIENTE É 3 DESENVOLVIMENTO, MAS DEVE SER CONFIGURADO COMO 1 - PRODUCAO
501 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_AMBNFCE","C" , STR0019, "X6_DSCSPA1 ,X6_DSCENG1 ,X6_DESC2 ,X6_DSCSPA2 ,X6_DSCENG2 , X6_CONTEUD ,X6_CONTSPA ,X6_CONTENG ,X6_PROPRI ,X6_PYME" , "X6_DSCSPA" , "X6_DSCENG" , "X6_DESC1" , "X6_DSCSPA1",;
502 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJNFCE","N" , STR0021, "X6_DSCSPA1 ,X6_DSCENG1 ,X6_DESC2 ,X6_DSCSPA2 ,X6_DSCENG2 , X6_CONTEUD ,X6_CONTSPA ,X6_CONTENG ,X6_PROPRI ,X6_PYME" , "X6_DSCSPA" , "X6_DSCENG" , "X6_DESC1" , "X6_DSCSPA1",;
503 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJQTITE","N" , STR0022, "X6_DSCSPA1 ,X6_DSCENG1 ,X6_DESC2 ,X6_DSCSPA2 ,X6_DSCENG2 , X6_CONTEUD ,X6_CONTSPA ,X6_CONTENG ,X6_PROPRI ,X6_PYME" , "X6_DSCSPA" , "X6_DSCENG" , "X6_DESC1" , "X6_DSCSPA1",;
504
505 dbSelectArea("SX6")
506 SX6->(dbSetOrder(1))
507 For nI := 1 to Len(aSX6)
508     IncProc()
509     If !SX6->(dbSeek(aSX6[nI][1] + PadR(aSX6[nI][2],Len(SX6->X6_VAR))))
510         RecLock("SX6",.T.)

```

```

511 For nX := 1 to Len(aEstru)
512     If SX6->(FieldPos(aEstru[nX])) > 0
513         SX6->&(aEstru[nX]) := aSX6[nI][nX]
514     Endif
515 Next nX
516 dbCommit()
517 MsUnlock()
518 Endif
519
520 Next nI
521
522 RestArea(aArea)
523
524
525 Return
526
527
528 //-----
529 /*/{Protheus.doc} AjustaSX6
530 Criação de parâmetros se PDV for First
531 @param
532 @return
533 @obs
534 @sample
535 */
536 //-----
537 Static Function AjustaSX6()
538 Local aArea := GetArea() //WorkArea
539 Local aSX6 := {} //Parametros
540 Local nI := 0 //Contador
541 Local nX := 0 //Contador
542
543
544 aEstru := { "X6_FIL" , "X6_VAR" , "X6_TIPO" , "X6_DESCRIC" , "X6_DSCSPA" , "X6_DSCENG" , "X6_DESC1" , "X6_DSCSPA1" , ;
545             "X6_DSCENG1" , "X6_DESC2" , "X6_DSCSPA2" , "X6_DSCENG2" , "X6_CONTEUD" , "X6_CONTPA" , "X6_CONTENG" , "X6_PROPRI" , "X6_PYME" }
546
547
548 /// X6_FIL,X6_VAR ,X6_TIPO,X6_DESCRIC ,X6_DSCSPA ,X6_DSCENG ,X6_DESC1 ,X6_DSCSPA1 ,X6_DSCENG1 ,X6_DESC2 ,X6_DSCSPA2
549 ,X6_DSCENG2 ,X6_CONTEUD ,X6_CONTPA ,X6_CONTENG ,X6_PROPRI ,X6_PYME
549 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJFRSET","C" , STR0014, STR0014,STR0014,"" , "" , "" , "" , "" , "" , ""
550 , "" , "" , "" , "S" , "S"}) //Token da Integração First
550 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJFRSEP","C" , STR0015,STR0015,STR0015,"" , "" , "" , "" , "" , "" , ""
551 , "" , "" , "" , "S" , "S"}) //Codigo da Entidade PDV
551 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJFRSEF","C" , STR0016,STR0016,STR0016,"" , "" , "" , "" , "" , "" , ""
552 , "" , "" , "" , "S" , "S"}) //Codigo da Entidade First
552 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJFRFIL","N" , STR0017,STR0017,STR0017,"" , "" , "" , "" , "" , "" , ""
553 , "4" , "" , "" , "S" , "S"}) //Tamanho do campo filial no First
553 aAdd(aSX6,{ Space(FWSIZEFILIAL()),"MV_LJFILIN","C" , STR0023,STR0023,STR0023,"" , "" , "" , "" , "" , "" , ""
554 , "01" , "" , "" , "S" , "S"}) //Filial da Integração
555
556 dbSelectArea("SX6")

```



```

557 SX6->(dbSetOrder(1))
558 For nI := 1 to Len(aSX6)
559     IncProc()
560     If !SX6->(dbSeek(aSX6[nI][1] + PadR(aSX6[nI][2], Len(SX6->X6_VAR))))
561         RecLock("SX6", .T.)
562         For nX := 1 to Len(aEstru)
563             If SX6->(FieldPos(aEstru[nX])) > 0
564                 SX6->&(aEstru[nX]) := aSX6[nI][nX]
565             Endif
566         Next nX
567         dbCommit()
568         MsUnlock()
569     Endif
570
571 Next nI
572
573 RestArea(aArea)
574
575
576 Return
577
578 //-----
579 /*{Protheus.doc} AjustaSX5
580 Criação de tabelas para numeradores NFce
581 @param
582 @return
583 @obs
584 @sample
585 */
586 //-----
587 Static Function AjustaSX5()
588 Local aArea := GetArea() //WorkArea
589 Local nTamTab := Len( SX5->X5_TABELA) //Tabela
590 Local cSerie := SuperGetMv("MV_LJSERE5", .F., "SE5") //Serie da Sangria/Suprimento de ECNF
591 Local cSerieNF := STFGetStat("SERIE") //Serie da Estação
592 Local aSX5 := {} //Campos SX5
593 Local nI := 0 //Contador 1
594 Local nJ := 0 //Contador
595
596
597 aEstrut := { "X5_FILIAL", "X5_TABELA", "X5_CHAVE", "X5_DESCRI", "X5_DESCSPA", "X5_DESCENG" }
598
599 aAdd( aSX5, { ;
600     Space(FWSIZEFILIAL()) , ; //X5_FILIAL
601     PadR('01', nTamTab) , ; //X5_TABELA
602     cSerie , ; //X5_CHAVE
603     '000001' , ; //X5_DESCRI
604     '000001' , ; //X5_DESCSPA
605     '000001' } ) //X5_DESCENG
606
607 If !Empty(cSerieNF)
608     aAdd( aSX5, { ;

```

```

609      Space(FWSIZEFILIAL())
610      PadR('01',nTamTab)
611      cSerieNF
612      '000001'
613      '000001'
614      '000001'
615 EndIf
616
617 dbSelectArea( "SX5" )
618 SX5->( dbSetOrder( 1 ) )
619
620 For nI := 1 To Len( aSX5 )
621
622
623     If !SX5->( dbSeek( aSX5[nI][1] + aSX5[nI][2] + aSX5[nI][3]) )
624         RecLock( "SX5", .T. )
625         For nJ := 1 To Len( aSX5[nI] )
626             If FieldPos( aEstrut[nJ] ) > 0
627                 FieldPut( FieldPos( aEstrut[nJ] ), aSX5[nI][nJ] )
628             EndIf
629         Next nJ
630
631         MsUnLock()
632
633     EndIf
634
635
636 Next nI
637
638 RestArea(aArea)
639 Return
640
641 //-----
642 /*{Protheus.doc} STFCheckCo
643 Função responsável por realizar o teste de conexão com a retaguarda (o teste é realizado em background, não influencia na venda)
644
645 @param      cEmp, Caracter, Empresa para abertura de ambiente
646 @param      cFil, Caracter, Filial para abertura de ambiente
647 @return     Nil, Nulo
648 */
649 //-----
650
651 Function STFCheckCo(cEmp,cFil)
652 Local lConnHost      := .F.  // -- Variavel local que indica se existe comunicação com a retaguarda
653 Local lPosOpen       := .T.  // -- Indica se o TOTVS PDV esta aberto (STIPOSMAIN)
654
655 // -- Necessario abertura de ambiente devido ao uso da função FWHostPing()
656 RPCSetType(3)
657 PutGlbVars("lConnHost", .F.)
658 lOpenEnv := RpcSetEnv(cEmp,cFil,Nil,Nil,"FRT")
659
660 If lOpenEnv

```

```
661
662     PutGlbVars("lPosOpen", .T.) // -- Indica que o PDV esta aberto.
663
664     While lPosOpen // -- Enquanto o PDV continuar aberto continuo testando a comunicação, a variavel Global lPosOpen é alimentada com .F. no STFExit
665         lConnHost := FWHostPing()
666         PutGlbVars("lConnHost", lConnHost)
667         Sleep(10000)
668         GetGlbVars( "lPosOpen", @lPosOpen)
669     End
670
671     // -- Caso saia da tela do TOTVS PDV limpo as variaveis de memoria.
672     ClearGlbValue("lConnHost")
673     ClearGlbValue("lPosOpen")
674     RpcClearEnv()
675
676 EndIf
677
678 Return
```

Framework/Start/STFVldCfg.prw

122 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "POSCSS.CH"
3 #INCLUDE "STFVLDCFG.CH"
4
5 //-----
6 /*{Protheus.doc} STFVldCfg
7 Faz validações importantes para o correto funcionamento do PDV.
8 Podendo ser versões de fonte, Lib, Binário, Dicionário etc.
9 @type function
10 @param
11 @return Nil
12 */
13 //-----
14 Function STFVldCfg()
15
16 Local cMsg := ""      //Mensagem
17 Local lRet := .T.     //Retorno
18
19 LjGrvLog( "Valida_Config" , "ID_INICIO" )
20
21 //validacao compartilhamento tabelas
22 STFVldComp()
23
24 lRet := STFVldIdx()
25
26 LjGrvLog( "Valida_Config" , "ID_FIM" )
27
28 Return lRet
29
30
31 //-----
32 /*{Protheus.doc} STFVldComp
33 Valida compartilhamento da tabelas
34 @type function
35 @param
36 @return Nil
37 */
38 //-----
39 Static Function STFVldComp()
40
41 Local cMsg := ""      //Mensagem
42 Local cSL1 := ""      //Controle compartilhamento
43 Local cSL2 := ""      //Controle compartilhamento
44 Local cSL4 := ""      //Controle compartilhamento
45
46 //validacao compartilhamento l=Empresa
47 cSL1 := FwModeAccess("SL1",1)
48 cSL2 := FwModeAccess("SL2",1)
49 cSL4 := FwModeAccess("SL4",1)
50
51 If (cSL1 <> cSL2) .OR. (cSL1 <> cSL4) .OR. (cSL2 <> cSL4)
52     cMsg += STR0001 + STR0002 + ": SL1(" + cSL1 + ") SL2(" + cSL2 + ") SL4(" + cSL4 + ")." + CHR(10) //"Nível " "Empresa"

```

```

53 EndIf
54
55 //validacao compartilhamento 2=Unidade de Negócio
56 cSL1 := FwModeAccess("SL1",2)
57 cSL2 := FwModeAccess("SL2",2)
58 cSL4 := FwModeAccess("SL4",2)
59
60
61 If (cSL1 <> cSL2) .OR. (cSL1 <> cSL4) .OR. (cSL2 <> cSL4)
62     cMsg += STR0001 + STR0003 + ": SL1(" + cSL1 + ") SL2(" + cSL2 + ") SL4(" + cSL4 + ")." + CHR(10) //"Nível " "Unidade de Negócio"
63 EndIf
64
65 //validacao compartilhamento 3=Filial
66 cSL1 := FwModeAccess("SL1",3)
67 cSL2 := FwModeAccess("SL2",3)
68 cSL4 := FwModeAccess("SL4",3)
69
70 If (cSL1 <> cSL2) .OR. (cSL1 <> cSL4) .OR. (cSL2 <> cSL4)
71     cMsg += STR0001 + STR0004 + ": SL1(" + cSL1 + ") SL2(" + cSL2 + ") SL4(" + cSL4 + ")." + CHR(10) //"Nível " "Filial"
72 EndIf
73
74 If !Empty(cMsg)
75     STFMessage(ProcName(),"POPUP",STR0005 + CHR(10)+ cMsg) //"Atenção, o compartilhamento das tabelas devem ser iguais."
76     STFShowMessage(ProcName())
77     LjGrvLog( "Valida_Config" , STR0005 + CHR(10)+ cMsg)
78 EndIf
79
80 Return Nil
81
82
83 //-----
84 /*{Protheus.doc} STFVldIdx
85 Valida campo LI_SEQ e índice 1 se houver preenchido o campo LI_SEQ
86 @type function
87 @param
88 @return Nil
89 */
90 //-----
91 Function STFVldIdx()
92
93 Local cMsg := "" //Mensagem
94 Local lRet := .T. //Retorno
95 Local aArea := {} //Guarda a área atual
96
97 #IFDEF TOP
98     aArea := GetArea()
99     If SLI->(ColumnPos("LI_SEQ")) = 0 //Se o campo LI_SEQ não for criado
100         cMsg += STR0007 + CHR(10) //Favor criar o campo LI_SEQ, caracter de 9 posições."
101         cMsg += STR0008 + CHR(10) //Em seguida, "
102         cMsg += STR0009 + CHR(10) //Favor alterar, na tabela SIX, o conteúdo da ordem 1 do índice em SLI: LI_FILIAL+LI_ESTACA0+LI_TIP0+LI_SEQ"
103         cMsg += STR0010 + CHR(10) //Favor alterar, na tabela SX2, campo X2_UNICO da chave SLI para: LI_FILIAL+LI_ESTACA0+LI_TIP0+LI_SEQ"
104     Else //Pesquisar se tem LI_SEQ no conteúdo de SIX.

```

```
105     SIX->(DbSetOrder(1))      //INDICE+ORDEM
106     If SIX->(DbSeek("SLI1"))
107         If At( "LI_SEQ",SIX->CHAVE ) = 0      //Verifico se o LI_SEQ está presente na ordem 1 do índice em SLI
108             cMsg += STR0009 + CHR(10)      //"Favor alterar, na tabela SIX, o conteúdo da ordem 1 do índice em SLI: LI_FILIAL+LI_ESTACA0+LI_TIPO+LI_SEQ"
109             cMsg += STR0010 + CHR(10)      //"Favor alterar, na tabela SX2, campo X2_UNICO da chave SLI para: LI_FILIAL+LI_ESTACA0+LI_TIPO+LI_SEQ"
110         EndIf
111     EndIf
112 EndIf
113 If !Empty(cMsg)
114     STFMessage(ProcName(),"POPUP",STR0006 + CHR(10)+ cMsg) //"Atenção!"
115     STFShowMessage(ProcName())
116     LjGrvLog( "Valida_Config" , STR0006 + CHR(10)+ cMsg)
117     lRet := .F.
118 EndIf
119 RestArea(aArea)
120 #ENDIF
121
122 Return lRet
```

Framework/TotalUpdate/STBValues.prw

56 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*/{Protheus.doc} STBValues
6 Tratamento dos valores da venda
7
8 @param
9 @return Nil
10 @obs
11 @sample
12 */
13 //-----
14 Function STBValues( nDescTot )
15 Local oTotal      := STFGetTot()           // Totalizador
16 Local lDesTot     := ExistFunc("STBCDPGDes") .And. STBCDPGDes()
17 Local nL1_descto:= 0
18
19 Default nDescTot:= 0
20
21 /*/
22     Atualiza totalizador
23 /*/
24 STFSetTot( "L1_VLRLIQ" , oTotal:GetValue("L1_VLRTOT"))
25 STFSetTot( "L1_VALBRUT" , oTotal:GetValue("L1_VALMERC") - oTotal:GetValue("L1_DESCONT") )
26
27 If lDesTot
28     STFSetTot("L1_DESCFIN", oTotal:GetValue("L1_DESCFIN"))
29
30     nL1_descto := STDGPBasket("SL1","L1_DESCTOT")
31     STDSPBasket("SL1","L1_DESCTOT", IIf( nL1_descto > 0 , nL1_descto, nDescTot))
32
33     /* L1_DESCNF - Não altera a referência de desconto L1_DESCNF pois o desconto
34        inserido aqui será o da Condição de Pagamento na função STBApliDes*/
35 Else
36     /*Porcentagem de Desconto no Total*/
37     STFSetTot( "L1_DESCNF" , STBDiscConvert(oTotal:GetValue("L1_DESCONT"), "V", oTotal:GetValue("L1_VALBRUT"))[2] )
38 EndIf
39
40 /*/
41     Atualiza Cesta
42 /*/
43 STDSPBasket( "SL1" , "L1_VALMERC" , oTotal:GetValue( "L1_VALMERC" ) )
44 STDSPBasket( "SL1" , "L1_VLRTOT" , oTotal:GetValue( "L1_VLRTOT" ) )
45 STDSPBasket( "SL1" , "L1_DESCONT" , oTotal:GetValue( "L1_DESCONT" ) )
46 STDSPBasket( "SL1" , "L1_DESCNF" , oTotal:GetValue( "L1_DESCNF" ) )
47 STDSPBasket( "SL1" , "L1_VLRLIQ" , oTotal:GetValue( "L1_VLRLIQ" ) )
48 STDSPBasket( "SL1" , "L1_VALBRUT" , oTotal:GetValue( "L1_VALBRUT" ) )
49 STDSPBasket( "SL1" , "L1_JUROS" , oTotal:GetValue( "L1_JUROS" ) )
50 STDSPBasket( "SL1" , "L1_BONIF" , oTotal:GetValue( "L1_BONIF" ) )
51
52 If lDesTot

```

```
53     STDSPBasket(      "SL1"      ,      "L1_DESCFIN"      ,      oTotal:GetValue( "L1_DESCFIN"      )      )  
54 EndIf  
55  
56 Return Nil
```